
Documentação de Projeto

para o sistema

MobU

Versão 1.0

Projeto de sistema elaborado pelo(s) aluno(s) Joaquim de Moura Thomaz Neto e apresentado ao curso de **Engenharia de Software** da **PUC Minas** como parte do Trabalho de Conclusão de Curso (TCC) sob orientação de conteúdo dos professores Danilo de Quadros Maia Filho, Leonardo Vilela Cardoso, Raphael Ramos Dias Costa, orientação acadêmica do professor Cleiton Silva Tavares e orientação de TCC II do professor (a ser definido no próximo semestre).

16/11/2025

Tabela de Conteúdo

Tabela de Conteúdo	ii
Histórico de Revisões	ii
1. Introdução	1
2. Modelos de Usuário e Requisitos	2
2.1 Descrição de Atores	2
2.2 Modelos de Usuários	3
2.3 Modelo de Casos de Uso e Histórias de Usuários	5
2.3.1 Diagrama de Casos de Uso	5
2.3.2 Histórias de Usuário	8
2.4 Diagrama de Sequência do Sistema e Contrato de Operações	10
2.5 Requisitos Não Funcionais	21
3. Modelo de Projeto	22
3.1 Diagrama de Classes	22
3.2 Diagramas de Sequência	26
3.3 Diagramas de Comunicação	34
3.4 Arquitetura	38
3.5 Diagramas de Estados	40
3.6 Diagrama de Componentes e Implantação	45
4. Projeto de Interface com Usuário	48
4.1 Interfaces Comuns	48
4.2 Módulo do Administrador	49
4.3 Módulo do Motorista	52
4.4 Módulo do Passageiro	55
5. Glossário e Modelos de Dados	58
6. Casos de Teste	79
6.1 Testes de Aceitação	80
6.1.1 Necessidade 1: Passageiro solicitar corrida	80
6.1.2 Necessidade 2 — Motorista aceitar e realizar corrida	81
6.1.3 Necessidade 3 — Pagamento via PIX Off-line	83
6.1.4 Necessidade 4 — Administração de Usuários e Monitoramento do Sistema	84
6.2 Testes de Integração	86
6.2.1 Necessidade 5 — Cadastro e Aprovação de Motoristas	92
6.2.2 Necessidade 6 — Recusa de Corrida pelo Motorista	94
6.2.3 Novos Casos de Teste de Integração	94
6.3 Testes Unitários	97
6.3.1 Necessidade — Integridade do Cadastro e Autenticação	97
6.3.2 Necessidade — Controle de Status e Aceite de Corrida pelo Motorista	99
6.3.3 Necessidade — Resiliência da Estimativa de Corrida	100
7. Cronograma e Processo de Implementação	103
7.1 Cronograma	104
8. Trabalhos Futuros	109
8.1 Integração com Gateway de Pagamento PIX em Tempo Real	110
8.2 Publicação e Distribuição do Aplicativo Android	110
8.3 Evolução Arquitetural do Aplicativo Android	111
8.4 Desenvolvimento de Aplicativo iOS	111
8.5 Verificação Automatizada de Documentos de Motoristas	111
8.6 Infraestrutura de Produção e Implantação Contínua	112

8.7	Escalabilidade e Resiliência	112
8.8	Monitoramento, Observabilidade e Alertas	113
8.9	Funcionalidades Operacionais Avançadas	113
8.10	Expansão para Múltiplas Cidades e Modelo Multi-Tenant	113
8.11	Testes, Qualidade e Cobertura	114
9.	Post-mortem	115
9.1	Experiências Positivas	115
9.2	Experiências Negativas	115
9.3	Lições Aprendidas	115
10.	Repositório do Trabalho	116

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Entrega 3	17/09/2025	Início do documento, definindo seções 2.1, 2.2, 2.3 e 4	1.0
Entrega 4	06/10/2025	Inclusão das seções 2.4, 3.1, 3.2 e 3.3	2.0
Entrega 5	31/10/2025	Inclusão das seções 3.4, 3.5, 3.6 e 5	3.0
Entrega 6	12/11/2025	Inclusão da seção 6, 7 e 7.1	4.0
Entrega 7	25/11/2025	Inclusão da seção 6.1, 6.1.2, 6.1.3, 6.1.4 e 6.2	5.0
Entrega 8	20/05/2026	Atualização documentação	6.0
Entrega 9	10/06/2026	Atualização final documentação	7.0

1. Introdução

A mobilidade urbana em cidades de pequeno e médio porte apresenta desafios relacionados à disponibilidade de motoristas, previsibilidade de preço, segurança no acompanhamento das corridas e organização operacional do serviço. Embora aplicativos de transporte sejam amplamente utilizados em grandes centros, soluções locais costumam apresentar limitações de usabilidade, baixa transparência sobre ganhos e tarifas, pouca integração entre passageiro e motorista e ausência de ferramentas administrativas para monitoramento do negócio. Nesse contexto, identifica-se a necessidade de uma plataforma digital que ofereça uma experiência simples para usuários finais e, ao mesmo tempo, forneça recursos de gestão para a operação do serviço.

Este trabalho apresenta o projeto e o desenvolvimento do MobU, uma solução de mobilidade urbana composta por aplicativo *mobile* Android, API *backend*, banco de dados relacional e painel web administrativo. A aplicação desenvolvida permite que passageiros criem conta, solicitem corridas, visualizem estimativas de tarifa, acompanhem o motorista em tempo real e realizem pagamentos em dinheiro ou por Pix *off-line*. Para motoristas, o sistema oferece cadastro, controle de disponibilidade, aceite e execução de corridas, histórico de viagens e acompanhamento financeiro. Para administradores, o painel web permite gerenciar usuários, motoristas, regiões de operação, relatórios, taxas e informações operacionais da plataforma.

O objetivo geral deste trabalho é especificar, implementar e validar uma plataforma de mobilidade urbana capaz de conectar passageiros, motoristas e administradores em um fluxo integrado de solicitação, execução, pagamento e acompanhamento de corridas. A proposta busca resolver o problema de fragmentação e baixa rastreabilidade observado em soluções locais, oferecendo uma base tecnológica que combine aplicativo móvel, serviços de *backend*, persistência de dados, integração com mapas, notificações e recursos administrativos.

Como objetivos específicos, o trabalho busca: levantar e documentar os atores, necessidades e histórias de usuário do sistema; modelar os principais casos de uso, fluxos, entidades, componentes e estados da aplicação; projetar interfaces adequadas para passageiros, motoristas e administradores; implementar os módulos essenciais do ecossistema MobU; integrar funcionalidades como autenticação, estimativa de corrida, acompanhamento, Pix *off-line*, histórico e relatórios; e validar o comportamento do sistema por meio de testes de aceitação, integração, *build*, *lint* e execução em emuladores Android.

Este documento está organizado da seguinte forma: a Seção 2 apresenta o modelo de requisitos, incluindo atores, personas, casos de uso e histórias de usuário que fundamentam as funcionalidades do sistema. A Seção 3 descreve o modelo de projeto, contemplando diagramas de classes, sequência, comunicação, arquitetura, estados, componentes e implantação. A Seção 4 apresenta o projeto de interface com o usuário para os diferentes perfis da aplicação. A Seção 5 detalha o glossário e o modelo de dados, incluindo o diagrama entidade-relacionamento. A Seção 6 descreve os casos de teste de aceitação e integração, bem como os resultados obtidos na validação do sistema. Por fim, a Seção 7 apresenta o cronograma e o processo de implementação adotado no desenvolvimento do MobU.

Dessa forma, a documentação consolida a visão funcional, técnica e operacional do sistema desenvolvido, garantindo rastreabilidade entre problema, requisitos, projeto, implementação e testes. Além de servir como registro acadêmico do Trabalho de Conclusão de Curso, o documento também funciona como base de manutenção e evolução futura da plataforma MobU.

O sistema MobU enquadra-se nos objetivos de um Trabalho de Conclusão de Curso em Engenharia de *Software* por envolver, de forma integrada, atividades de levantamento e análise de requisitos, modelagem de domínio, definição arquitetural, projeto de interface, implementação de componentes de *software* e validação por meio de testes. A solução desenvolvida não se limita à construção de uma única tela ou funcionalidade isolada, mas compreende um ecossistema composto por aplicativo *mobile* Android, *API backend*, banco de dados relacional, painel web administrativo e integrações com serviços externos, como mapas, geolocalização, notificações e pagamento Pix *off-line*.

Dessa forma, o trabalho contempla práticas fundamentais da Engenharia de *Software*, como rastreabilidade entre necessidades, requisitos, casos de uso, modelos de projeto, persistência de dados e testes. O desenvolvimento do MobU permite demonstrar a aplicação prática de conceitos estudados ao longo do curso, incluindo arquitetura cliente-servidor, modelagem orientada a objetos, persistência, autenticação, integração entre sistemas, projeto de interfaces e validação funcional.

2. Modelos de Usuário e Requisitos

Esta seção tem como propósito caracterizar os usuários e atores envolvidos no sistema MobU, além de apresentar os requisitos que orientam sua implementação. Para isso, são descritos os principais perfis de usuários, suas necessidades, dores e objetivos, permitindo relacionar o problema identificado às funcionalidades propostas.

A partir dessa caracterização, são apresentados o diagrama de casos de uso e as histórias de usuário, que representam as principais interações esperadas entre passageiros, motoristas, administradores e o sistema. As funcionalidades levantadas foram organizadas de forma a manter rastreabilidade entre necessidades, casos de uso, histórias de usuário e testes, permitindo verificar se cada necessidade relevante do *software* foi contemplada pelo projeto.

As histórias de usuário foram elaboradas buscando atender às características *INVEST*, ou seja, devem ser independentes, negociáveis, agregar valor ao usuário, ser estimáveis, sucintas e testáveis. Dessa forma, cada requisito procura representar uma necessidade clara do sistema, evitando ambiguidades e permitindo sua validação por meio dos casos de teste apresentados na Seção 6.

2.1 Descrição de Atores

Passageiro: Usuário que utiliza o aplicativo para solicitar corridas. É responsável por informar origem e destino, acompanhar o trajeto em tempo real, realizar o pagamento e avaliar o motorista ao

final da viagem. Seu principal objetivo é realizar deslocamentos com praticidade, segurança e previsibilidade de custo.

Motorista: Usuário que utiliza o aplicativo para oferecer o serviço de transporte. Recebe solicitações de corridas, navega até o ponto de embarque e conduz o passageiro ao destino informado. Pode consultar seu histórico de corridas e acompanhar seus ganhos. É responsável por manter informações atualizadas no sistema e aceitar ou recusar corridas conforme disponibilidade.

Administrador: Responsável por gerenciar o funcionamento da plataforma por meio de um sistema web. Controla o cadastro de passageiros e motoristas, monitora as corridas realizadas, acompanha indicadores de utilização e gera relatórios financeiros. Também pode configurar taxas da plataforma e atuar em situações que demandem suporte ou resolução de conflitos.

2.2 Modelos de Usuários

Esta subseção tem como objetivo apresentar os modelos de usuários construídos a partir da definição de personas. Para a elaboração dessas personas, foram consideradas as características dos principais atores do sistema como passageiros, motoristas e administradores, bem como suas necessidades, dores e objetivos dentro do contexto da aplicação. As personas a seguir representam perfis típicos dos usuários previstos para a plataforma e servem como referência para orientar decisões de *design*, desenvolvimento e priorização de funcionalidades do sistema.

A Tabela 1 descreve a persona da usuária Mariana Silva, uma passageira que depende do transporte urbano para deslocamentos diários. É possível identificar que, apesar de estar familiarizada com aplicativos de uso comum como *WhatsApp* e *Instagram*, ela sente dificuldades em confiar nos aplicativos locais de transporte por considerá-los confusos e pouco práticos. Também é perceptível que Mariana valoriza a previsibilidade no preço e segurança durante as viagens. Por fim, após análise, observa-se que ela deseja ter maior clareza no custo antes da corrida e a possibilidade de acompanhar o trajeto em tempo real, o que lhe traria mais confiança no serviço.

Mariana Silva	
Descrição	Mariana possui 27 anos, nasceu e cresceu na cidade onde o aplicativo será implantado. Trabalha como recepcionista em uma clínica e depende de transporte diário para se deslocar ao trabalho e para compromissos pessoais. Costuma usar aplicativos como <i>WhatsApp</i> e <i>Instagram</i> , mas nunca utilizou os aplicativos de transporte já existentes na cidade, pois os considera confusos e pouco confiáveis. Busca uma alternativa prática que lhe permita se deslocar com previsibilidade de preço e segurança.
Dores	• Dificuldade em prever o valor de cada corrida com os aplicativos atuais. • Insegurança por não poder acompanhar a rota em tempo real.

	● Pouca confiança em aplicativos locais, pela falta de avaliação de motoristas.
Objetivos	● Ter acesso a corridas rápidas, seguras e acessíveis. ● Saber o valor estimado da corrida antes de embarcar. ● Acompanhar em tempo real o trajeto, sentindo-se mais segura.

Tabela 1. Persona Mariana Silva

A Tabela 2 descreve a persona do usuário Carlos Andrade, um motorista que utiliza o transporte por aplicativo como fonte de renda complementar. Identifica-se que ele já teve experiências anteriores com aplicativos locais, mas considera que eles carecem de recursos de apoio ao motorista e transparência em relação aos ganhos. Nota-se também que Carlos valoriza ferramentas que o ajudam a organizar seu trabalho de forma prática, sem exigir aprendizado complexo de novas tecnologias. Por fim, após análise, percebe-se que ele busca um sistema confiável que facilite o recebimento de corridas e forneça relatórios claros de viagens e rendimentos.

Carlos Andrade	
Descrição	Carlos tem 42 anos e trabalha como motorista de aplicativo em tempo parcial para complementar a renda da família. Já testou os aplicativos locais (LevaAli e MobyGo), mas considera que eles oferecem pouco suporte e não têm transparência nos ganhos. Ele valoriza poder organizar suas corridas e acompanhar de forma simples quanto está ganhando por dia e por semana. Carlos utiliza apenas o necessário em termos de tecnologia, mas aprende rápido quando percebe que o recurso facilita sua rotina.
Dores	● Falta de relatórios claros sobre corridas e ganhos nos aplicativos atuais. ● Necessidade de depender de chamadas aleatórias, sem controle. ● Dificuldade em manter contato com suporte ou resolver problemas por falta de recursos internos no <i>app</i>.
Objetivos	● Receber chamadas de corridas de forma simples e rápida. ● Acompanhar histórico de viagens e rendimento financeiro. ● Trabalhar em um sistema confiável que ofereça suporte adequado.

Tabela 2. Persona Carlos Andrade

A Tabela 3 descreve a persona da usuária Fernanda Oliveira, que atua como administradora da plataforma. É possível observar que ela precisa equilibrar a gestão do sistema com suas outras responsabilidades profissionais, o que exige relatórios padronizados e práticos para tomada de decisão. Identifica-se ainda que Fernanda encontra dificuldades em monitorar o desempenho da plataforma com os recursos limitados oferecidos pelos aplicativos concorrentes. Por fim, após análise, verifica-se que sua principal necessidade é dispor de ferramentas que permitam acompanhar a satisfação dos usuários, controlar cadastros e configurar taxas de forma objetiva.

Fernanda Oliveira	
Descrição	Fernanda possui 35 anos, formada em Administração e atua como secretária de gestor da plataforma. Seu papel é garantir que motoristas e passageiros estejam devidamente cadastrados e que o sistema funcione de forma estável. Ela precisa ter acesso a relatórios que mostrem número de corridas realizadas, ganhos, taxas e indicadores de uso do sistema, de modo a tomar decisões sobre melhorias e acompanhar o crescimento do aplicativo.
Dores	• Falta de relatórios padronizados nos aplicativos locais. • Dificuldade em monitorar qualidade do serviço (avaliações e <i>feedbacks</i>). • Necessidade de ajustar taxas e políticas de forma manual.
Objetivos	• Ter acesso a relatórios claros e detalhados sobre o funcionamento do sistema. • Acompanhar o desempenho de motoristas e satisfação de passageiros. • Configurar taxas e gerenciar usuários de forma prática.

Tabela 3. Persona Fernanda Oliveira

2.3 Modelo de Casos de Uso e Histórias de Usuários

Esta subseção apresenta os casos de uso e as histórias de usuário que orientam o desenvolvimento do sistema MobU. O diagrama de casos de uso representa as principais interações entre passageiros, motoristas, administradores e serviços externos, evidenciando como cada ator participa dos fluxos essenciais da aplicação.

As histórias de usuário complementam essa visão ao descrever, em linguagem simples e orientada a valor, as funcionalidades esperadas para cada perfil. Elas permitem relacionar as necessidades levantadas no Documento de Visão com funcionalidades implementáveis e testáveis, favorecendo a priorização, a estimativa de esforço e a validação do sistema.

2.3.1 Diagrama de Casos de Uso

A Figura 1 apresenta o diagrama de casos de uso do sistema proposto. No diagrama, são representados três atores principais: Passageiro, Motorista e Administrador, que interagem diretamente com o sistema. Além deles, são exibidos dois sistemas externos: o Serviço de Mapas e Geolocalização, utilizado para cálculo de rotas e estimativas de tempo. O passageiro acessa o sistema para criar conta, solicitar corridas, acompanhar o trajeto, efetuar pagamento e avaliar o motorista. O Motorista utiliza o sistema para receber corridas, aceitar ou recusar solicitações,

navegar até o passageiro e ao destino, marcar etapas da viagem e consultar relatórios de ganhos. O Administrador gerencia usuários, motoristas, taxas, regiões de operação, relatórios do sistema e solicitações de suporte. Ela representa visualmente essas interações, bem como as relações entre os casos de uso e os sistemas externos.

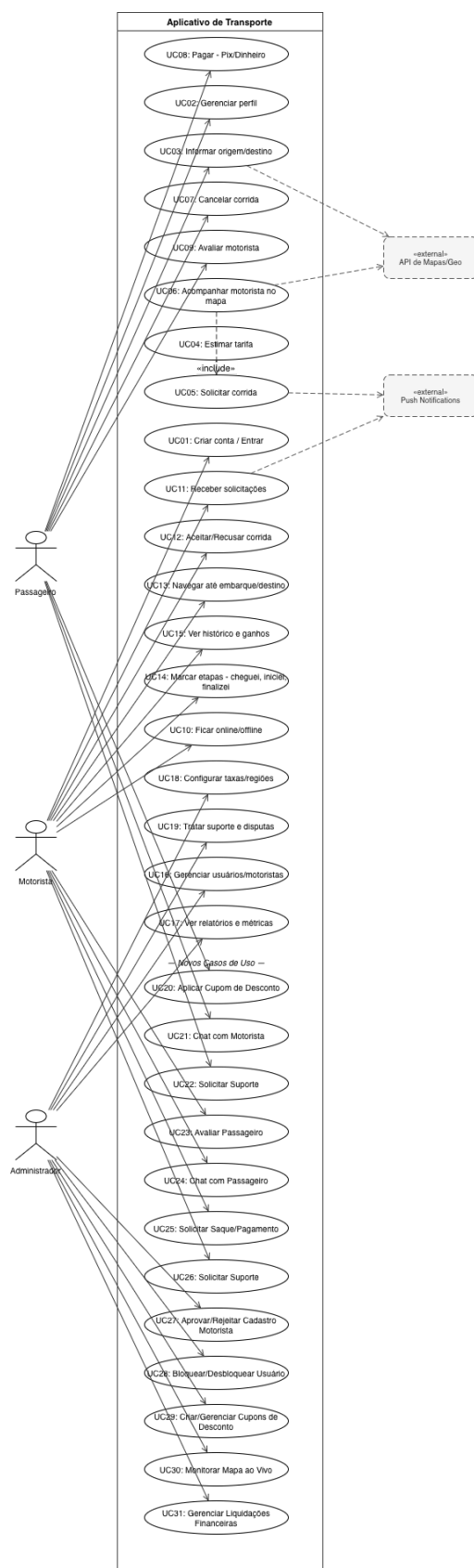


Figura 1 – Diagrama de Casos de Uso

2.3.2 Histórias de Usuário

Nesta seção, são listadas as histórias de usuário levantadas para o sistema proposto. Para fins de organização, utiliza-se identificadores no formato *US#ID*, em que *US* se refere a *User Story*.

As histórias de usuário identificadas para o sistema são:

US1. Como passageiro, eu gostaria de criar uma conta e gerenciar meu perfil para que eu possa acessar o sistema e utilizar seus serviços.

US2. Como passageiro, eu gostaria de informar origem e destino para que eu possa solicitar uma corrida.

US3. Como passageiro, eu gostaria de ver a estimativa de tarifa e tempo de chegada do motorista para decidir se quero confirmar a corrida.

US4. Como passageiro, eu gostaria de acompanhar a rota do motorista em tempo real para saber onde ele está.

US5. Como passageiro, eu gostaria de efetuar o pagamento via PIX ou em dinheiro para concluir a corrida com segurança.

US6. Como passageiro, eu gostaria de avaliar o motorista após a corrida para contribuir com a reputação da plataforma.

US7. Como motorista, eu gostaria de entrar no sistema e definir meu *status* como *online* ou *offline* para indicar quando estou disponível para receber corridas.

US8. Como motorista, eu gostaria de receber solicitações de corrida com informações de origem, destino e valor estimado para decidir se aceito a viagem.

US9. Como motorista, eu gostaria de navegar até o passageiro e depois até o destino usando o mapa do *app* para otimizar a rota.

US10. Como motorista, eu gostaria de marcar as etapas da corrida (cheguei, iniciei, finalizei) para registrar corretamente o andamento da viagem.

US11. Como motorista, eu gostaria de acessar um painel com meus ganhos diários e semanais para acompanhar meus resultados.

US12. Como administrador, eu gostaria de gerenciar contas de passageiros e motoristas para manter o sistema organizado e seguro.

US13. Como administrador, eu gostaria de acompanhar relatórios de corridas e métricas do sistema para tomar decisões estratégicas.

US14. Como administrador, eu gostaria de configurar taxas e regiões de operação para ajustar o funcionamento da plataforma conforme a cidade.

US15. Como administrador, eu gostaria de visualizar e responder solicitações de suporte ou disputas para resolver problemas de forma rápida.

US16. Como passageiro, eu gostaria de conversar com o motorista por meio do *chat* do aplicativo durante a corrida para me comunicar de forma prática sem precisar ligar.

US17. Como passageiro, eu gostaria de aplicar um cupom de desconto ao solicitar uma corrida para obter um valor reduzido na tarifa.

US18. Como passageiro, eu gostaria de abrir um *ticket* de suporte para relatar problemas ou tirar dúvidas com a equipe da plataforma.

US19. Como motorista, eu gostaria de avaliar o passageiro ao final da corrida para contribuir com a qualidade e segurança da plataforma.

US20. Como motorista, eu gostaria de conversar com o passageiro por meio do *chat* do aplicativo durante a corrida para me comunicar de forma prática sem precisar ligar.

US21. Como motorista, eu gostaria de solicitar o pagamento dos meus ganhos acumulados para receber minha remuneração de forma prática.

US22. Como motorista, eu gostaria de visualizar meus ciclos de faturamento semanais para acompanhar detalhadamente minhas corridas e valores a receber.

US23. Como motorista, eu gostaria de enviar meu cadastro com documentos (CNH, foto do veículo e *selfie*) para ser aprovado e começar a operar na plataforma.

US24. Como administrador, eu gostaria de aprovar ou rejeitar cadastros de motoristas com base nos documentos enviados para garantir a qualidade dos prestadores de serviço.

US25. Como administrador, eu gostaria de bloquear ou desbloquear contas de usuários para proteger a plataforma contra comportamentos inadequados.

US26. Como administrador, eu gostaria de criar e gerenciar cupons de desconto para incentivar o uso da plataforma.

US27. Como administrador, eu gostaria de monitorar as operações em tempo real por meio de um mapa ao vivo para acompanhar motoristas e corridas ativas.

US28. Como administrador, eu gostaria de registrar e gerenciar liquidações financeiras dos motoristas para manter o controle dos repasses da plataforma.

2.4 Diagrama de Sequência do Sistema e Contrato de Operações

Nesta seção, são apresentados os diagramas de sequência do sistema, assim como seus Contratos de Operações. O objetivo destes diagramas é descrever os fluxos de interação existentes entre os usuários e o sistema, representando a troca de mensagens e as dependências entre módulos da aplicação.

A Figura 2 representa o diagrama de sequência do sistema relacionado ao fluxo de solicitação de corrida. Nesse fluxo, o Passageiro informa sua origem e destino e, ao confirmar o pedido, o sistema consulta o Serviço de Mapas para calcular a rota e o tempo estimado.

Após essa etapa, a aplicação envia uma notificação em tempo real ao Motorista disponível mais próximo, que poderá aceitar ou recusar a corrida.

Esse diagrama está diretamente relacionado aos casos de uso UC03 (Informar origem/destino), UC04 (Estimar tarifa), UC05 (Solicitar corrida) e UC11 (Receber solicitação).

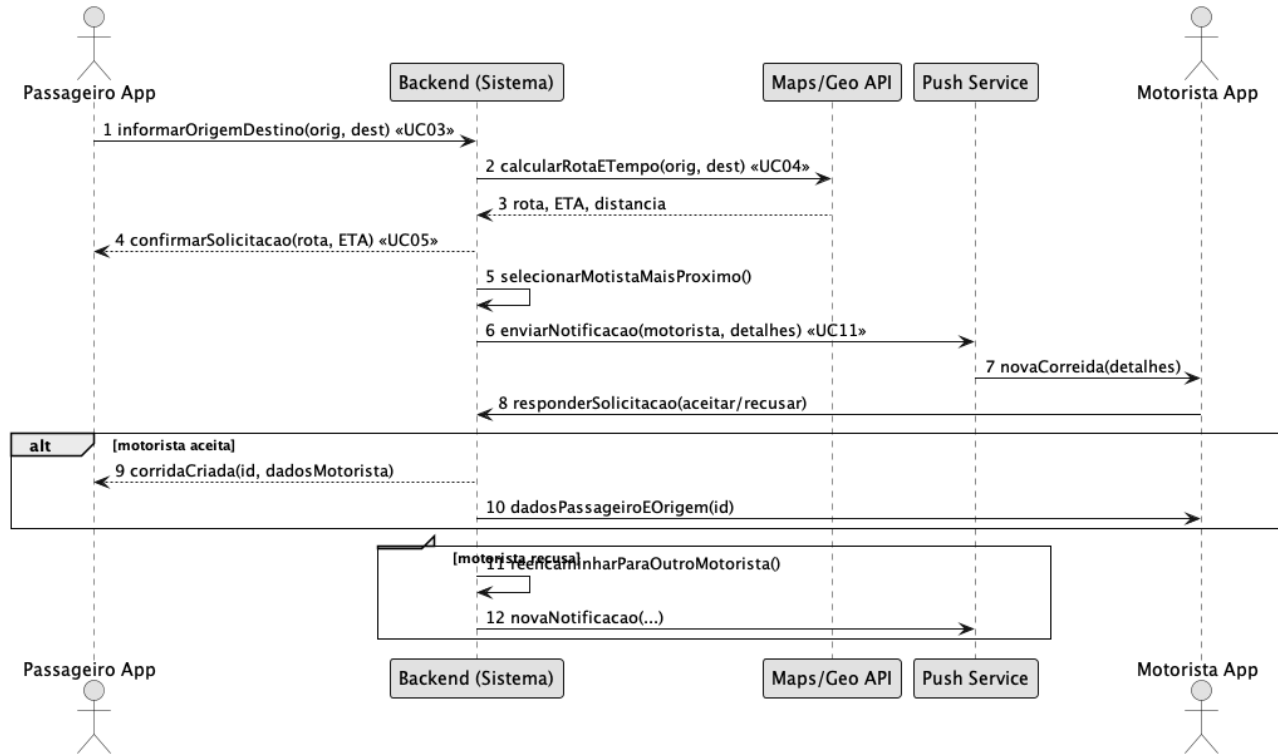


Figura 2 – Diagrama de Sequência Solicitar Corrida e Notificar Motorista

Contrato	Solicitar corrida e notificar motorista
Operação	<i>requestRide(origin: String, destination: String)</i>
Referências cruzadas	UC03 Informar origem/destino, UC04 Estimar tarifa, UC05 Solicitar corrida, UC11 Receber solicitação
Pré-condições	O passageiro deve estar autenticado no sistema e com geolocalização ativa
Pós-condições	O motorista mais próximo é notificado e a corrida é registrada no banco de dados como “pendente”

Tabela 4. Contrato Solicitar Corrida e Notificar Motorista

A Figura 3 mostra o diagrama de sequência referente ao fluxo de pagamento da corrida. Após o término da viagem, o sistema apresenta as opções de PIX ou Dinheiro. No processo de pagamento em dinheiro ou pix, o sistema atualiza o *status* localmente. Esse fluxo está relacionado aos casos de uso UC08 (Efetuar pagamento) e UC14 (Finalizar corrida).

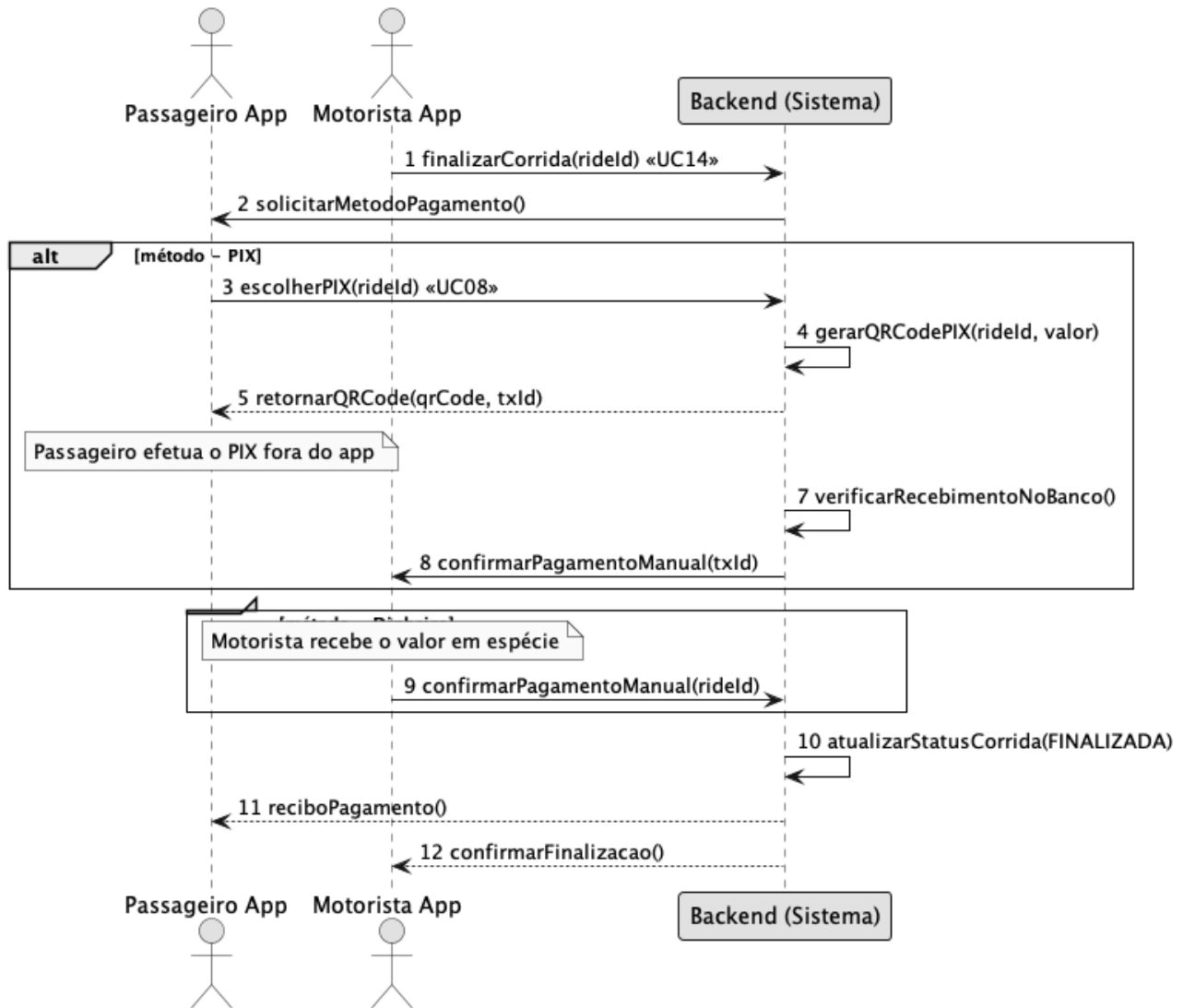


Figura 3 – Diagrama de Sequência Efetuar Pagamento

Contrato	Efetuar pagamento da corrida
Operação	<i>processPayment(paymentMethod: Enum, rideId: Int)</i>
Referências cruzadas	UC08 Efetuar pagamento, UC14 Finalizar corrida
Pré-condições	A corrida deve ter sido concluída e estar aguardando pagamento
Pós-condições	O pagamento é confirmado (PIX) ou marcado como concluído (dinheiro), alterando o <i>status</i> da corrida para “finalizada”

Tabela 5. Contrato Efetuar Pagamento da Corrida

A Figura 4 representa o diagrama de sequência do fluxo de avaliação do motorista. Após o encerramento da corrida e o processamento do pagamento, o Passageiro pode registrar uma avaliação que inclui nota e comentário. O sistema valida os dados e salva a avaliação, atualizando a média do motorista no banco de dados. Esse fluxo está associado ao caso de uso UC09 (Avaliar motorista).

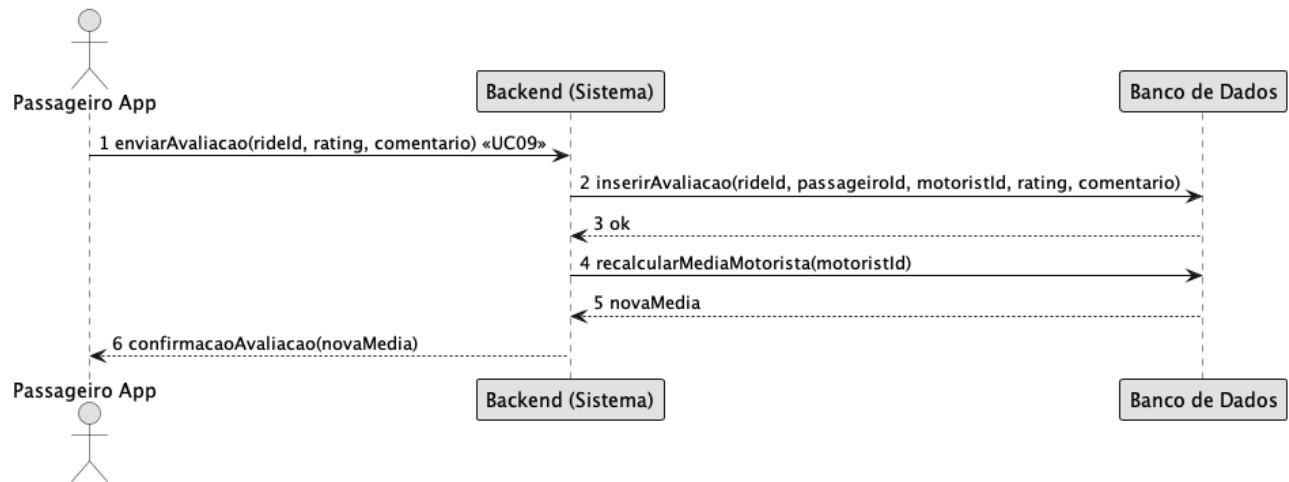


Figura 4 – Diagrama de Sequência Avaliação do Motorista

Contrato	Registrar avaliação do motorista
Operação	<i>submitReview(rideId: Int, rating: Int, comment: String)</i>
Referências cruzadas	UC09 Avaliar motorista
Pré-condições	A corrida deve ter sido finalizada e o pagamento concluído
Pós-condições	A avaliação é registrada e a nota média do motorista é atualizada no sistema

Tabela 6. Contrato Registrar Avaliação do Motorista

A Figura 5 ilustra o diagrama de sequência do painel administrativo, que demonstra como o Administrador acessa o sistema web, solicita relatórios e recebe informações agregadas de corridas, motoristas e usuários. O fluxo inclui a requisição dos dados, o processamento interno e a resposta exibida em forma de gráficos e métricas. Esse diagrama se relaciona aos casos de uso UC16 (Gerenciar usuários/motoristas), UC17 (Ver relatórios e métricas) e UC18 (Configurar taxas/regiões).

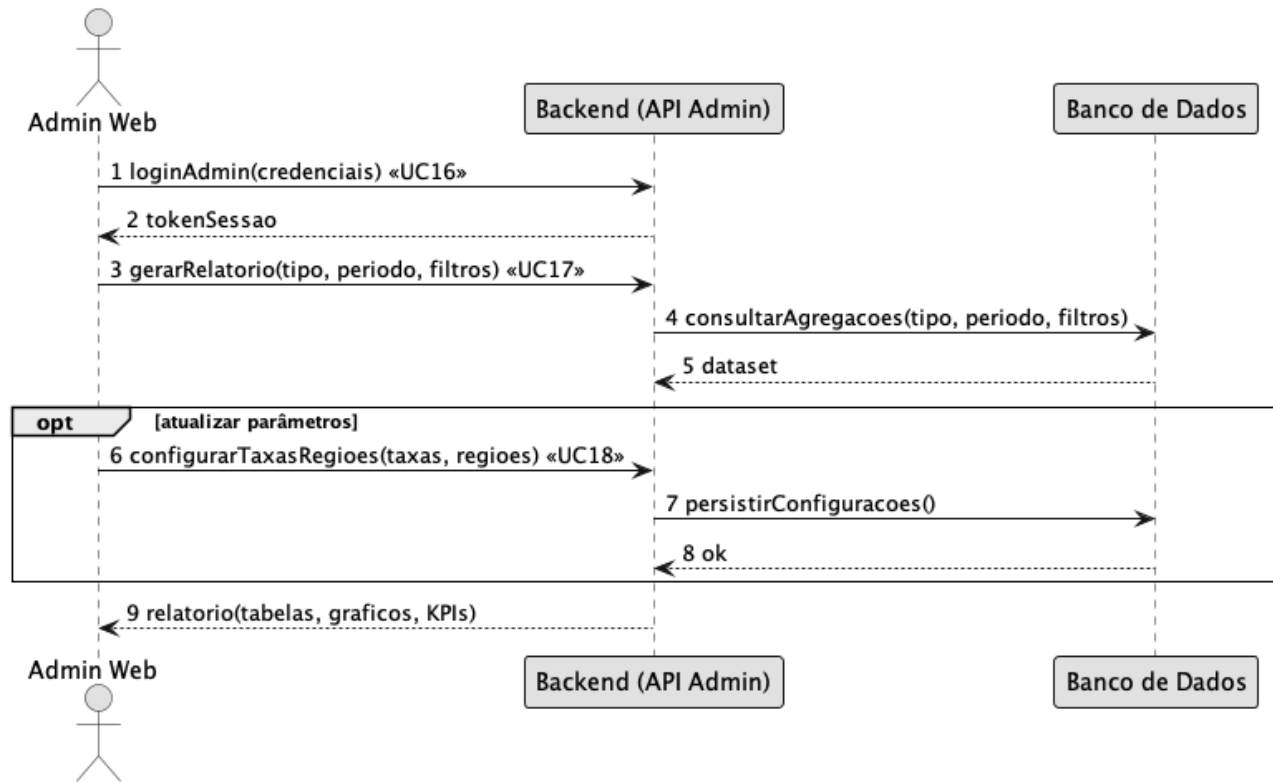


Figura 5 – Diagrama de Sequência Gerar Relatórios Administrativos

Contrato	Gerar relatórios administrativos
Operação	<i>generateReport(type: Enum, dateRange: String)</i>
Referências cruzadas	UC16 Gerenciar usuários, UC17 Ver relatórios, UC18 Configurar taxas
Pré-condições	O administrador deve estar autenticado no painel web
Pós-condições	O relatório solicitado é gerado e apresentado na interface em formato gráfico

Tabela 7. Contrato Gerar Relatórios Administrativos

A Figura 6 representa o diagrama de sequência do fluxo de *chat* em tempo real entre passageiro e motorista. Durante uma corrida ativa, ambos os usuários podem trocar mensagens por meio do *chat* do aplicativo. As mensagens são persistidas no banco de dados e entregues ao destinatário por meio de conexão *WebSocket*. Esse diagrama se relaciona aos casos de uso UC21 (*Chat* com motorista) e UC24 (*Chat* com passageiro).

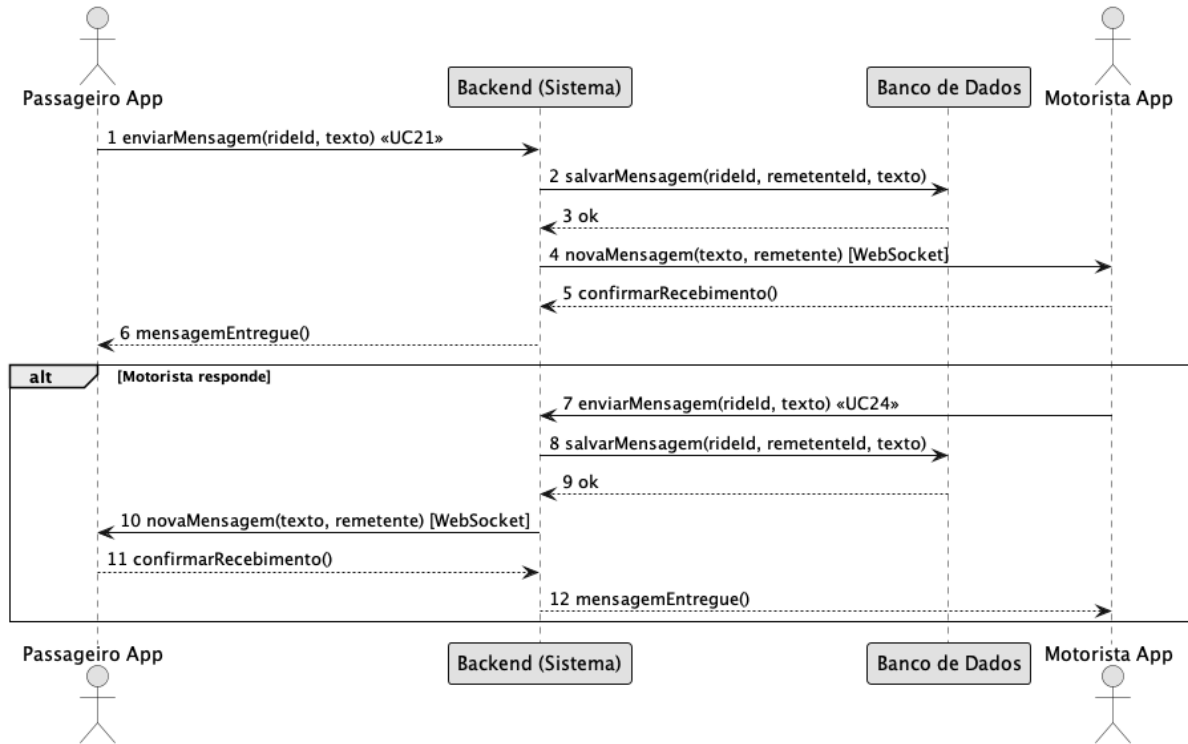


Figura 6 – Diagrama de Sequência Chat em Tempo Real

Contrato	Chat em tempo real entre passageiro e motorista
Operação	<i>sendMessage(rideId: Int, text: String): void</i>
Referências cruzadas	UC21 Chat com motorista, UC24 Chat com passageiro
Pré-condições	A corrida deve estar em andamento e ambos os usuários autenticados no sistema
Pós-condições	A mensagem é salva no banco de dados e entregue ao destinatário via WebSocket

Tabela 8. Contrato Chat em Tempo Real

A Figura 7 ilustra o diagrama de sequência do fluxo de cadastro e aprovação de motorista. O motorista submete seus dados e documentos pelo aplicativo. O administrador revisa as informações no painel web e decide pela aprovação ou rejeição, sendo o motorista notificado em ambos os casos. Esse fluxo se relaciona aos casos de uso UC23 (Registrar cadastro com documentos) e UC27 (Aprovar/Rejeitar cadastro de motorista).

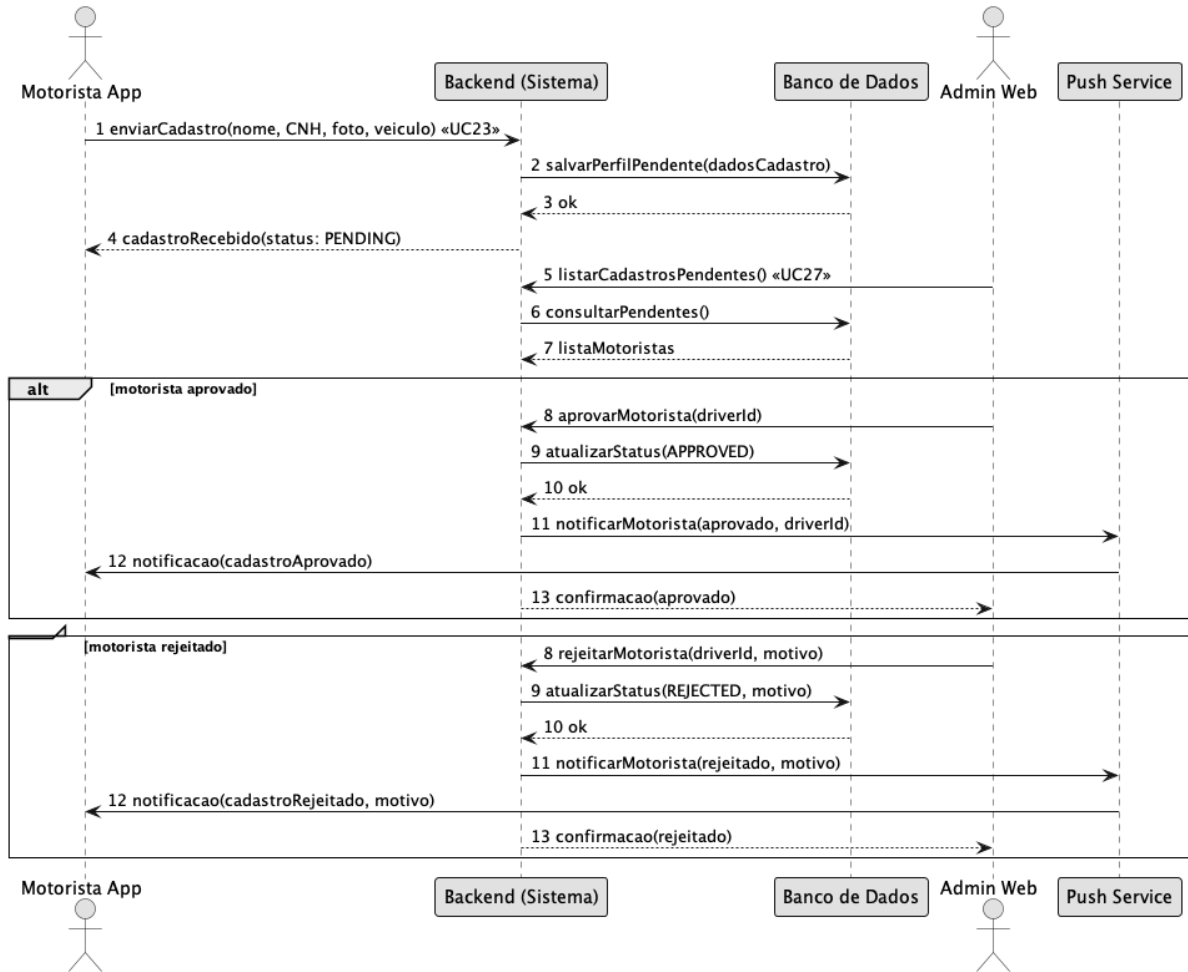


Figura 7 – Diagrama de Sequência Aprovação de Cadastro do Motorista

Contrato	Aprovar ou rejeitar cadastro de motorista
Operação	<i>approveDriver(driverId: Int): void / rejectDriver(driverId: Int, reason: String): void</i>
Referências cruzadas	UC23 Registrar cadastro com documentos, UC27 Aprovar/Rejeitar cadastro de motorista
Pré-condições	O motorista deve ter submetido o cadastro com todos os documentos obrigatórios
Pós-condições	O <i>status</i> do cadastro é atualizado para APROVADO ou REJEITADO e o motorista é notificado

Tabela 9. Contrato Aprovação de Cadastro do Motorista

A Figura 8 apresenta o diagrama de sequência do fluxo de aplicação de cupom de desconto. Antes de confirmar a solicitação de corrida, o passageiro pode informar um código promocional. O sistema valida o cupom junto ao banco de dados, verifica sua validade e usos restantes, e retorna o valor com o desconto aplicado. Esse diagrama está relacionado aos casos de uso UC20 (Aplicar cupom de desconto) e UC05 (Solicitar corrida).

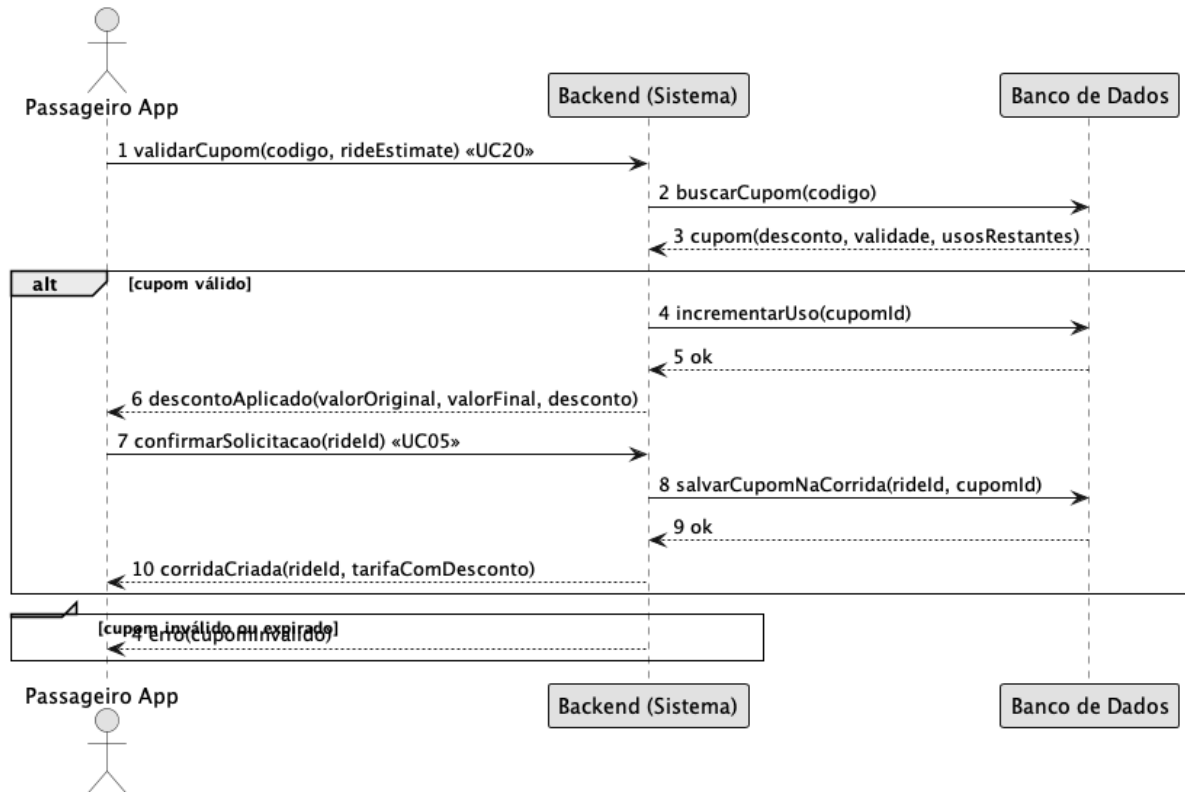


Figura 8 – Diagrama de Sequência Aplicar Cupom de Desconto

Contrato	Aplicar cupom de desconto na corrida
Operação	<i>validateCoupon(code: String, rideEstimate: Float): DiscountResult</i>
Referências cruzadas	UC20 Aplicar cupom de desconto, UC05 Solicitar corrida
Pré-condições	O passageiro deve estar autenticado e a estimativa de corrida deve ter sido calculada
Pós-condições	O desconto é aplicado à tarifa e o uso do cupom é registrado no banco de dados

Tabela 10. Contrato Aplicar Cupom de Desconto

A Figura 9 mostra o diagrama de sequência do fluxo de avaliação do passageiro pelo motorista. Após o encerramento da corrida, o motorista pode registrar uma avaliação com nota e comentário. O sistema valida a corrida, persiste a avaliação e recalcula a média do passageiro. Esse fluxo está associado ao caso de uso UC19 (Avaliar passageiro).

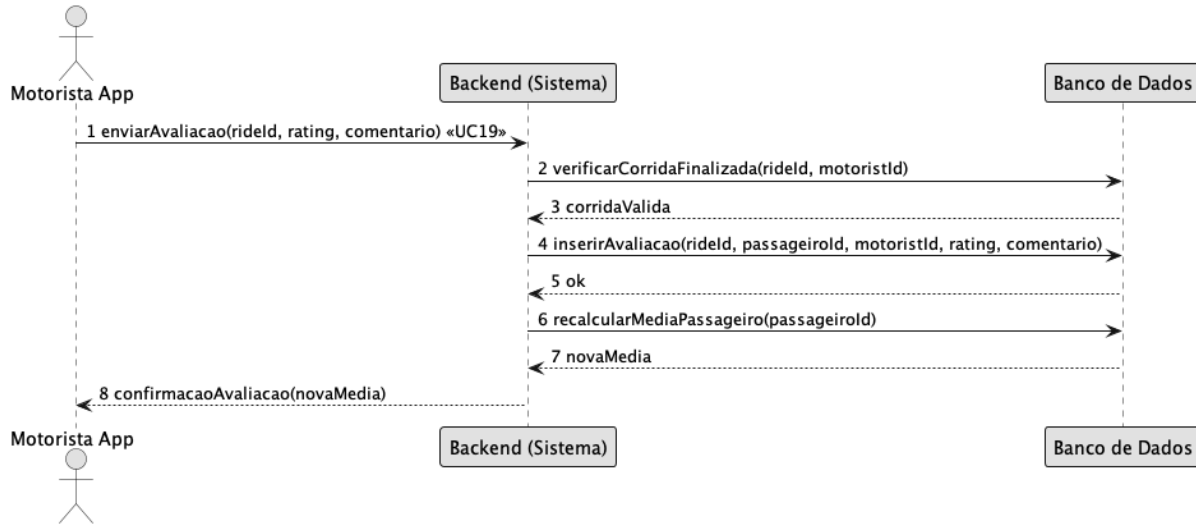


Figura 9 – Diagrama de Sequência Avaliação do Passageiro

Contrato	Registrar avaliação do passageiro
Operação	<i>submitPassengerReview(rideId: Int, rating: Int, comment: String): void</i>
Referências cruzadas	UC19 Avaliar passageiro
Pré-condições	A corrida deve ter sido finalizada e o pagamento concluído
Pós-condições	A avaliação é registrada e a nota média do passageiro é atualizada no sistema

Tabela 11. Contrato Registrar Avaliação do Passageiro

A Figura 10 representa o diagrama de sequência do fluxo de solicitação de saque do motorista. O motorista solicita o pagamento de seus ganhos acumulados informando o valor e a chave PIX. O sistema verifica o saldo disponível e, caso suficiente, registra a solicitação. O administrador posteriormente confirma o pagamento e o sistema registra a liquidação financeira. Esse fluxo está relacionado aos casos de uso UC25 (Solicitar saque/pagamento) e UC31 (Gerenciar liquidações financeiras).

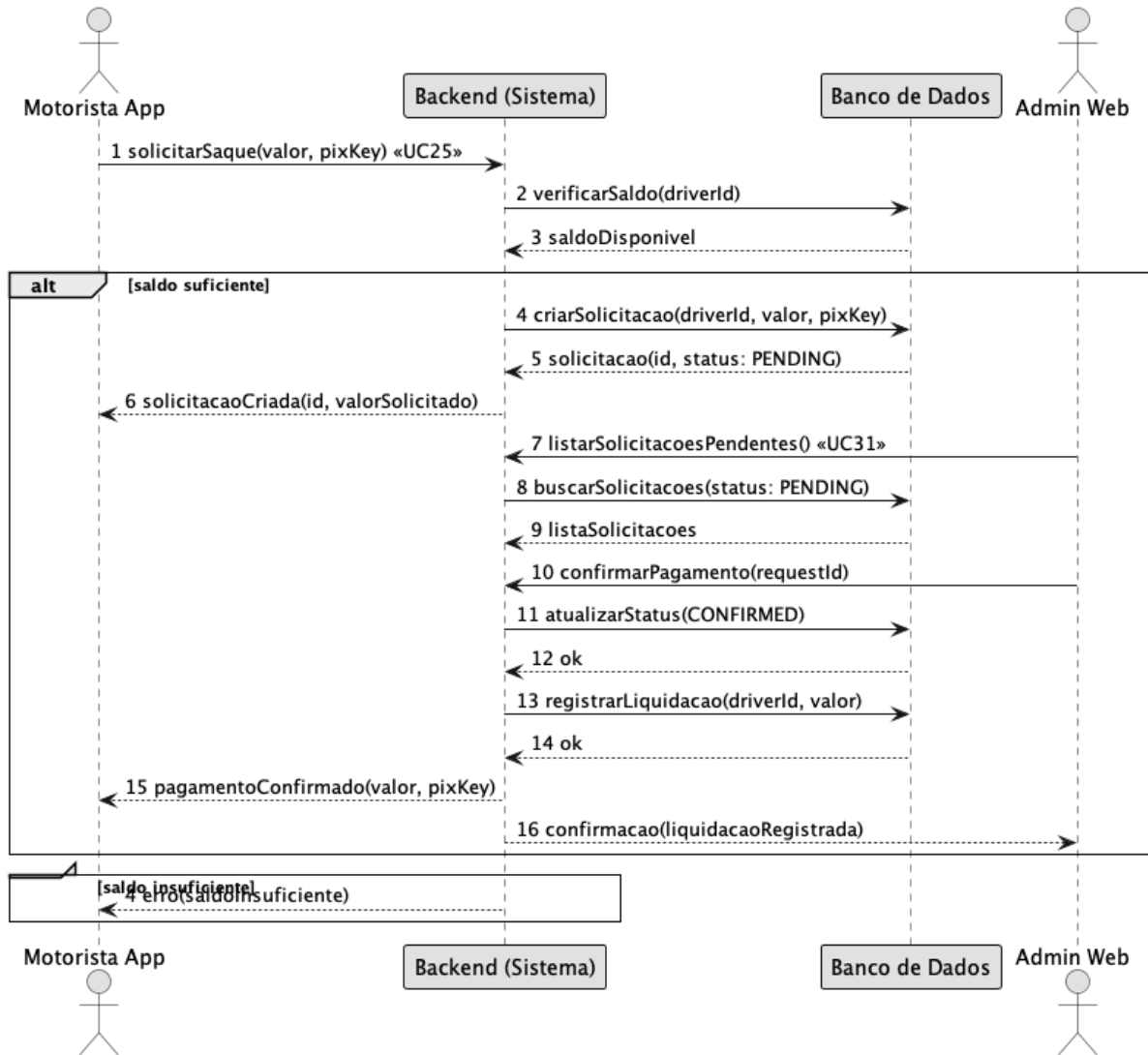


Figura 10 – Diagrama de Sequência Solicitação de Saque do Motorista

Contrato	Solicitar e confirmar saque do motorista
Operação	<i>createPaymentRequest(driverId: Int, amount: Float, pixKey: String): PaymentRequest</i>
Referências cruzadas	UC25 Solicitar saque/pagamento, UC31 Gerenciar liquidações financeiras
Pré-condições	O motorista deve estar autenticado e possuir saldo disponível superior ao valor solicitado

Pós-condições	A solicitação é registrada com <i>status</i> PENDENTE e, após confirmação do admin, a liquidação é persistida
----------------------	---

Tabela 12. Contrato Solicitação de Saque do Motorista

A Figura 11 ilustra o diagrama de sequência do fluxo de bloqueio e desbloqueio de usuário pelo administrador. Ao bloquear um motorista, o sistema também força seu *status* para *offline* e indisponível, impedindo que receba novas corridas. O fluxo contempla também o desbloqueio da conta. Esse diagrama está relacionado ao caso de uso UC28 (Bloquear/Desbloquear usuário).

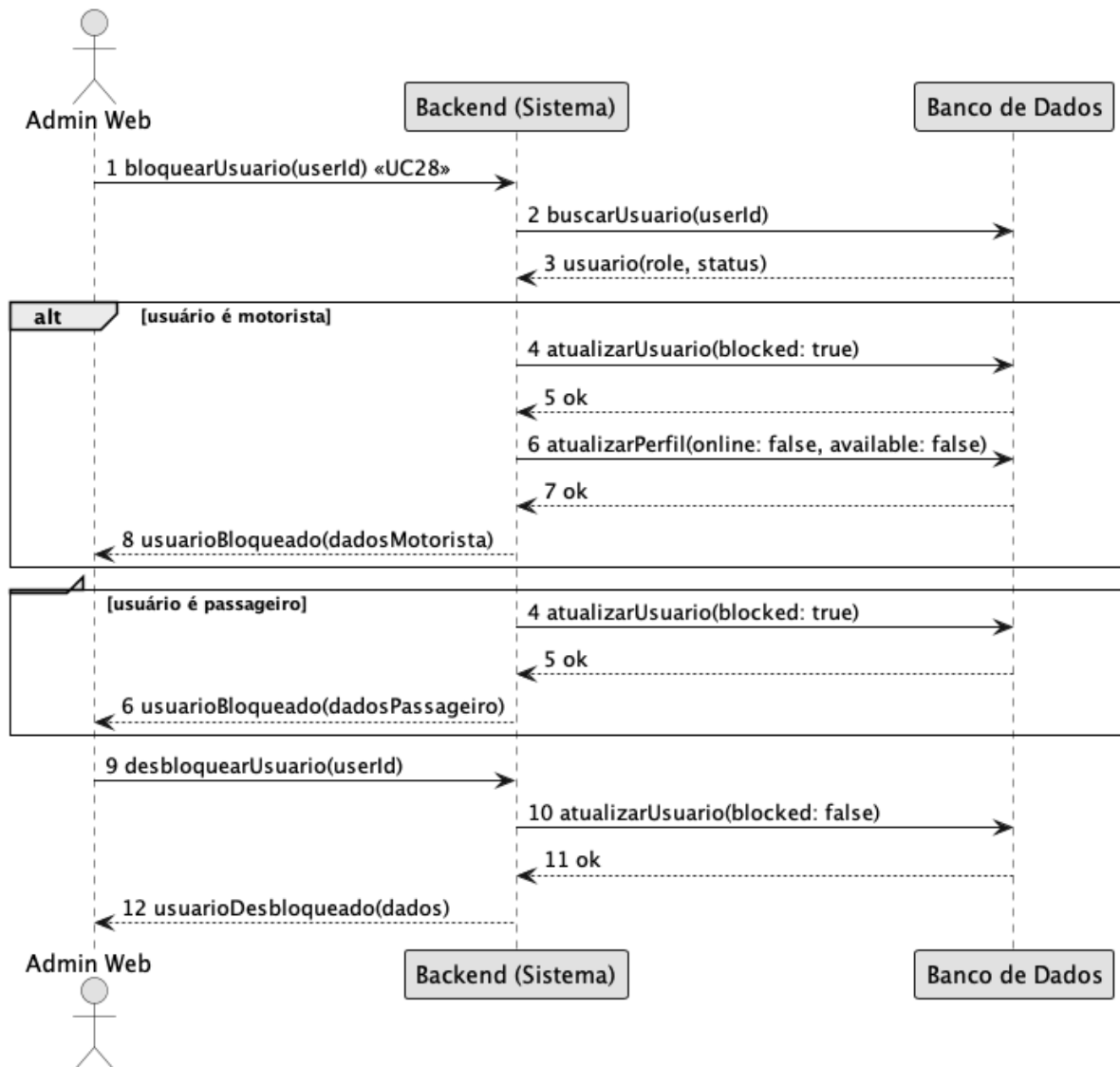


Figura 11 – Diagrama de Sequência Bloquear/Desbloquear Usuário

Contrato	Bloquear ou desbloquear conta de usuário
Operação	<i>blockUser(userId: Int): User / unblockUser(userId: Int): User</i>
Referências cruzadas	UC28 Bloquear/Desbloquear usuário, UC16 Gerenciar usuários/motoristas
Pré-condições	O administrador deve estar autenticado e o usuário-alvo não pode ser outro administrador
Pós-condições	A conta do usuário é bloqueada ou desbloqueada; se motorista, seu <i>status online</i> é redefinido para false

Tabela 13. Contrato Bloquear/Desbloquear Usuário

2.5 Requisitos Não Funcionais

Os requisitos não funcionais do sistema MobU definem as restrições de qualidade, desempenho e operação que devem ser observadas durante o desenvolvimento e a implantação da plataforma.

RNF-01 — Desempenho: a API deve responder a requisições de estimativa e criação de corrida em no máximo 3 segundos em condições normais de operação.

RNF-02 — Segurança: todas as senhas devem ser armazenadas com *hash BCrypt*; todas as comunicações entre cliente e servidor devem ocorrer via *HTTPS*; o acesso à API deve ser autenticado via *JWT* com expiração de 7 dias.

RNF-03 — Compatibilidade: o aplicativo Android deve ser compatível com dispositivos executando Android 8.0 (API 26) ou superior.

RNF-04 — Portabilidade: o *backend* e o banco de dados devem ser executáveis em qualquer ambiente com suporte a *Docker*, sem dependência de configuração manual do sistema operacional hospedeiro.

RNF-05 — Manutenibilidade: o esquema do banco de dados deve ser gerenciado exclusivamente via *Prisma Migrations*, garantindo rastreabilidade de todas as alterações estruturais ao longo do desenvolvimento.

RNF-06 — Escalabilidade: a arquitetura modular do *NestJS* deve permitir a adição de novos módulos de negócio sem refatoração da estrutura base existente.

RNF-07 — Rastreabilidade: toda alteração de estado de uma corrida deve ser persistida com *timestamp* no banco de dados, permitindo auditoria do ciclo de vida de cada solicitação.

RNF-08 — Disponibilidade: o sistema deve permanecer disponível durante o horário de operação definido pelo administrador, com recuperação automática de falhas garantida pela política de reinicialização dos containers Docker.

3. Modelo de Projeto

Esta seção apresenta os modelos de projeto utilizados para detalhar a estrutura e o comportamento do sistema MobU. Os diagramas descritos permitem transformar os requisitos levantados anteriormente em uma visão técnica da solução, apoiando a implementação dos principais fluxos da aplicação.

São apresentados modelos estruturais e comportamentais, incluindo diagrama de classes, diagramas de sequência, diagramas de comunicação, arquitetura lógica, diagrama de estados, diagrama de componentes e modelo de implantação. Esses modelos foram elaborados de forma coerente com os casos de uso e com o modelo de dados, permitindo compreender como as entidades, componentes e usuários interagem para realizar as funcionalidades propostas.

3.1 Diagrama de Classes

O diagrama de classes representa a estrutura principal do domínio do sistema MobU, contemplando as entidades responsáveis pelos fluxos de cadastro, solicitação de corrida, execução da viagem, pagamento, avaliação e administração. A modelagem considera os principais atores do sistema, como passageiro, motorista e administrador, além das entidades necessárias para registrar corridas, pagamentos, regiões, históricos e informações operacionais.

A entidade relacionada à corrida ocupa papel central no modelo, pois conecta passageiro, motorista, localização, *status* da viagem e dados de pagamento. Essa estrutura permite representar adequadamente os casos de uso levantados e mantém coerência com o Diagrama de Entidade e Relacionamento apresentado posteriormente. Dessa forma, o diagrama de classes favorece a implementação do sistema e contribui para a manutenção e evolução da solução.

A Figura 12 ilustra o diagrama de classes do pacote *controllers*, evidenciando as dependências diretas entre os controladores e os serviços correspondentes que contêm as regras de negócio. É possível observar a relação entre *PassageiroController* e *CorridaService*, *MotoristaController* e *RelatorioService*, entre outras.

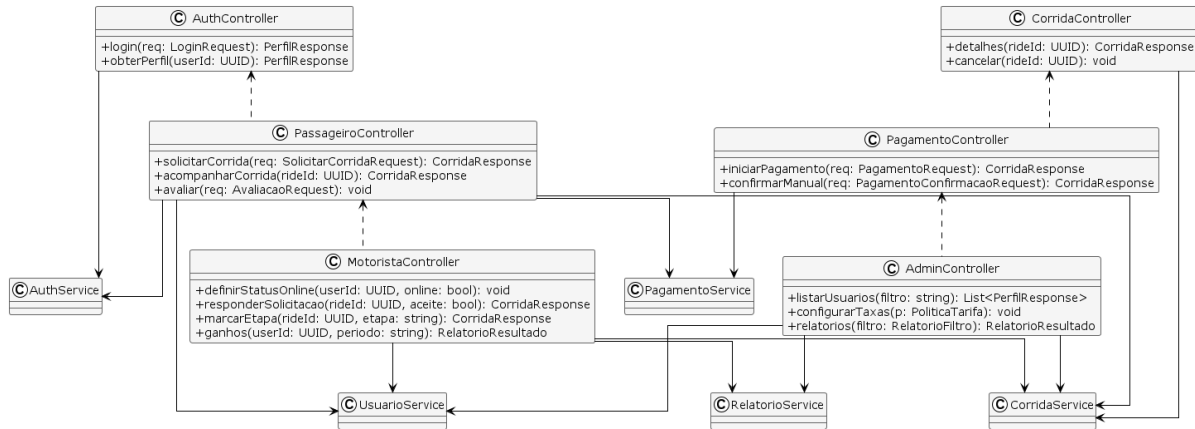


Figura 12 – Diagrama de Classes: Controllers

O diagrama de classes apresentado a seguir representa o pacote de *services*, responsável pela camada de regras de negócio do aplicativo.

Essas classes implementam as operações centrais da aplicação, como o gerenciamento de corridas, cálculo de tarifas, pareamento entre motoristas e passageiros, processamento de pagamentos e geração de relatórios.

O pacote inclui serviços de autenticação (*AuthService*), gestão de usuários (*UsuarioService*), cálculo de preço dinâmico (*PrecificacaoService*), correspondência de motoristas (*MatchingService*), gerenciamento de *status* (*MotoristaStatusService*), controle de corridas (*CorridaService*), pagamentos (*PagamentoService*), notificações (*NotificacaoService*), *chat* em tempo real (*ChatService*), suporte ao usuário (*SupportService*) e avaliação de passageiros (*ReviewService*).

As Figuras 13 e 14 apresentam o diagrama de classes dos *services*, mostrando as dependências com os repositórios de persistência e com integrações externas como *APIs* de mapas e serviços de *push*.

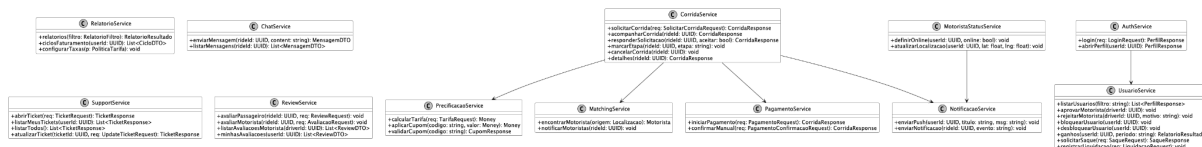


Figura 13 – Diagrama de Classes: Services01



Figura 14 – Diagrama de Classes: Services02

O diagrama de classes a seguir descreve o pacote domain, que contém os modelos de dados fundamentais do sistema. Essas classes representam as entidades centrais que sustentam o

funcionamento do aplicativo, como *Usuario*, *Passageiro*, *Motorista*, *Veiculo*, *Corrida*, *Pagamento* e *Avaliacao*.

Cada entidade foi estruturada com seus respectivos atributos e relacionamentos, garantindo coerência com os processos do negócio. Por exemplo, a classe *Corrida* mantém vínculos diretos com *Passageiro*, *Motorista* e *Rota*, além de conter informações de *status*, valores e horários. A classe *Pagamento* está associada à *Corrida*, representando tanto pagamentos via PIX quanto em dinheiro, ambos processados manualmente no aplicativo. Além disso, o modelo inclui entidades auxiliares como *Localizacao*, *PoliticaTarifa*, *Regiao* e *Money*, usadas para padronizar endereços, políticas de preço e valores monetários.

A Figura 15 apresenta o diagrama de classes do domínio, permitindo visualizar de forma clara as relações entre as entidades e seus papéis dentro do fluxo geral da aplicação.

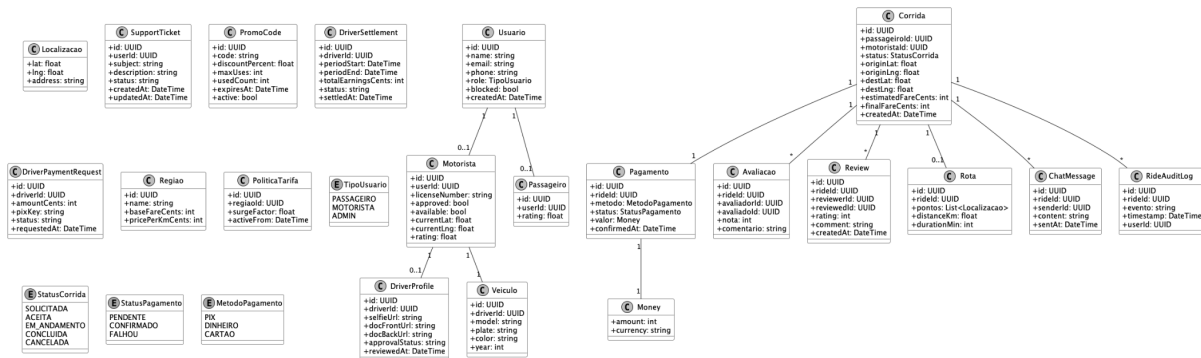


Figura 15 – Diagrama de Classes: Domain / Models

O diagrama de classes a seguir representa o pacote de *repositories*, responsável pela camada de persistência de dados. Essas classes são definidas como interfaces, pois a implementação pode variar de acordo com a tecnologia de banco de dados adotada, mantendo assim o princípio da abstração e separação de responsabilidades.

Entre os repositórios estão *UsuarioRepository*, *CorridaRepository*, *PagamentoRepository*, *AvaliacaoRepository*, *ChatMessageRepository*, *SupportTicketRepository*, *ReviewRepository*, *PromoCodeRepository* e *DriverSettlementRepository*, que lidam com as operações de CRUD e consultas filtradas.

A Figura 16 exibe o diagrama de classes de *repositories*, demonstrando a estrutura modular de acesso aos dados e sua conexão com a camada de *services*.

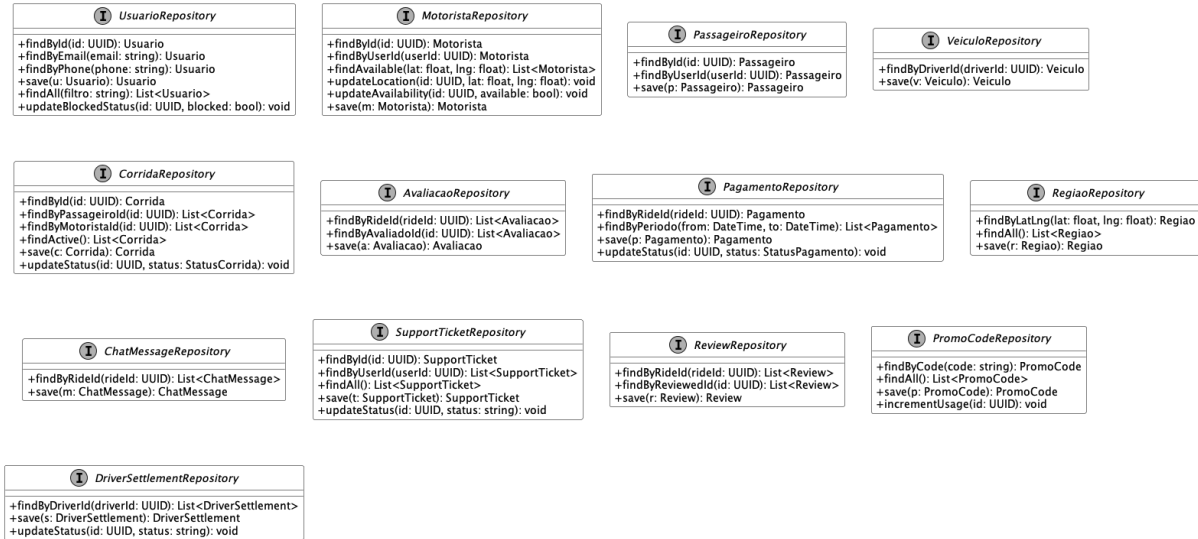


Figura 16 – Diagrama de Classes: Repositories

Por fim, a figura a seguir apresenta o diagrama de classes referente aos *Data Transfer Objects* (DTOs), que têm como função padronizar a comunicação entre o cliente (aplicativo móvel) e o servidor (API). Esses objetos simplificam o envio e recebimento de dados, evitando o tráfego de entidades completas e protegendo a estrutura interna do sistema.

Também foram definidos *ChatMensagemDTO*, *TicketRequest*, *TicketResponse*, *ReviewRequest*, *ReviewDTO*, *CupomRequest*, *CupomResponse*, *SaqueRequest*, *SaqueResponse* e *LiquidacaoRequest*, relacionados às funcionalidades de *chat*, suporte, avaliação de passageiros e gestão financeira de motoristas.

A Figura 17 apresenta o diagrama de classes dos DTOs, mostrando a utilização de tipos de domínio como *Localizacao*, *Rota* e *Money* para garantir consistência entre os dados de transporte e pagamento.

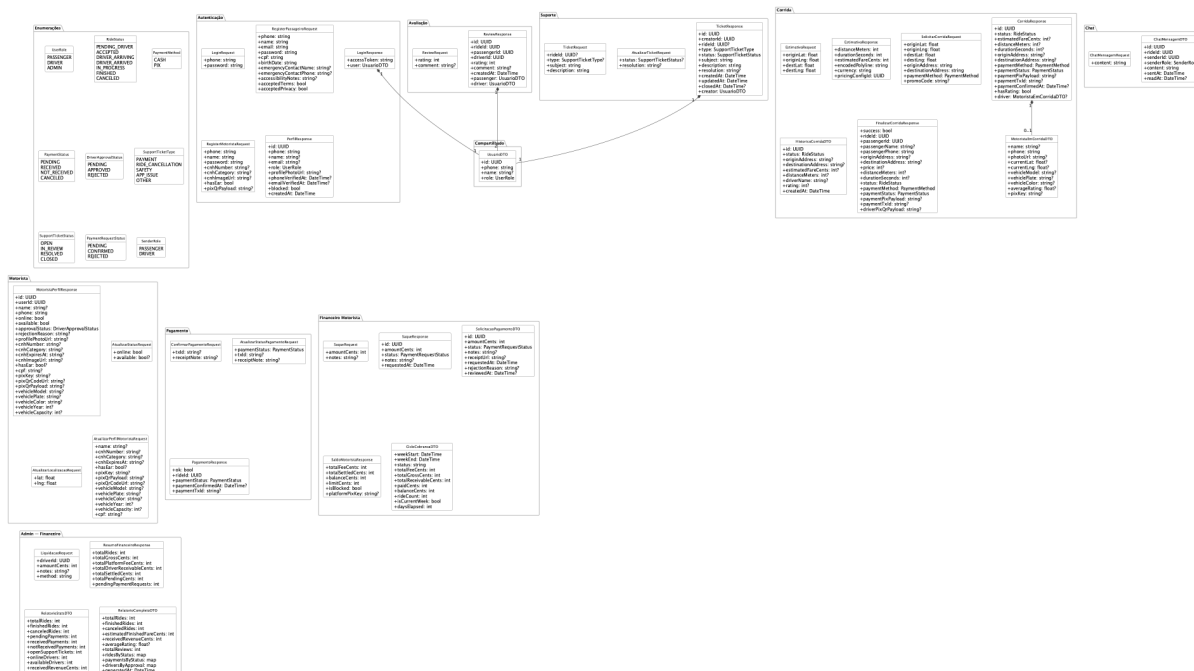


Figura 17 – Diagrama de Classes: DTOs

3.2 Diagramas de Sequência

Nesta seção, são apresentados os diagramas de sequência que modelam os fluxos principais do sistema de transporte proposto. Esses diagramas têm como objetivo mapear os caminhos críticos de uso da aplicação, evidenciando quem se comunica com quem, quais mensagens são trocadas e em que ordem.

Foram selecionados dez fluxos representativos: solicitação de corrida, encerramento e pagamento (PIX/dinheiro, confirmação manual), avaliação do motorista, painel administrativo, *chat* em tempo real, aprovação de cadastro de motorista, validação de cupom de desconto, avaliação de passageiro, solicitação de saque e bloqueio de usuário. Juntos, eles cobrem as interações essenciais entre Passageiro, Motorista e Administrador, além das integrações necessárias com serviços externos.

A Figura 18 representa o fluxo de solicitação de corrida. O Passageiro informa origem e destino; o sistema consulta o serviço de mapas para obter rota, distância e tempo estimado; em seguida, o *MatchingService* identifica o motorista disponível mais próximo e envia uma notificação via *Push Service*. O motorista pode aceitar ou recusar. Em caso de recusa, a solicitação é reencaminhada a outro motorista. Este fluxo está relacionado aos casos de uso UC03 (Informar origem/destino), UC04 (Estimar tarifa), UC05 (Solicitar corrida) e UC11 (Receber solicitação).

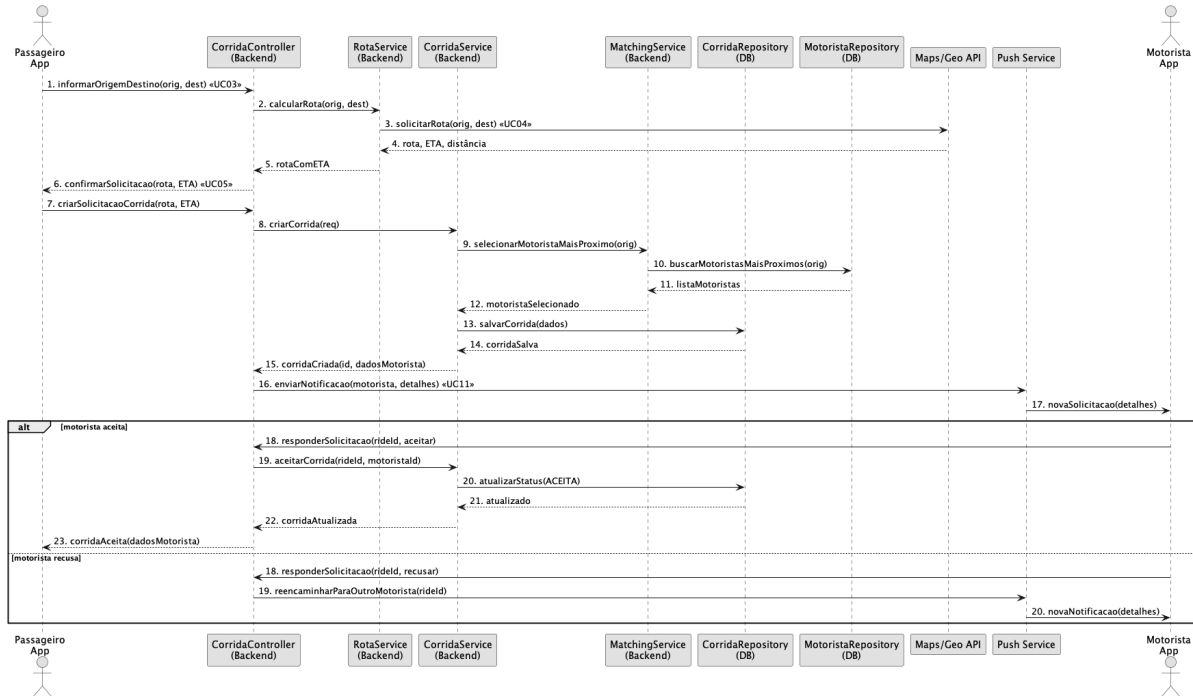


Figura 18 – Diagrama de Sequência Solicitação de Corrida

A Figura 19 descreve o encerramento da corrida e o pagamento. O sistema apresenta os métodos PIX e Dinheiro. Para PIX, o *backend* gera um *QR Code* internamente (sem integração bancária automática); o motorista verifica manualmente no *app* do banco se o valor entrou e confirma no sistema. Para Dinheiro, o motorista confirma manualmente o recebimento. Em ambos os casos, a corrida passa a Finalizada e recibos são disponibilizados. Este fluxo cobre UC08 (Efetuar pagamento) e UC14 (Finalizar corrida).

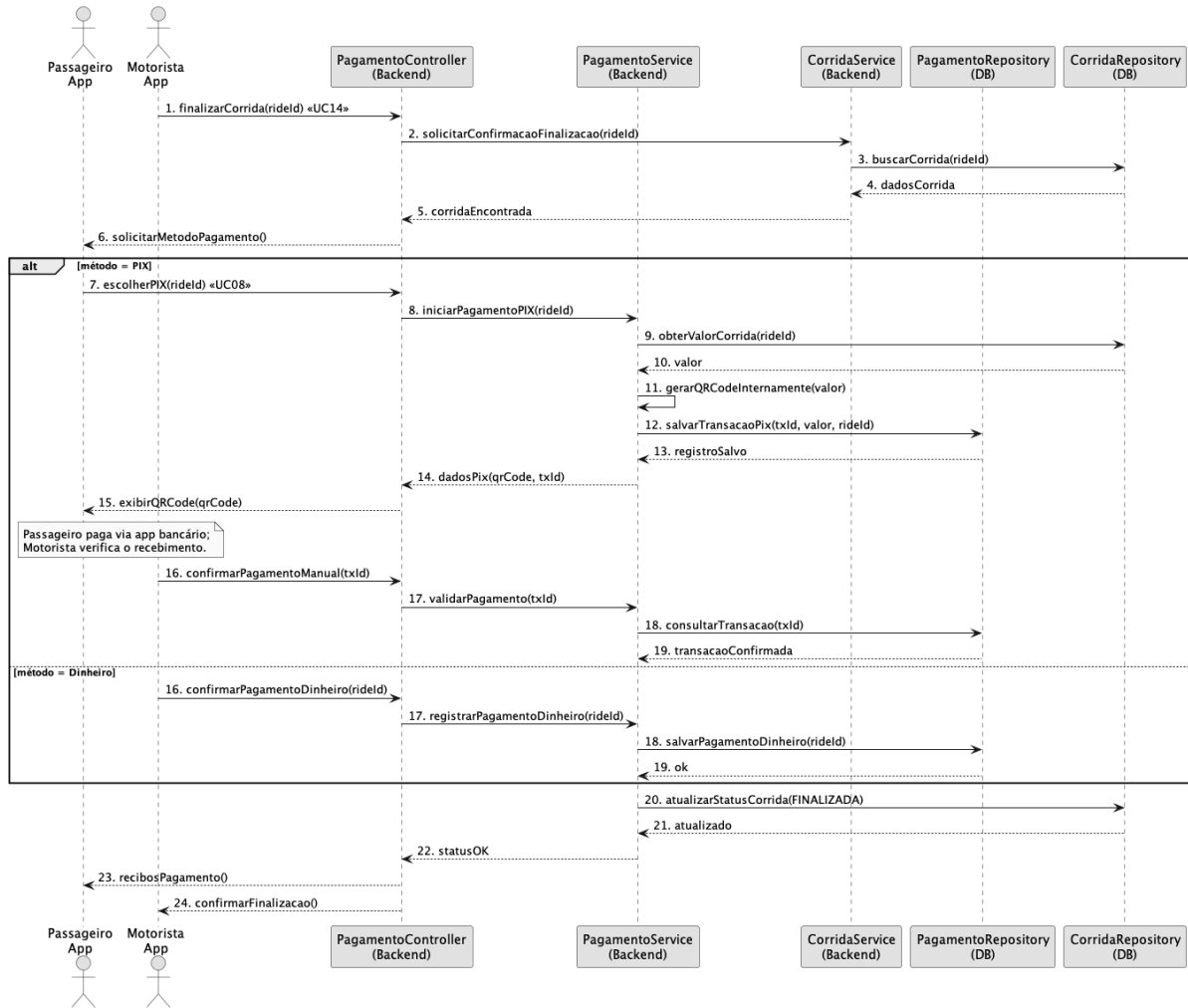


Figura 19 – Diagrama de Sequência Encerramento da Corrida e Pagamento

A Figura 20 mostra o fluxo de avaliação do motorista pelo Passageiro após o pagamento. O cliente informa nota e comentário; o sistema armazena a avaliação e recalcula a média do motorista. Esse processo retroalimenta a qualidade do serviço e influencia futuras seleções de motoristas. Relaciona-se ao caso de uso UC09 (Avaliar motorista).

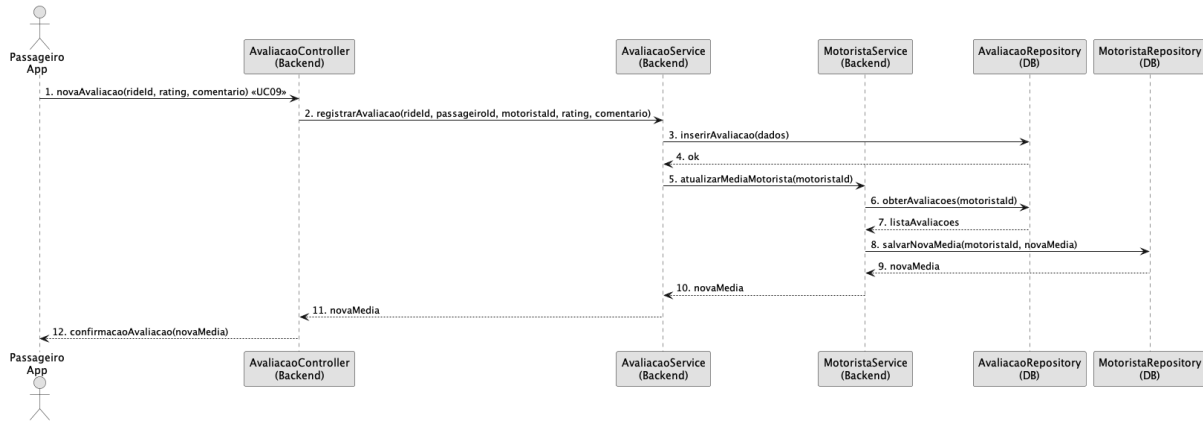


Figura 20 – Diagrama de Sequência Avaliação do Motorista

A Figura 21 ilustra o painel administrativo. O Administrador realiza *login*, solicita relatórios (por período, região, motorista etc.), e o *backend* consulta o banco de dados para agregar dados de corridas e pagamentos. Opcionalmente, o Administrador pode ajustar configurações (taxas, regiões), que são persistidas para uso nos cálculos. Relaciona-se aos casos de uso UC16 (Gerenciar usuários/motoristas), UC17 (Ver relatórios e métricas) e UC18 (Configurar taxas/regiões).

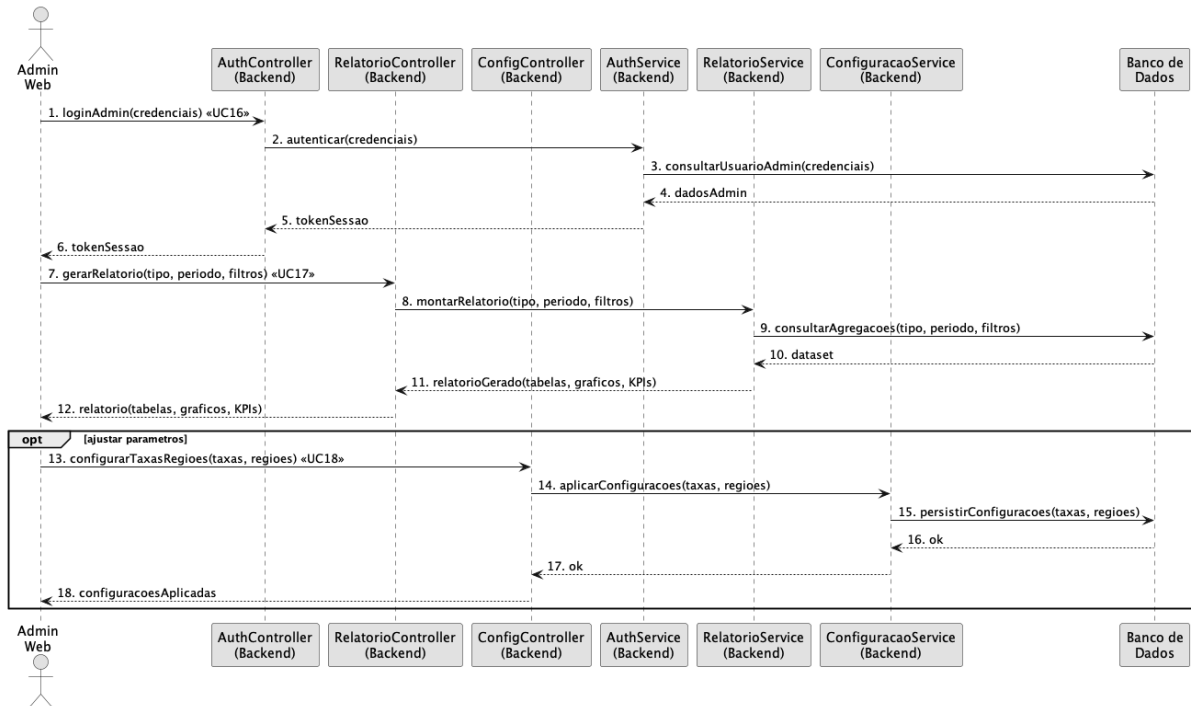


Figura 21 – Diagrama de Sequência Painel Administrativo

A Figura 22 apresenta o fluxo de *chat* em tempo real durante uma corrida. O usuário (passageiro ou motorista) envia uma mensagem; o *ChatService* persiste a mensagem no banco de dados e o *WebSocket Gateway* entrega instantaneamente ao outro participante. O histórico de mensagens pode ser consultado a qualquer momento. Este fluxo cobre UC20 (Enviar mensagem) e UC21 (Consultar histórico de *chat*).

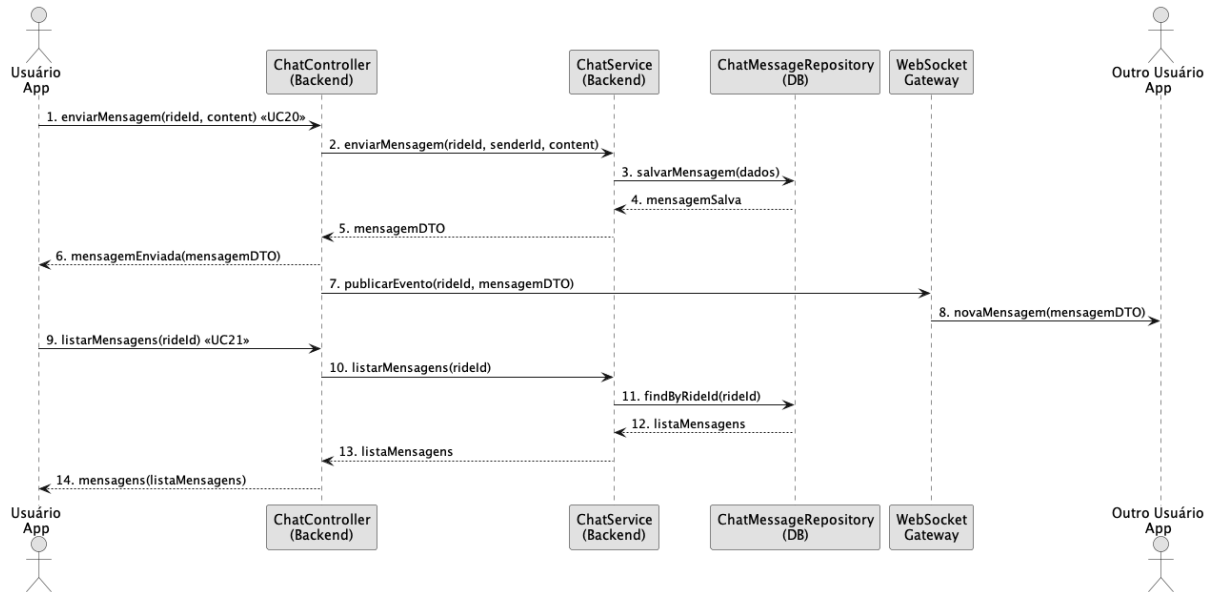


Figura 22 – Diagrama de Sequência Chat em Tempo Real

A Figura 23 descreve o fluxo de aprovação do cadastro de motorista pelo Administrador. O Administrador lista os candidatos pendentes, analisa documentos e *selfie*, e aprova ou rejeita o cadastro. O motorista é notificado automaticamente via *Push Service*. Este fluxo cobre UC22 (Aprovar motorista) e UC23 (Rejeitar motorista).

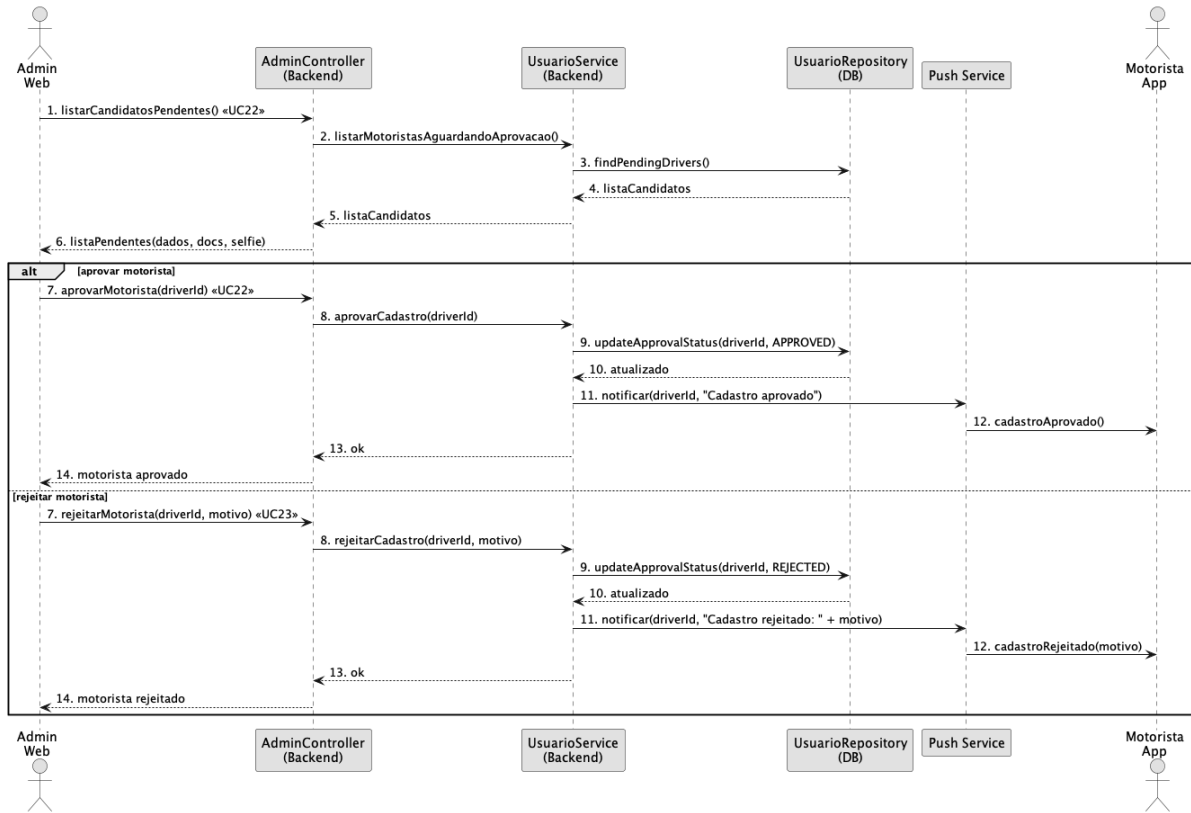


Figura 23 – Diagrama de Sequência Aprovação de Cadastro de Motorista

A Figura 24 mostra o fluxo de validação e aplicação de cupom de desconto. O Passageiro informa o código do cupom antes de solicitar a corrida; o *PrecificacaoService* verifica a validade e o desconto no *PromoCodeRepository*, aplica o desconto à tarifa calculada e registra o uso do cupom. Este fluxo cobre UC25 (Validar cupom) e UC05 (Solicitar corrida com desconto).

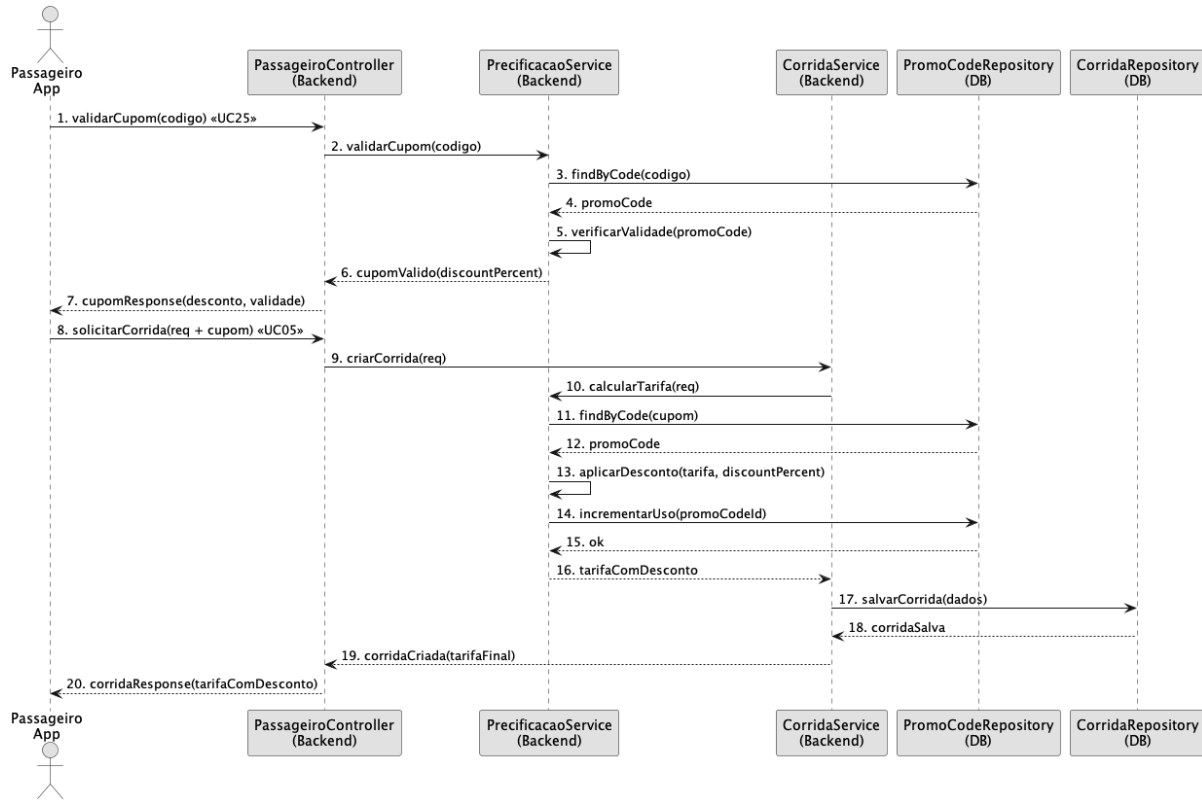


Figura 24 – Diagrama de Sequência Cupom de Desconto

A Figura 25 apresenta o fluxo de avaliação do passageiro pelo Motorista após o encerramento da corrida. O motorista informa nota e comentário; o *ReviewService* persiste a avaliação e recalcula a média do passageiro. Esse mecanismo permite manter histórico de qualidade de ambas as partes. Este fluxo cobre UC24 (Avaliar passageiro).

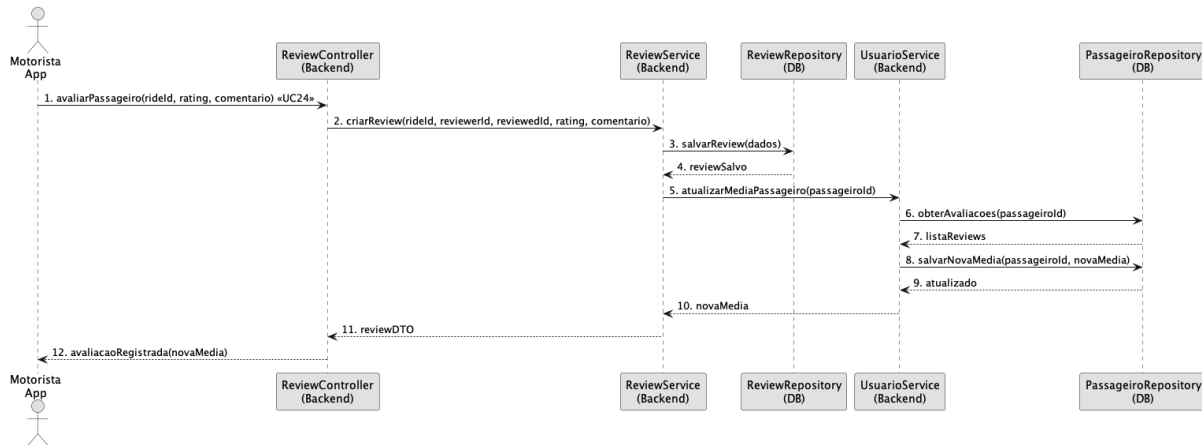


Figura 25 – Diagrama de Sequência Avaliação do Passageiro

A Figura 26 descreve o fluxo de solicitação de saque pelo motorista e de liquidação pelo Administrador. O motorista solicita o saque informando o valor e a chave PIX; o sistema registra a solicitação com *status* pendente. O Administrador lista as solicitações, processa o pagamento externamente e registra a liquidação no sistema. Este fluxo cobre UC26 (Solicitar saque) e UC30 (Registrar liquidação).

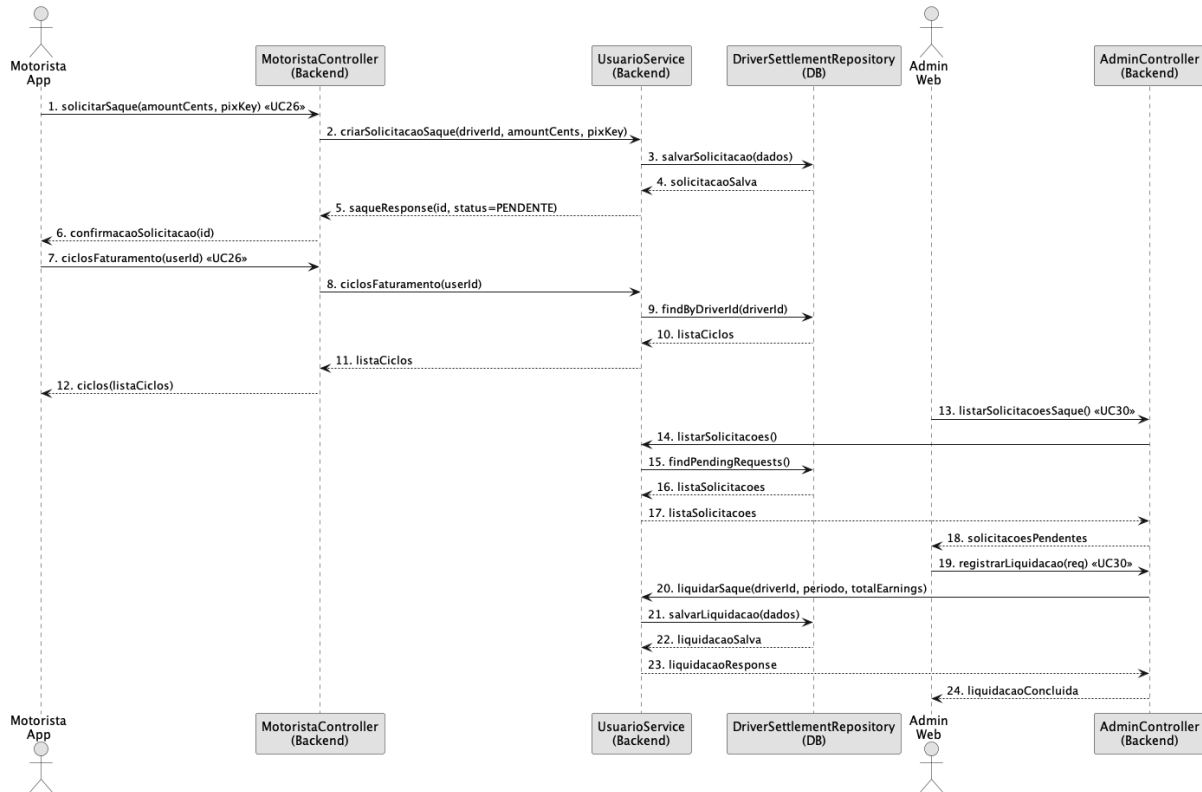


Figura 26 – Diagrama de Sequência Solicitação de Saque

A Figura 27 ilustra o fluxo de bloqueio e desbloqueio de usuário pelo Administrador. Ao bloquear uma conta, o sistema atualiza o *status* no banco de dados e notifica o usuário via *Push Service*. O desbloqueio segue o mesmo fluxo invertido. Este fluxo cobre UC27 (Bloquear/desbloquear usuário).

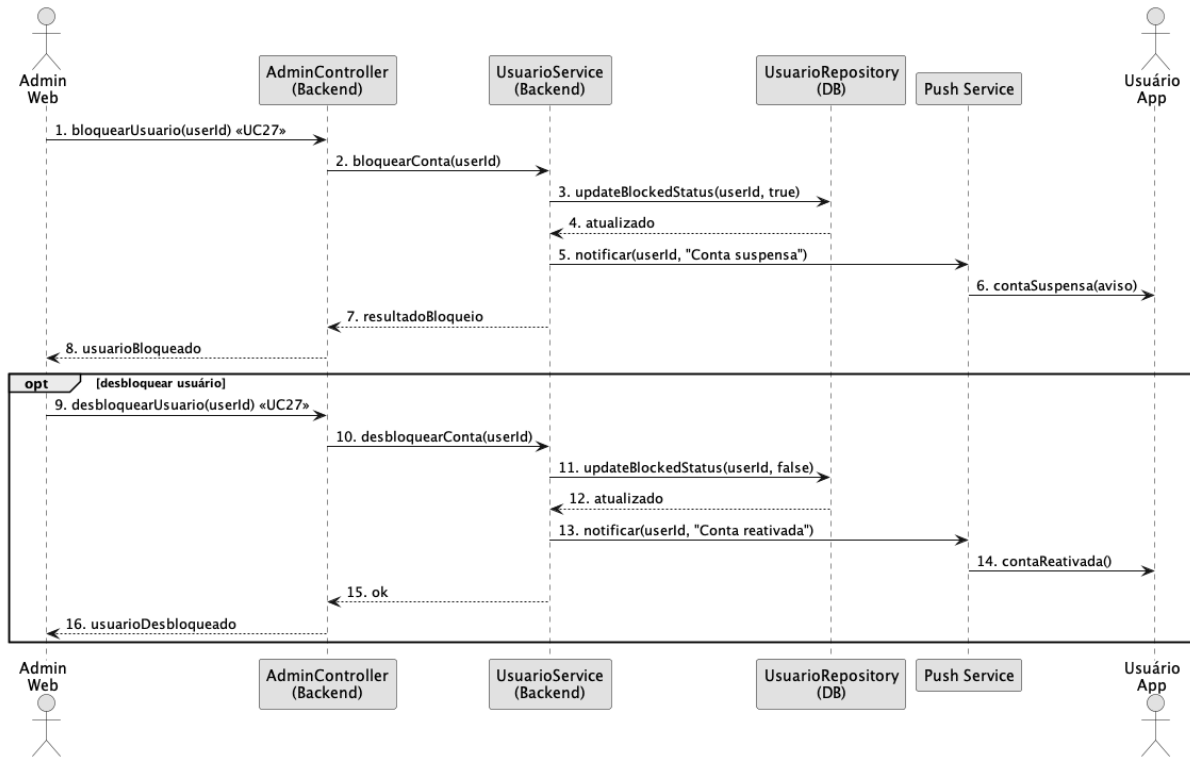


Figura 27 – Diagrama de Sequência Bloquear Usuário

3.3 Diagramas de Comunicação

Nesta seção, são apresentados os diagramas de comunicação que modelam as trocas de mensagens entre atores, camadas e serviços do sistema de transporte. O objetivo é documentar quem se comunica com quem e em que ordem, cobrindo os principais fluxos: autenticação, solicitação e despacho de corrida, atualização de *status*, pagamento, *chat* em tempo real, aprovação de motorista e gestão de cupons e saques. Os diagramas a seguir complementam os diagramas de sequência, servindo como registro das interfaces e contratos de mensagem utilizados na aplicação.

A Figura 28 apresenta as trocas de mensagens para autenticação de usuários (passageiros e motoristas). O aplicativo envia as credenciais para a *API*, que delega a validação ao *AuthService*. Em seguida, os dados do usuário são obtidos no *UsuarioRepository* e um *token* de sessão é retornado ao cliente. Esse fluxo garante a segurança de acesso e inicializa o contexto de sessão para as demais operações do sistema.

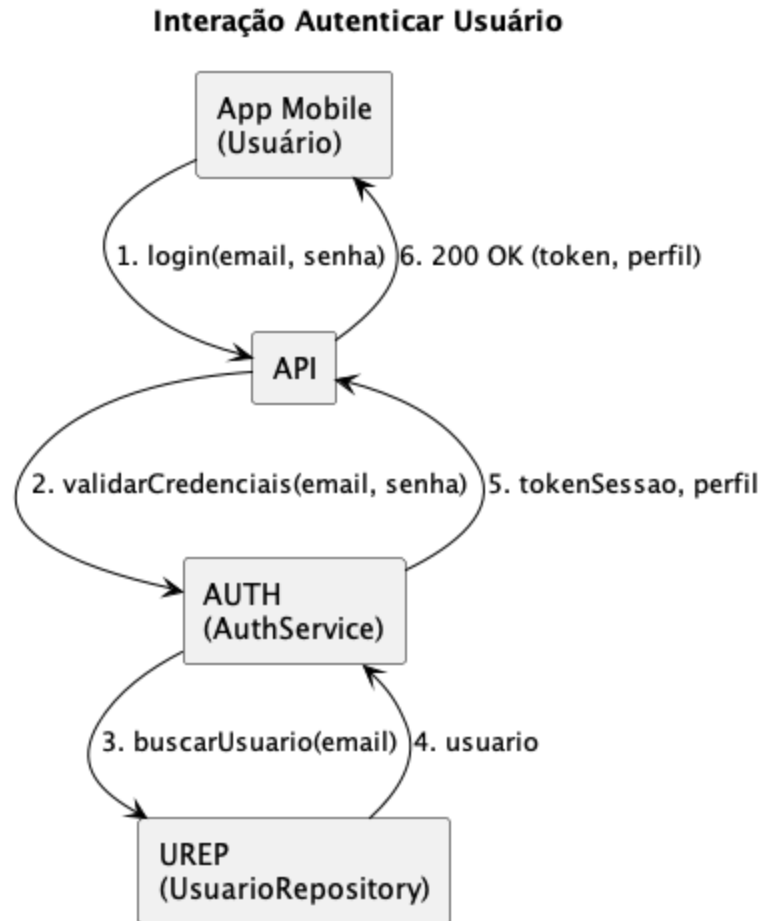


Figura 28 – Diagrama de Comunicação Autenticar Usuário

A Figura 29 descreve o fluxo de comunicação para solicitação de corrida. Após informar origem/destino, a *API* consulta a *Maps/Geo API* para estimativa de rota/tempo, chama o *MatchingService* para selecionar o motorista disponível mais próximo e utiliza o *PushService* para notificar o motorista. A resposta do motorista (aceite/recusa) retorna à *API*, que confirma ao passageiro e publica o estado inicial da corrida.

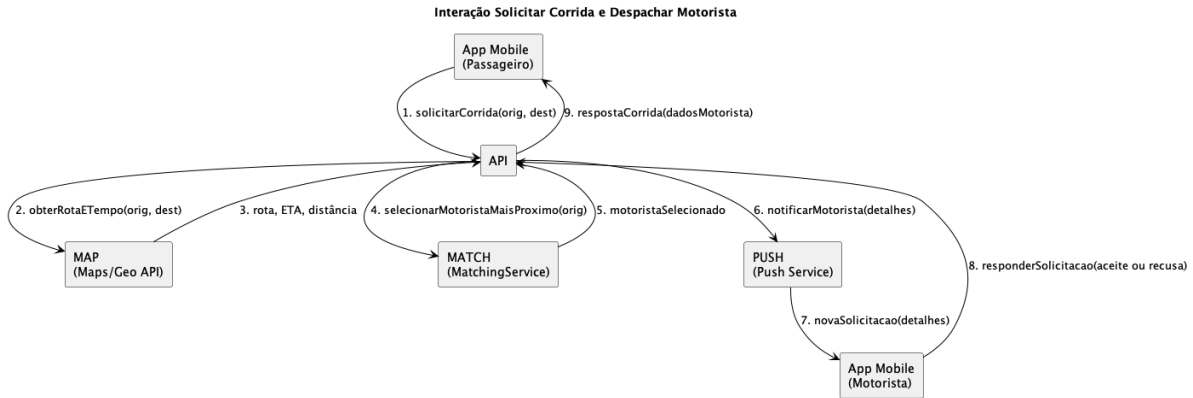


Figura 29 – Diagrama de Comunicação Solicitar Corrida e Despachar Motorista

A Figura 30 mostra as mensagens trocadas durante a execução da corrida. O motorista envia eventos de *status* (a caminho, chegou, iniciou, finalizou) à *API*. A *API* persiste no *CorridaRepository*, recalcula ETA quando necessário via *Maps/Geo API* e notifica passageiro/motorista com atualizações relevantes. Esse fluxo mantém ambas as partes informadas e cria o histórico operacional da corrida.

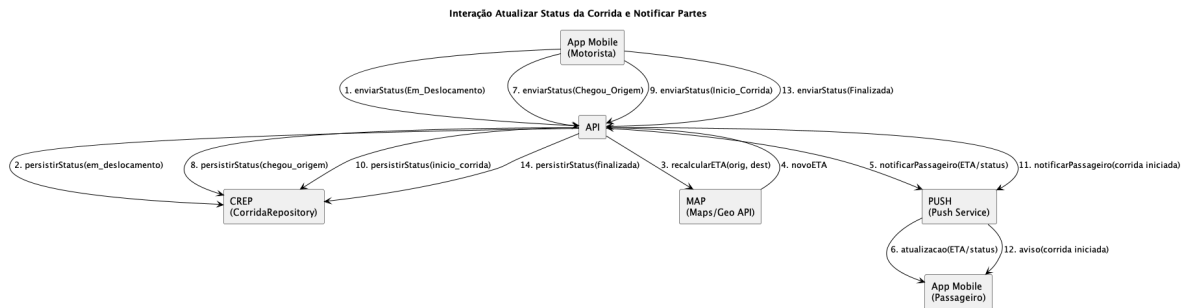


Figura 30 – Diagrama de Comunicação Atualizar Status da Corrida e Notificar Partes

A Figura 31 detalha as comunicações no pagamento. Ao finalizar a corrida, o passageiro escolhe PIX ou dinheiro. Para PIX, a *API* gera o *QR Code* internamente e o exibe no *app*; o motorista confere manualmente no seu banco e confirma na *API*. Para dinheiro, o motorista apenas confirma o recebimento. Em ambos os casos, a *API* atualiza a Corrida para "Finalizada" e emite recibos.

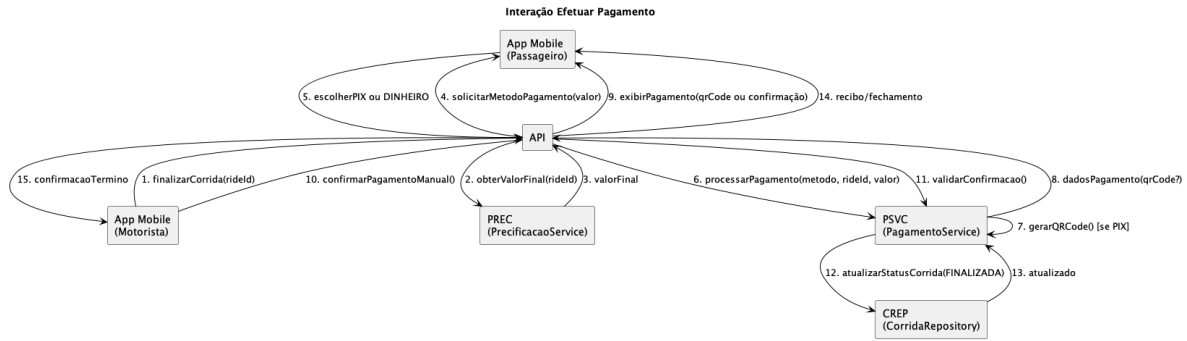


Figura 31 – Diagrama de Comunicação Efetuar Pagamento

A Figura 32 apresenta o fluxo de comunicação do *chat* em tempo real. O usuário envia uma mensagem via *app* à *API*, que delega ao *ChatService* a persistência no *ChatMessageRepository*. O *WebSocket Gateway* entrega a mensagem em tempo real ao outro participante da corrida.

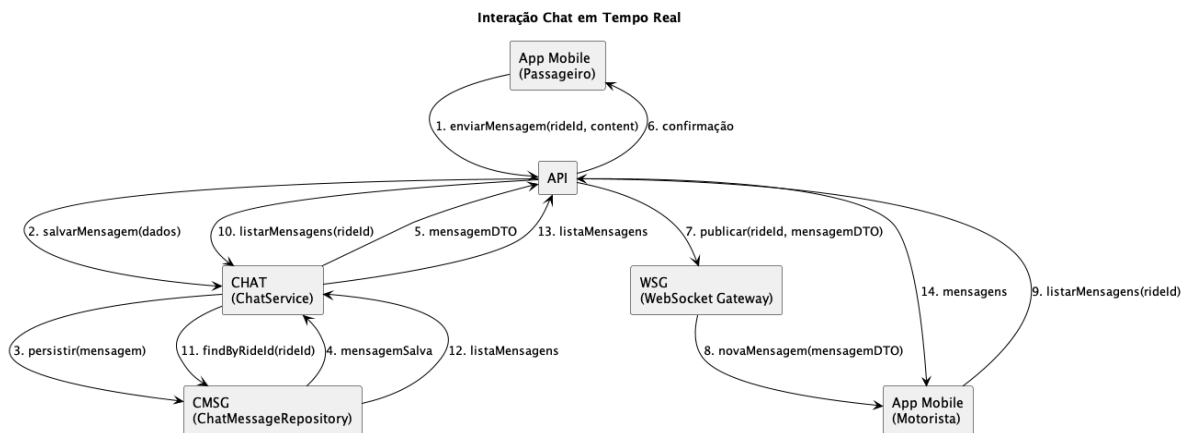


Figura 32 – Diagrama de Comunicação Chat em Tempo Real

A Figura 33 mostra as trocas de mensagem no fluxo de aprovação de cadastro de motorista. O Administrador autentica-se, lista candidatos pendentes, e aprova ou rejeita via *API*. O *UsuarioService* atualiza o *status* no repositório e o *Push Service* notifica o motorista do resultado.

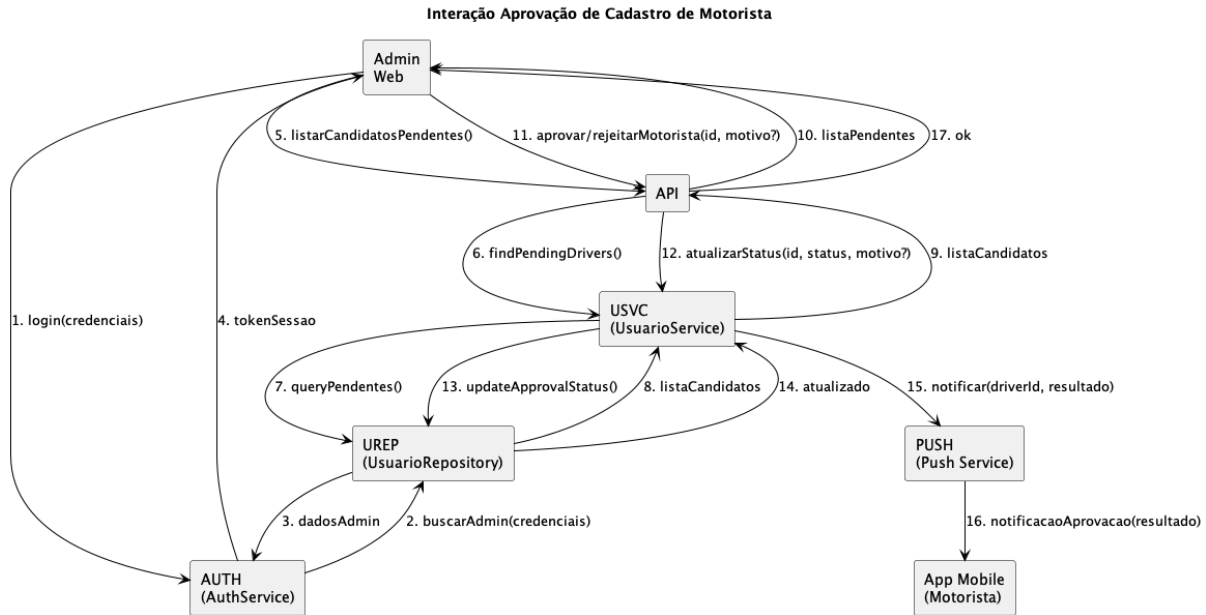


Figura 33 – Diagrama de Comunicação Aprovação de Cadastro de Motorista

A Figura 34 ilustra as comunicações para validação de cupom de desconto e solicitação de saque. Na validação de cupom, o passageiro consulta o código via *API* antes de solicitar a corrida; o *PrecificacaoService* aplica o desconto e registra o uso. Na solicitação de saque, o motorista envia os dados de pagamento; o Administrador processa externamente e registra a liquidação no sistema.

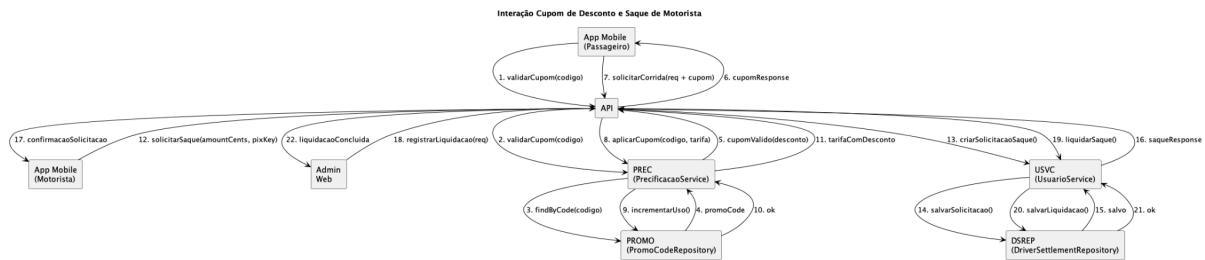


Figura 34 – Diagrama de Comunicação Cupom de Desconto e Saque de Motorista

3.4 Arquitetura

As decisões tecnológicas que fundamentam a arquitetura do MobU foram tomadas com base em critérios de adequação ao domínio, produtividade de desenvolvimento e facilidade de manutenção. O *NestJS* foi adotado como *framework* do *backend* por sua organização modular nativa e pelo contêiner de injeção de dependências embutido, que favorecem a separação de responsabilidades e a escrita de testes unitários com *mocks*. Em comparação ao *Express* puro, o *NestJS* impõe uma

estrutura que reduz a divergência de padrões entre desenvolvedores e agiliza a incorporação de novos módulos. O *PostgreSQL* foi escolhido por oferecer conformidade *ACID*, essencial para operações que envolvem registros financeiros como pagamentos e liquidações, e por sua integração nativa com o *Prisma ORM*, que gera tipos *TypeScript* automaticamente a partir do esquema e elimina erros de tipagem nas consultas ao banco. O aplicativo Android nativo em *Kotlin* foi preferido em relação a soluções multiplataforma por oferecer melhor integração com o *Google Maps SDK*, suporte mais completo ao *Firebase Cloud Messaging* e desempenho superior para atualizações de localização em tempo real. O *Next.js* foi selecionado para o painel administrativo por combinar renderização no servidor (*SSR*) com o ecossistema *React*, adequada para painéis com dados dinâmicos e necessidade de atualização frequente. Por fim, a containerização com *Docker* desde o início do projeto garantiu paridade entre os ambientes de desenvolvimento e produção, eliminando inconsistências de configuração e simplificando a implantação do sistema.

A Figura 35 representa o diagrama de pacotes, também conhecido como diagrama de arquitetura, do sistema MobU. Nele é possível observar a organização estrutural dos principais módulos da aplicação, representando a separação de responsabilidades entre as camadas que compõem o sistema.

O primeiro pacote representa a aplicação *mobile*, que contém a Interface de Usuário (*UI*, do inglês *User Interface*), responsável pela interação direta com os diferentes perfis de usuários — passageiro, motorista e administrador. Essa camada é responsável por exibir as informações e capturar as ações realizadas pelos usuários durante o uso do aplicativo.

A aplicação *mobile* se comunica com a MobU API, que corresponde à camada intermediária do sistema e é responsável por processar as regras de negócio e gerenciar as comunicações com os serviços externos. Dentro desta camada, estão definidos os pacotes *Controllers*, *Services*, *Jobs* e *Utils*, os quais implementam as funcionalidades principais da aplicação.

- O pacote *Controllers* contém os pontos de entrada da API e é responsável por intermediar as requisições entre o cliente e o *backend*.
- O pacote *Services* contém a lógica de negócio central do sistema, como o gerenciamento de corridas, pagamentos, autenticações, *chat* em tempo real, suporte ao usuário, avaliações e gestão de cupons e saques.
- O pacote *Jobs* trata de tarefas assíncronas e agendadas, como envio de notificações, processamento de relatórios e liquidações.
- O pacote *Utils* contém funções auxiliares reutilizáveis em diferentes partes da aplicação.
- O pacote *Libs* reúne bibliotecas e adaptadores compartilhados entre serviços.

Por fim, a camada Data é responsável pelo acesso e persistência dos dados, representando a comunicação com o banco de dados e os modelos de entidades da aplicação. Ela contém o pacote *Models*, que define as estruturas de dados principais do sistema, como *Usuário*, *Corrida*, *Pagamento*, *Avaliação*, *ChatMessage*, *SupportTicket*, *Review*, *PromoCode* e *DriverSettlement*.

Essa arquitetura segue o padrão camada UI → API (*business*) → Data, proporcionando alta coesão e baixo acoplamento entre os módulos, o que facilita a manutenção, a escalabilidade e futuras expansões da aplicação.

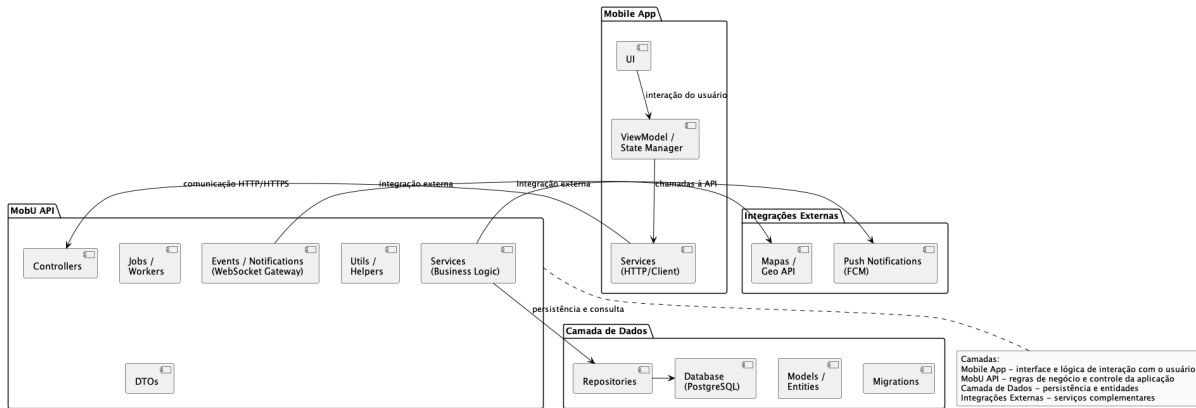


Figura 35 – Diagrama de Pacotes

3.5 Diagramas de Estados

O diagrama representado pela Figura 36 contém a lógica de mudança de estados pelos quais uma corrida passa no sistema MobU. É possível observar que a corrida percorre uma sequência de estados que refletem as diferentes fases do processo de transporte, desde sua solicitação até a finalização ou cancelamento.

Inicialmente, o estado **Solicitada** representa a criação da corrida pelo passageiro. Em seguida, a corrida pode ser **Despachada**, momento em que o sistema tenta associar um motorista disponível à solicitação. A partir desse ponto, o motorista pode aceitar ou recusar a corrida, resultando respectivamente nos estados **Aceita** ou **Recusada** — neste último caso, o sistema pode reencaminhar a solicitação para outro motorista disponível.

Quando aceita, a corrida avança para o estado **A Caminho da Origem**, no qual o motorista se dirige até o ponto de embarque do passageiro. Ao chegar, a corrida transita para o estado **Chegou à Origem** e, após o embarque, é iniciada, entrando no estado **Em Andamento**.

Por fim, o estado **Finalizada** indica a conclusão da corrida, com o registro das informações de trajeto e valor.

O diagrama também contempla transições de cancelamento, que podem ocorrer em diferentes fases do processo, seja por parte do passageiro, do motorista ou por algum motivo forçado pelo sistema. Essas transições levam a corrida ao estado **Cancelada**, que, assim como o estado final de **Finalizada**, encerra o ciclo de vida da corrida dentro do sistema.

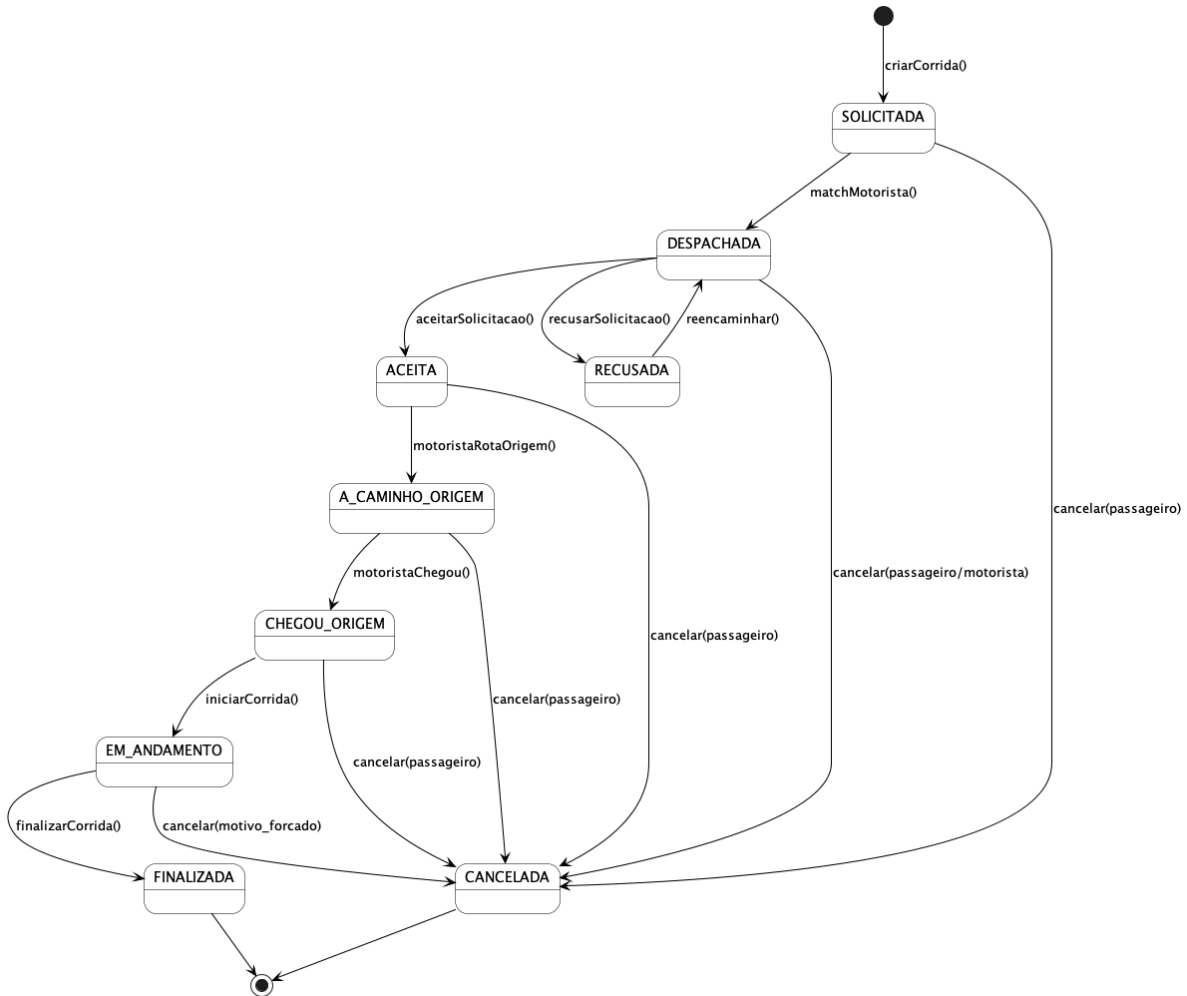


Figura 36 – Diagrama de Estados da Corrida

A Figura 37 representa o Diagrama de Estados do Pagamento no sistema MobU. Esse diagrama descreve o ciclo de vida de um pagamento associado a uma corrida, desde o momento em que a corrida é finalizada até a confirmação ou cancelamento da transação.

O processo se inicia no estado **Aguardando**, quando a corrida é concluída e o usuário deve escolher o método de pagamento. A partir desse ponto, existem dois fluxos possíveis:

1. Caso o usuário escolha PIX, o sistema gera um *QR Code* internamente, e o pagamento entra no estado PIX Gerado.
2. Caso o usuário opte por dinheiro, o pagamento segue para o estado Aguardando Confirmação do Motorista, onde o motorista deve confirmar o recebimento.

A partir do estado **PIX Gerado**, o pagamento pode ser **Confirmado** após a validação do txId ou pode **Expirar** caso o *QR Code* não seja utilizado dentro do prazo estabelecido. Além disso, o pagamento pode ser **Cancelado** manualmente em qualquer um desses estados intermediários.

Por fim, os estados **Confirmado**, **Expirado** e **Cancelado** representam os estados finais do processo de pagamento, encerrando o ciclo de transações para aquela corrida específica.

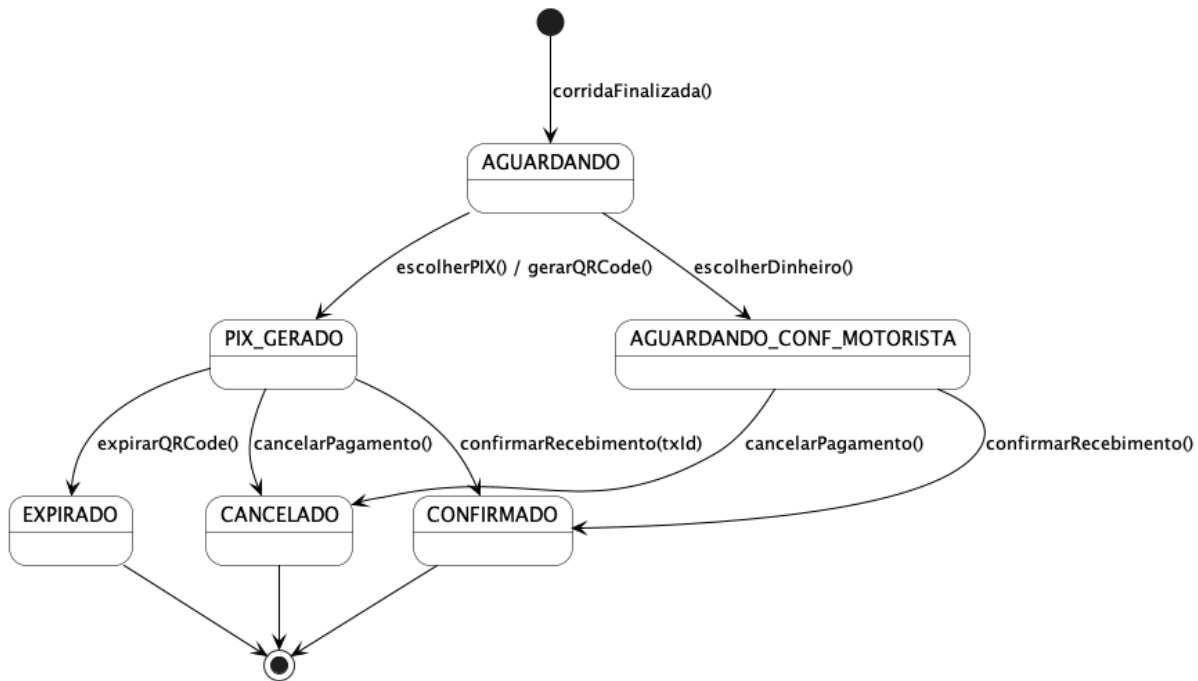


Figura 37 – Diagrama de Estados do Pagamento

Além dos estados de corrida e pagamento, o sistema MobU contempla outros ciclos de vida relevantes às novas funcionalidades implementadas: aprovação de cadastro de motorista (Figura 38), *ticket* de suporte (Figura 39) e solicitação de saque (Figura 40).

A Figura 38 descreve o ciclo de vida do cadastro de um motorista. Após submeter os documentos e a *selfie*, o cadastro entra no estado **Pendente**. O administrador inicia a análise, movendo para **Em Análise**. Após avaliação, o cadastro pode ser **Aprovado** ou **Rejeitado**. Em caso de rejeição, o motorista pode re-submeter os documentos. Uma conta aprovada pode ser **Bloqueada** pelo administrador e posteriormente desbloqueada.

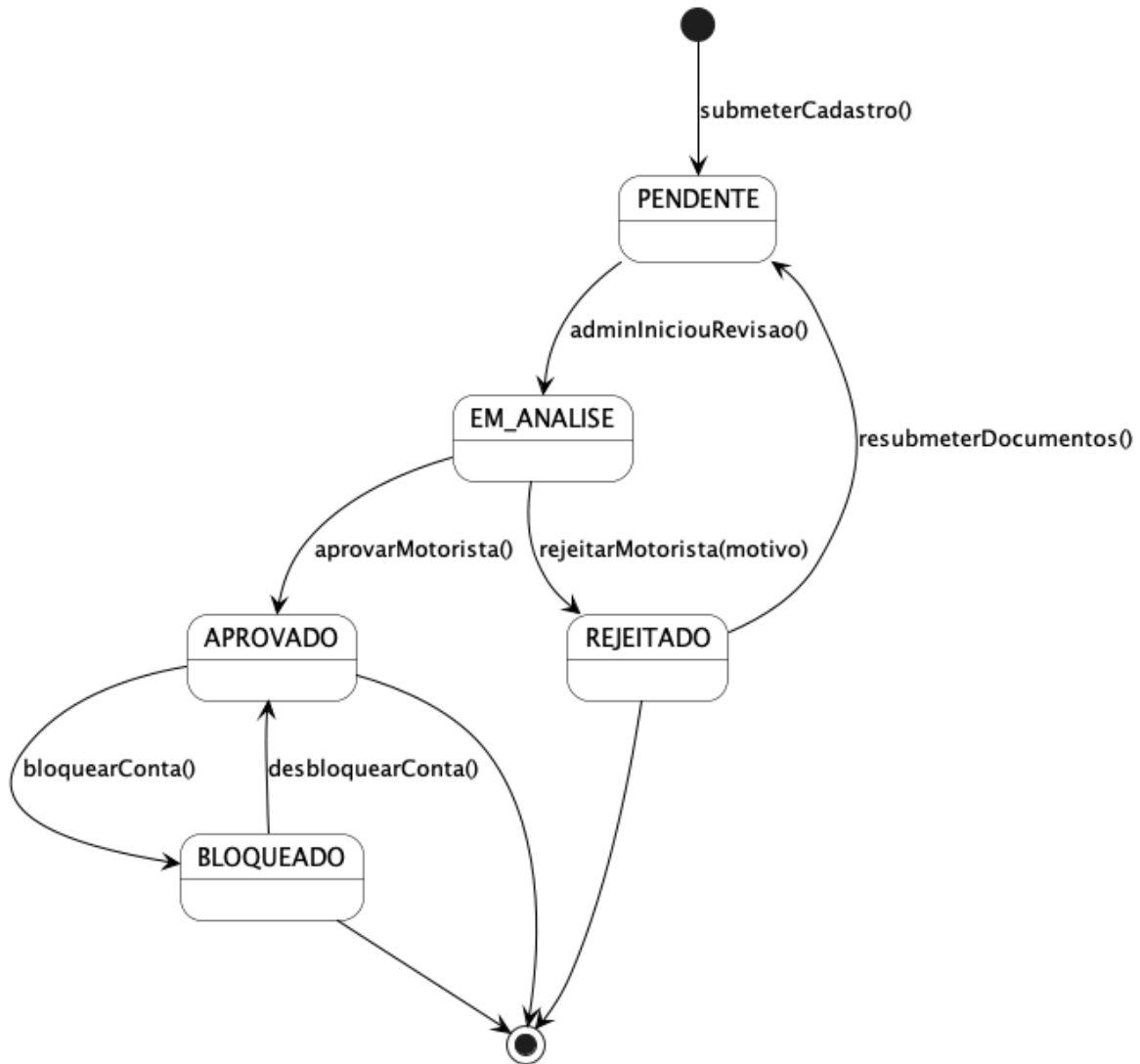


Figura 38 – Diagrama de Estados da Aprovação de Motorista

A Figura 39 representa o ciclo de vida de um *ticket* de suporte. O usuário abre um chamado (**Aberto**), o administrador o assume (**Em Andamento**) e pode solicitar informações ao usuário (**Aguardando Usuário**). Após resolução, o *ticket* é **Fechado** ou pode ser **Cancelado** em qualquer etapa.

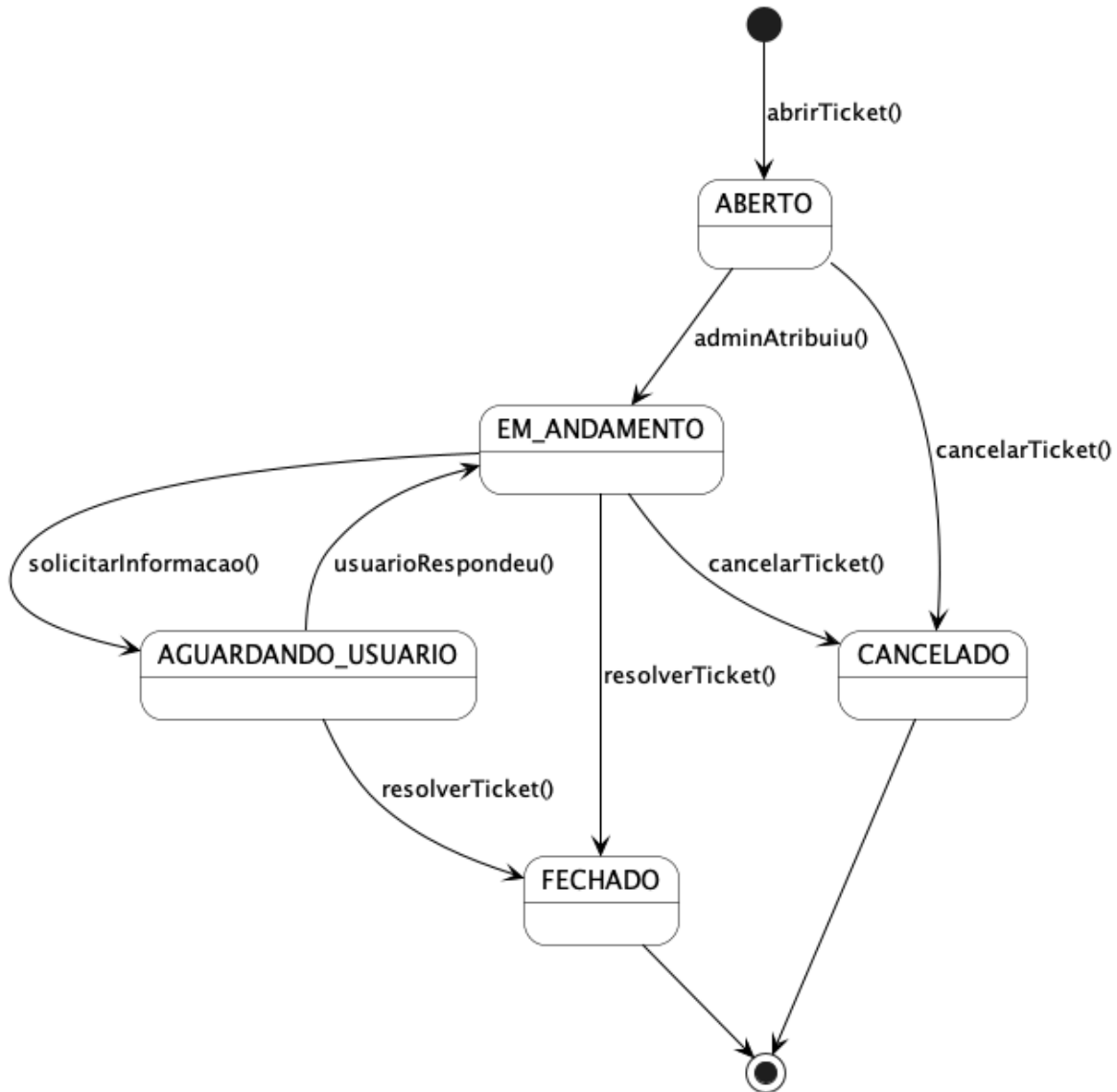


Figura 39 – Diagrama de Estados do Ticket de Suporte

A Figura 40 descreve o ciclo de vida de uma solicitação de saque do motorista. A solicitação inicia no estado **Pendente**, é analisada pelo administrador (**Em Análise**), e finaliza com **Liquidado** após o pagamento externo ser processado e registrado, ou **Cancelado** em caso de recusa.

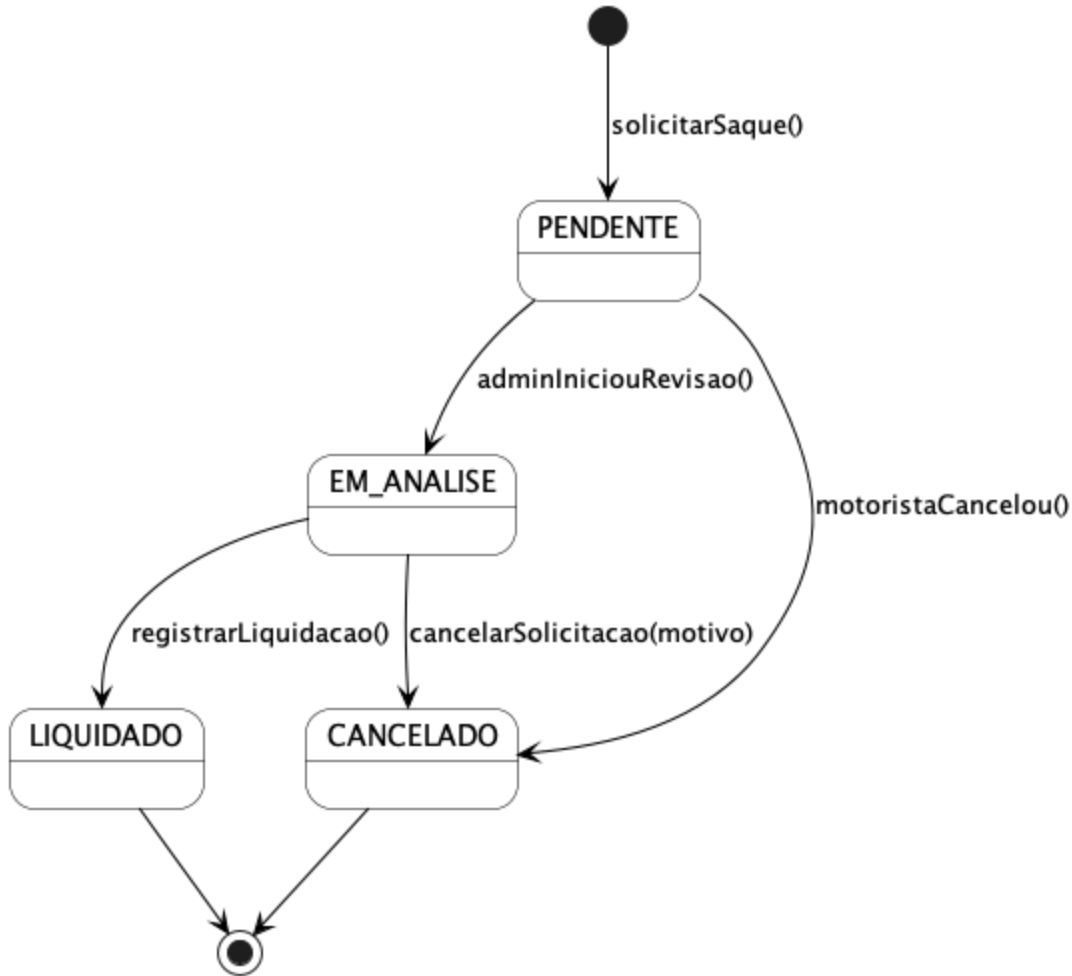


Figura 40 – Diagrama de Estados da Solicitação de Saque

3.6 Diagrama de Componentes e Implantação

O diagrama representado pela Figura 41 mostra a organização dos principais componentes que compõem a arquitetura lógica do sistema *MobU*. Nele é possível observar dois grandes módulos: o *Mobile App* e a *MobU API*, além de seus elementos de integração com serviços externos e infraestrutura de persistência de dados.

O *Mobile App* representa a aplicação móvel executada em dispositivos *Android*, composta pela camada de Interface de Usuário (*UI*), responsável pelas páginas, componentes visuais e estados da aplicação. Essa camada se comunica com a *MobU API* por meio de requisições *HTTP/HTTPS*, realizadas através do módulo *Services*, que encapsula as funções de acesso à *API* e controle de fluxo de dados entre o cliente e o servidor.

A *MobU API* representa o núcleo de processamento do sistema, responsável por tratar as regras de negócio e integrar as diferentes partes da aplicação. Dentro dessa camada, encontram-se pacotes especializados:

- *Controllers*, que atuam como pontos de entrada das requisições;
- *Services*, que implementam a lógica de negócio principal, incluindo gestão de corridas, pagamentos, *chat*, suporte, avaliações e saques;
- *Jobs*, responsáveis por tarefas assíncronas e agendadas;
- *Utils* e *Libs*, que contêm funções e bibliotecas auxiliares;
- *Models*, que representam as entidades e dados do sistema;
- *APIs*, dedicadas às integrações externas, e *Events*, que tratam notificações e mensagens internas via *WebSocket*.

O SGBD *PostgreSQL 16* é responsável pelo armazenamento persistente de informações, gerenciado via *Prisma ORM*. Além disso, o sistema se integra a serviços externos — *Google Maps/Geo API* para geocodificação e roteamento, e *Firebase FCM* para notificações *push* — e utiliza *WebSocket* para comunicação em tempo real, garantindo a troca de mensagens eficiente entre o aplicativo móvel e o servidor.

Essa estrutura modular facilita a manutenção, o escalonamento e a reutilização de componentes, promovendo um acoplamento reduzido e alta coesão entre as camadas.

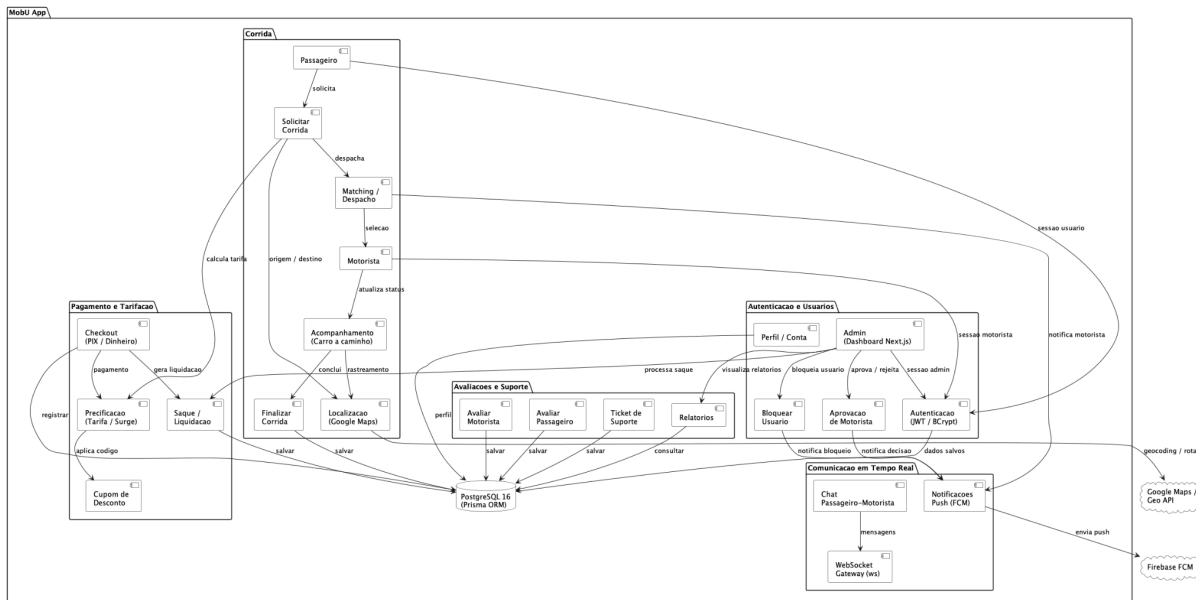


Figura 41 – Diagrama de Componentes da Aplicação

A Figura 42 apresenta o Diagrama de Componentes da aplicação *MobU*, organizado em grupos funcionais que evidenciam a separação de responsabilidades entre os módulos do sistema. O diagrama compreende os componentes internos da aplicação, a base de dados *PostgreSQL 16* (via *Prisma ORM*) e as integrações com serviços externos.

O grupo Autenticação e Usuários concentra os componentes responsáveis pelo controle de acesso e gestão de contas. O componente Autenticação (*JWT / BCrypt*) valida credenciais e gerencia sessões para os três perfis: Passageiro, Motorista e Admin (*Dashboard Next.js*). O componente Perfil / Conta

permite a visualização e edição dos dados do usuário, enquanto Aprovação de Motorista e Bloquear Usuário são operações administrativas que disparam notificações via *Firestore FCM*.

O grupo Corrida reúne os componentes do fluxo principal de transporte. O Passageiro inicia uma solicitação pelo componente Solicitar Corrida, que aciona o *Matching / Despacho* para identificar um motorista disponível e notificá-lo. O componente Localização (*Google Maps*) é utilizado para determinar a posição das partes e calcular rotas via *Google Maps / Geo API*. O acompanhamento do deslocamento é feito pelo componente Acompanhamento (Carro a caminho), e a conclusão da viagem é registrada pelo componente Finalizar Corrida.

O grupo Pagamento e Tarifação é composto pelos componentes Precificação (Tarifa / Surge), responsável pelo cálculo do valor da corrida com base na região e fator de surge, e Cupom de Desconto, que aplica promoções válidas. O *Checkout* (PIX / Dinheiro) gerencia o processo de pagamento — por PIX com *QR Code* gerado internamente ou por dinheiro com confirmação do motorista — e o componente Saque / Liquidação trata das solicitações de repasse financeiro aos motoristas.

O grupo Comunicação em Tempo Real é formado pelo componente *Chat* Passageiro-Motorista, que permite a troca de mensagens durante a corrida, pelo *WebSocket Gateway (ws)*, responsável pela conexão bidirecional persistente, e pelas Notificações *Push (FCM)*, que realizam o envio de alertas via *Firestore Cloud Messaging* para eventos como despacho, aprovação e bloqueio de conta.

O grupo Avaliações e Suporte engloba os componentes Avaliar Motorista e Avaliar Passageiro, que registram as avaliações mútuas ao término de uma corrida, o *Ticket* de Suporte, para abertura e acompanhamento de chamados, e os Relatórios, que fornecem ao administrador visibilidade sobre corridas realizadas, receitas e desempenho.

De forma geral, o diagrama demonstra uma arquitetura modular e bem definida, com separação clara de responsabilidades entre os componentes. Essa organização favorece a manutenção, a escalabilidade e a evolução futura da aplicação *MobU*, permitindo a inclusão de novos serviços e funcionalidades de maneira estruturada e controlada.

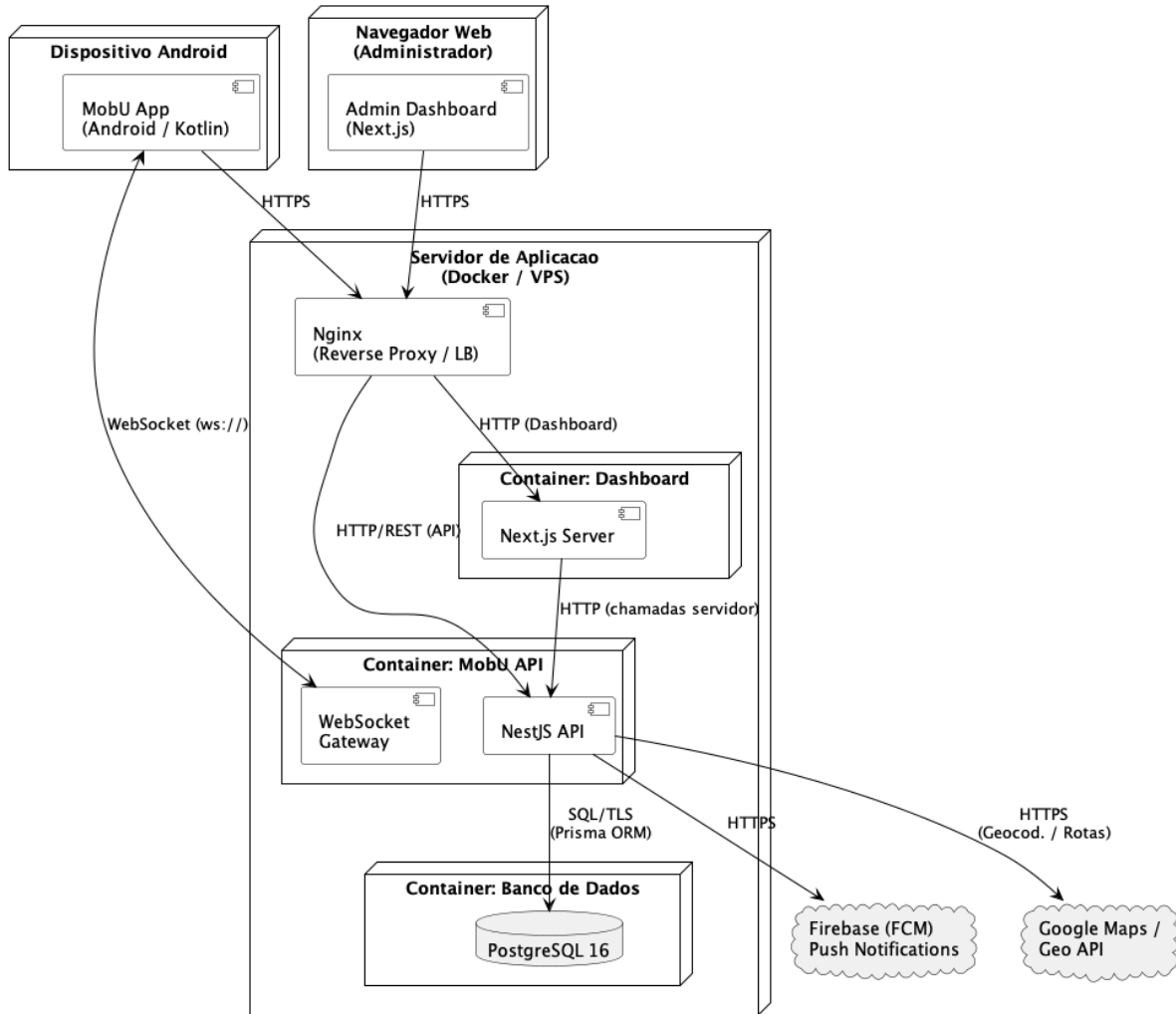


Figura 42 – Diagrama de Implantação da Aplicação

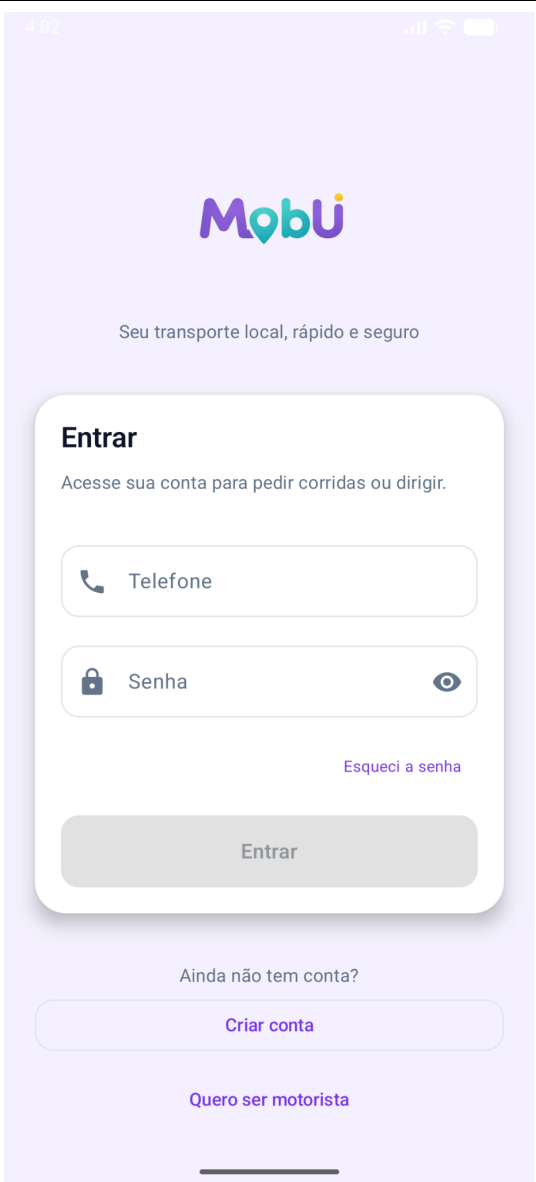
4. Projeto de Interface com Usuário

Esta seção apresenta as interfaces de interação com o usuário que compõem o sistema MobU, exibindo as telas reais da aplicação implementada. As interfaces são organizadas por perfil de acesso — passageiro, motorista e administrador — e cobrem os principais fluxos funcionais do sistema, desde o cadastro e autenticação até a solicitação de corridas, pagamentos, suporte e gestão administrativa.

4.1 Interfaces Comuns

A Figura 43 representa a tela de *Splash Screen* do aplicativo MobU, exibida ao iniciar o *app* enquanto o sistema verifica a sessão do usuário. Caso autenticado, o usuário é redirecionado

automaticamente para a tela principal. A Figura 44 apresenta a tela de *login*, onde o usuário informa telefone e senha para acessar a aplicação, com opções de cadastro e recuperação de senha.

 A screenshot of the MobU splash screen. It features a light gray background with the MobU logo (a stylized 'M' in purple and blue) centered. The status bar at the top shows the time 3:54, a lock icon, and cellular signal and battery indicators.	 A screenshot of the MobU login and registration screen. The background is light purple. At the top is the MobU logo and the tagline 'Seu transporte local, rápido e seguro'. Below this is a white rounded rectangle containing the 'Entrar' (Login) section. It includes a title 'Entrar', a subtitle 'Acesse sua conta para pedir corridas ou dirigir.', a phone number input field, a password input field with a toggle eye icon, a link 'Esqueci a senha', and a 'Entrar' button. Below the login section are links for 'Ainda não tem conta?' leading to 'Criar conta' and 'Quero ser motorista'.
Figura 43 – Splash Screen	Figura 44 – Tela de Login / Cadastro

4.2 Módulo do Administrador

O painel administrativo do MobU é acessado via navegador web (*Next.js*) e oferece controle completo sobre operações, usuários, tarifas e finanças da plataforma. A Figura 45 apresenta a tela de *login* do administrador e a Figura 46 exibe o *dashboard* principal com indicadores de corridas, receita total, pagamentos pendentes e solicitações de aprovação em aberto.

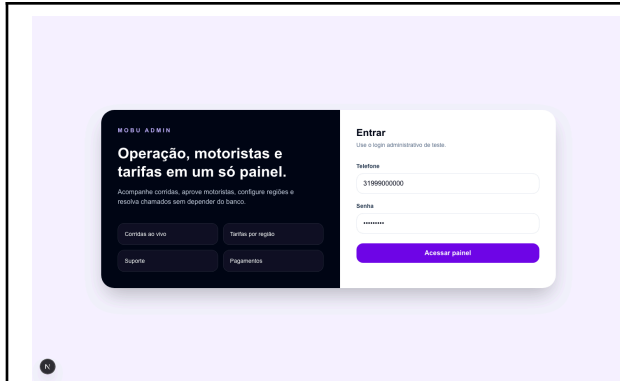


Figura 45 – Login do Administrador

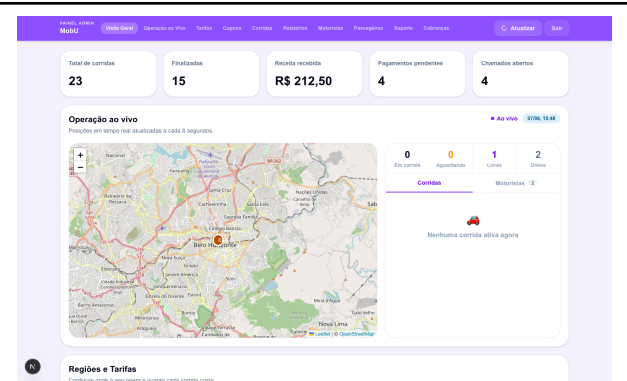


Figura 46 – Dashboard Principal

A Figura 47 exibe a seção de Operação ao Vivo, com mapa em tempo real mostrando a posição dos motoristas ativos e o painel lateral com corridas e motoristas *online*.

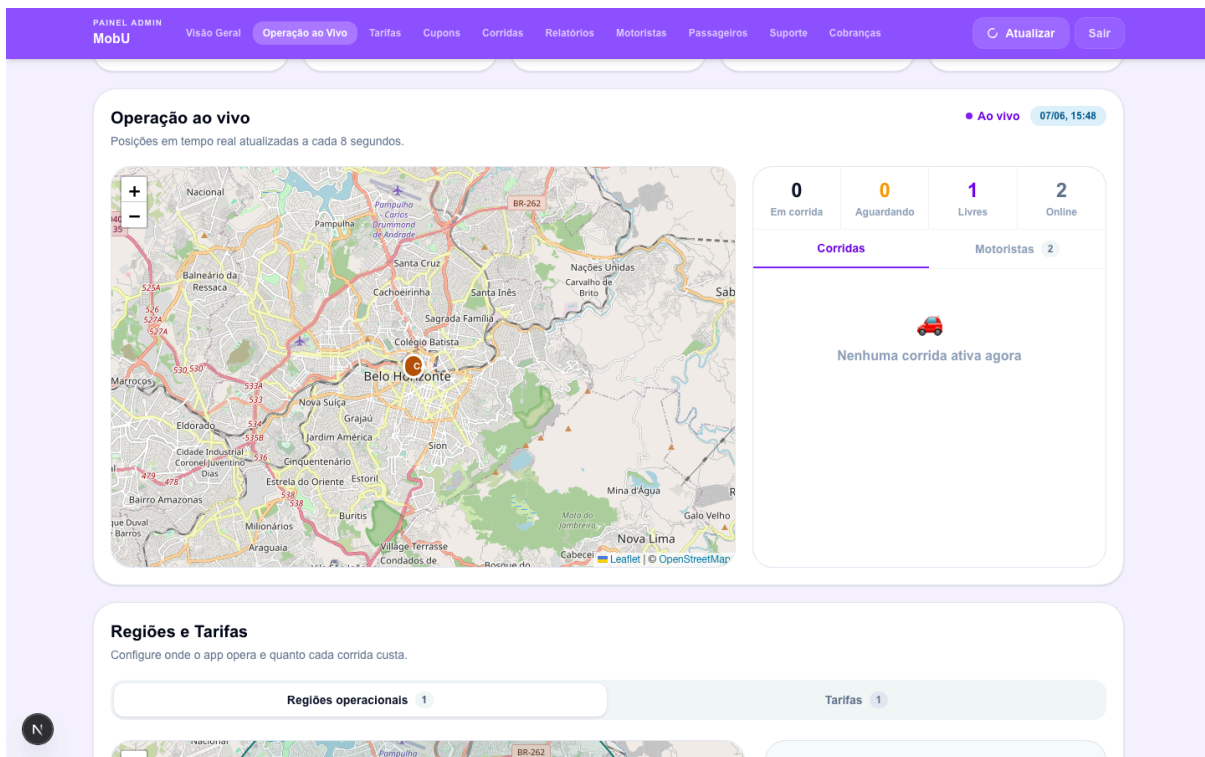


Figura 47 – Operação ao Vivo

A Figura 48 apresenta a gestão de motoristas e passageiros, com listagem, filtros por *status* (pendente, aprovado, rejeitado) e acesso aos detalhes de cada perfil. A Figura 49 exibe o módulo de

relatórios com métricas operacionais e financeiras, incluindo receita recebida, taxa de conclusão, *ticket* médio e avaliação média.

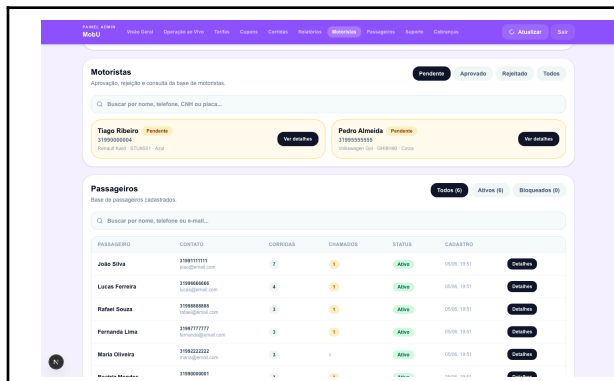


Figura 48 – Gestão de Motoristas e Passageiros

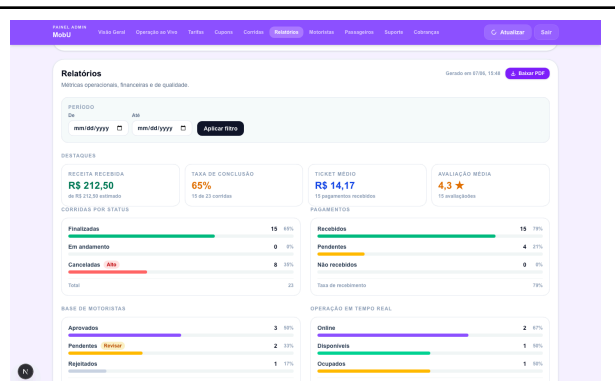


Figura 49 – Relatórios e Métricas

A Figura 50 apresenta o módulo de Tarifas e Regiões Operacionais, onde o administrador define regiões no mapa, valores base e fatores de surge. A Figura 51 exibe a gestão de Cupons de Desconto, com criação de códigos promocionais e configuração de percentual e validade.

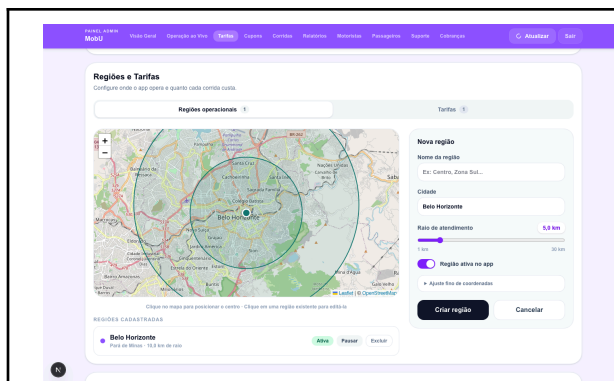


Figura 50 – Regiões e Tarifas

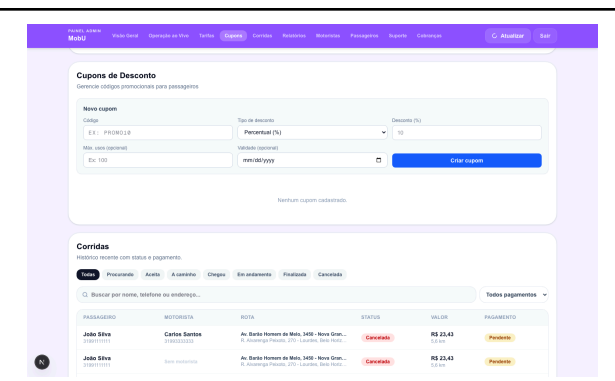
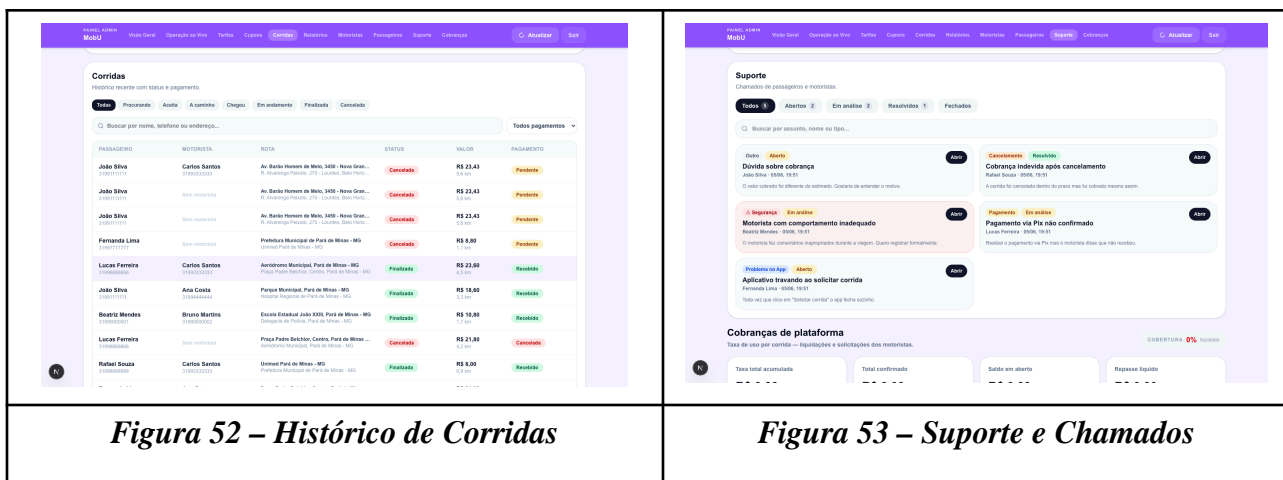


Figura 51 – Cupons de Desconto

A Figura 52 apresenta o histórico completo de Corridas com filtros por *status* e método de pagamento. A Figura 53 exibe o módulo de Suporte, com todos os chamados abertos pelos usuários, categorizados por tipo e *status*.



A Figura 54 apresenta o módulo de Cobranças de Plataforma, com o saldo por motorista, taxa acumulada e configurações de repasse e chave PIX da plataforma.

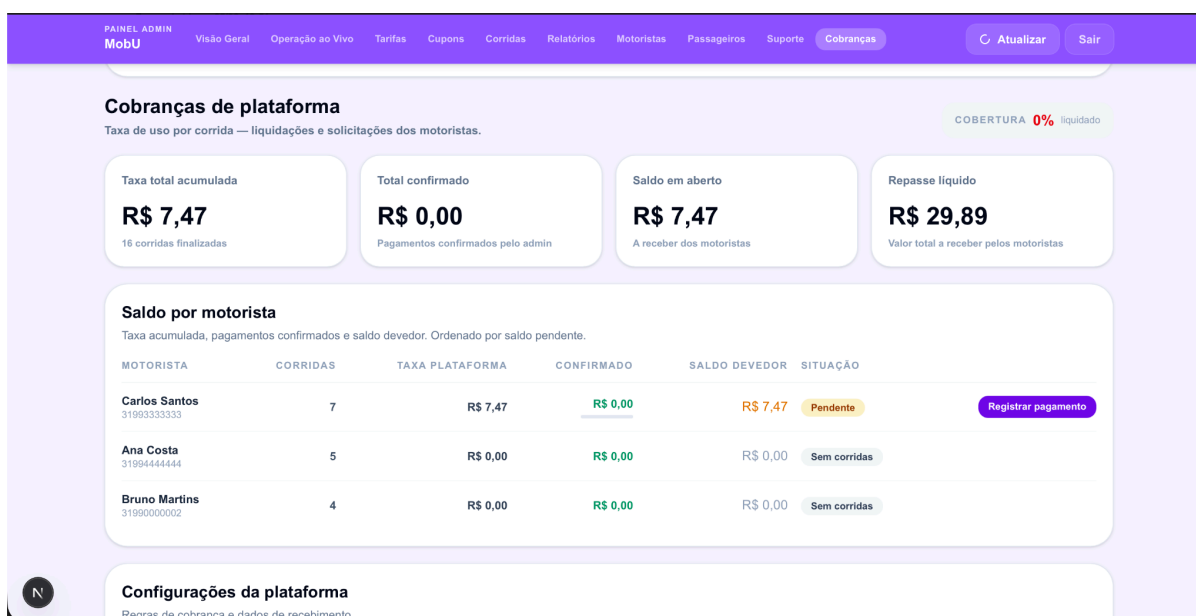
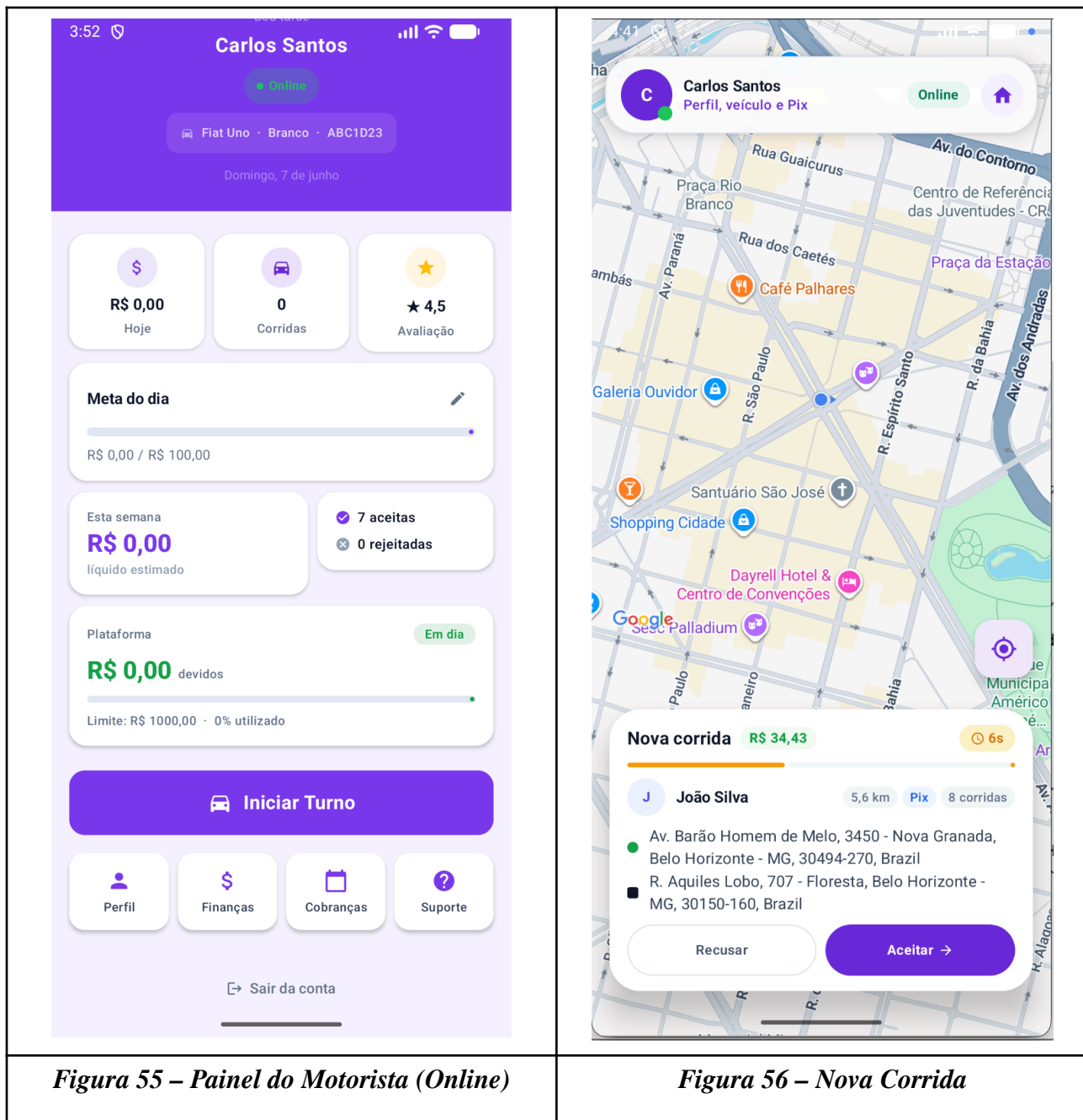


Figura 54 – Cobranças de Plataforma

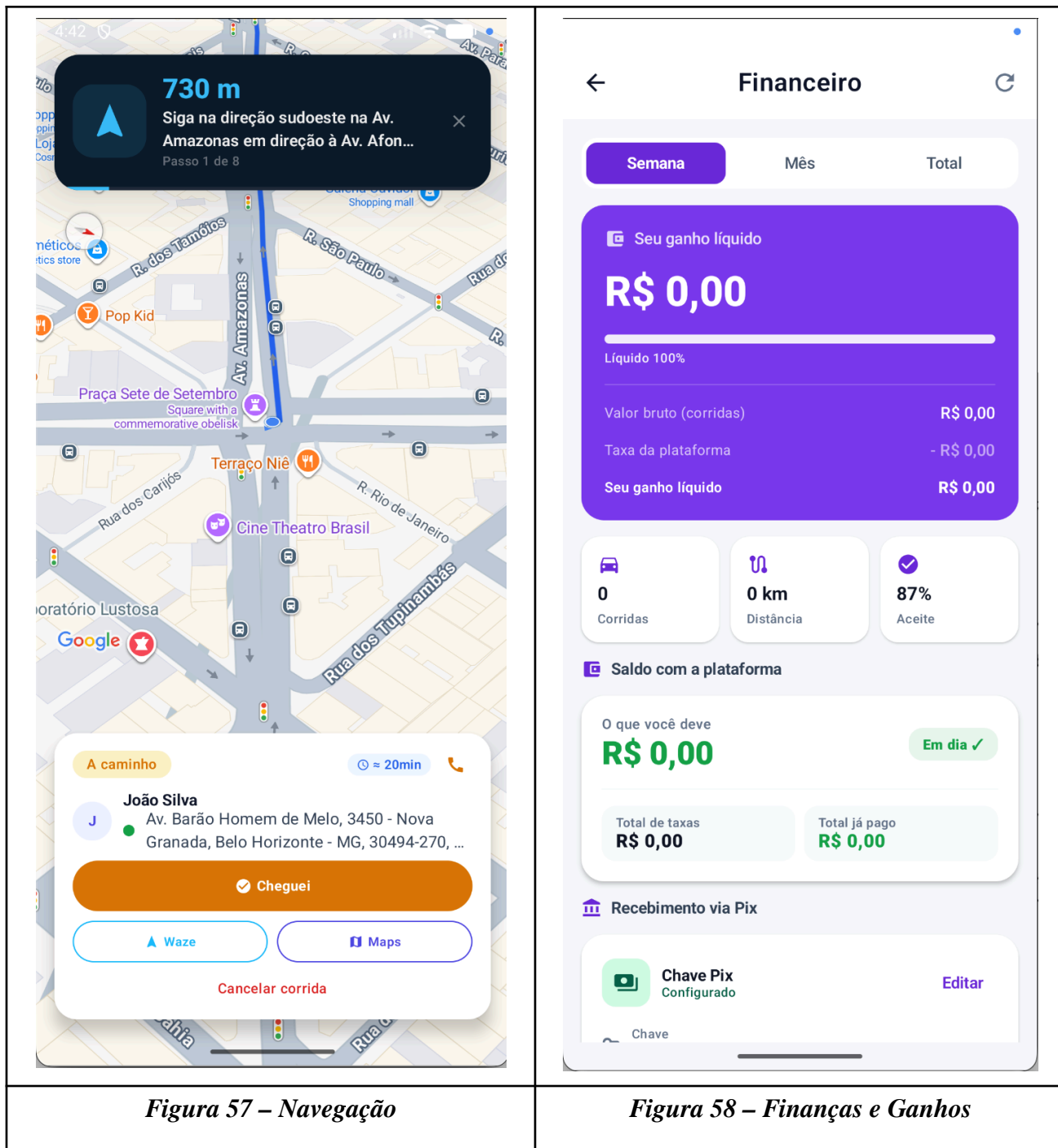
4.3 Módulo do Motorista

Após autenticar-se, o motorista é direcionado ao painel principal (Figura 55), que exibe os ganhos do dia, número de corridas realizadas, avaliação média, meta diária e o *status* de plataforma. O botão "Iniciar Turno" alterna o estado para *online*, tornando o motorista disponível para receber solicitações de corrida.

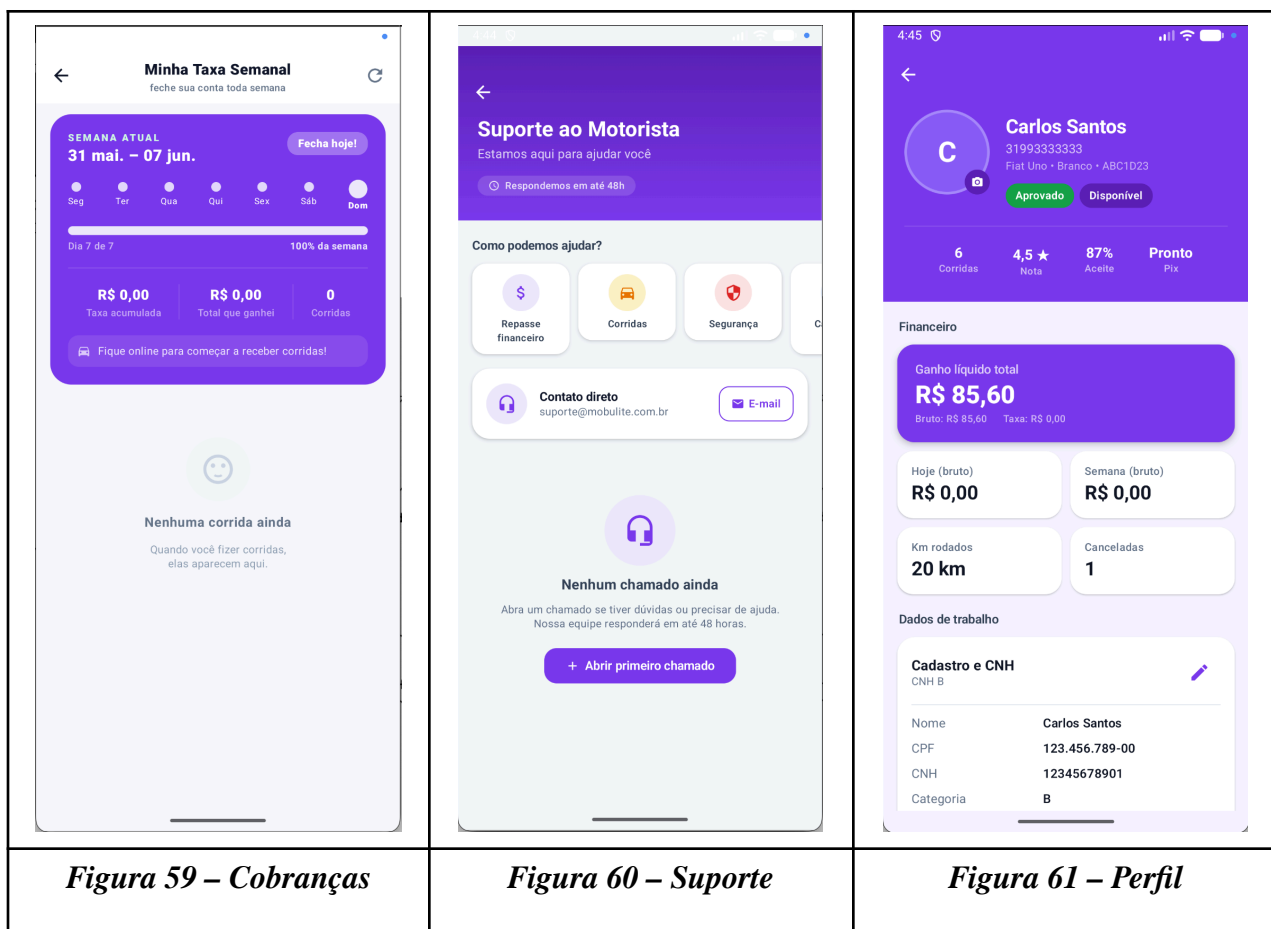
Quando uma nova solicitação é despachada pelo sistema, o motorista recebe uma notificação e visualiza a tela de Nova Corrida (Figura 56), com os dados do passageiro, endereço de origem, destino e valor estimado, podendo aceitar ou recusar.



Ao aceitar a corrida, o motorista visualiza a tela de Navegação (Figura 57) com o mapa e rota até o passageiro, além dos botões de ação "Cheguei", "Iniciar viagem" e "Finalizar viagem". A Figura 58 apresenta a tela de Finanças, com histórico de corridas finalizadas, ganhos diários e meta da semana.

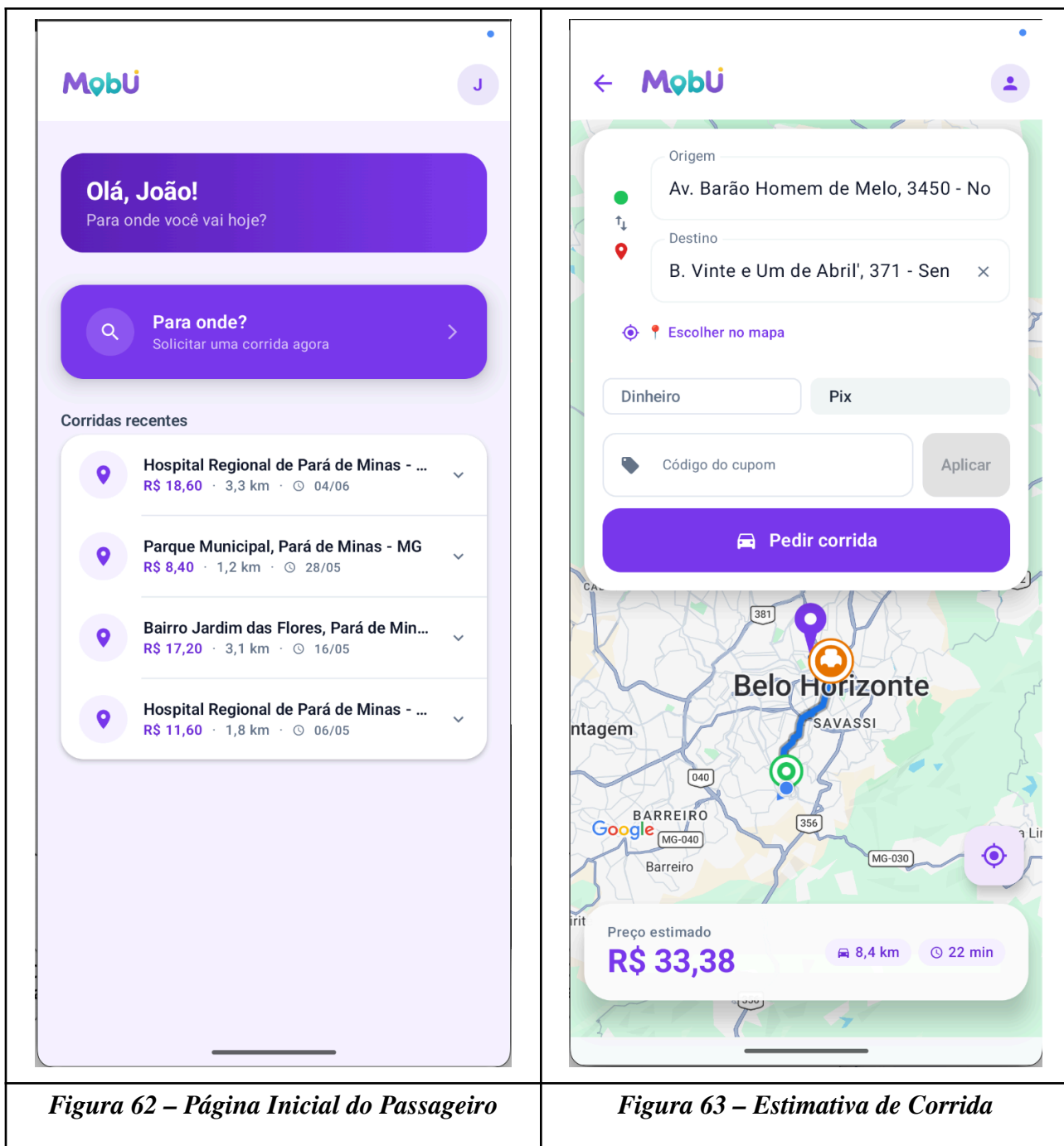


O motorista também tem acesso às telas de Cobranças (Figura 59), com o saldo devedor à plataforma e histórico de liquidações, Suporte (Figura 60) para abertura de chamados, e Perfil (Figura 61) com dados pessoais e do veículo.



4.4 Módulo do Passageiro

A Figura 62 representa a tela principal do passageiro, exibindo o mapa com a localização atual, o histórico de corridas recentes e o campo para informar o destino. Após selecionar o destino, o sistema calcula a rota e exibe a tela de Estimativa de Corrida (Figura 63) com o valor previsto, tempo estimado de chegada do motorista, a seleção do método de pagamento (PIX ou dinheiro) e as opções para confirmar ou cancelar a solicitação.



Após confirmar a solicitação e um motorista aceitar a corrida, o passageiro visualiza a tela de Acompanhamento (Figura 64), que exibe em tempo real a posição do motorista no mapa, as informações do veículo e o botão de cancelamento.

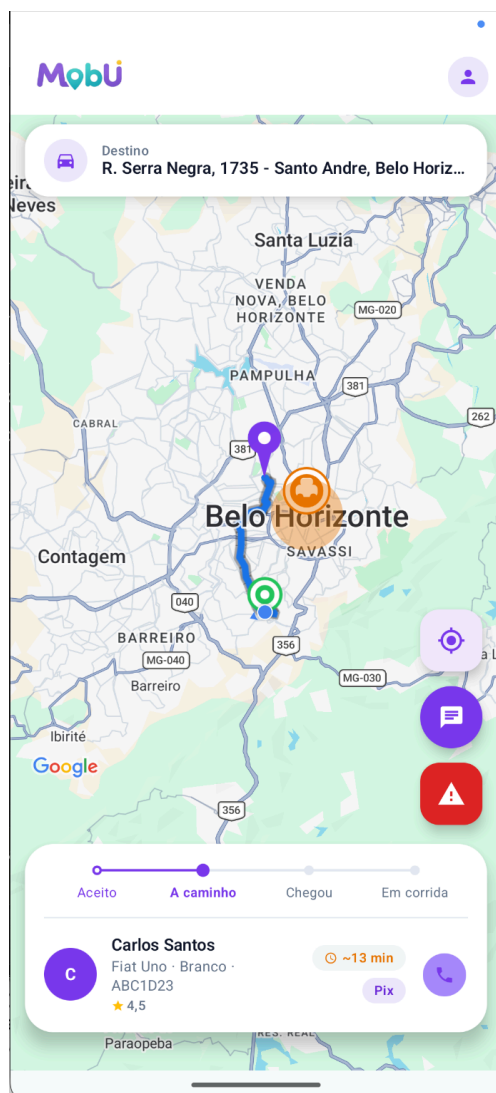
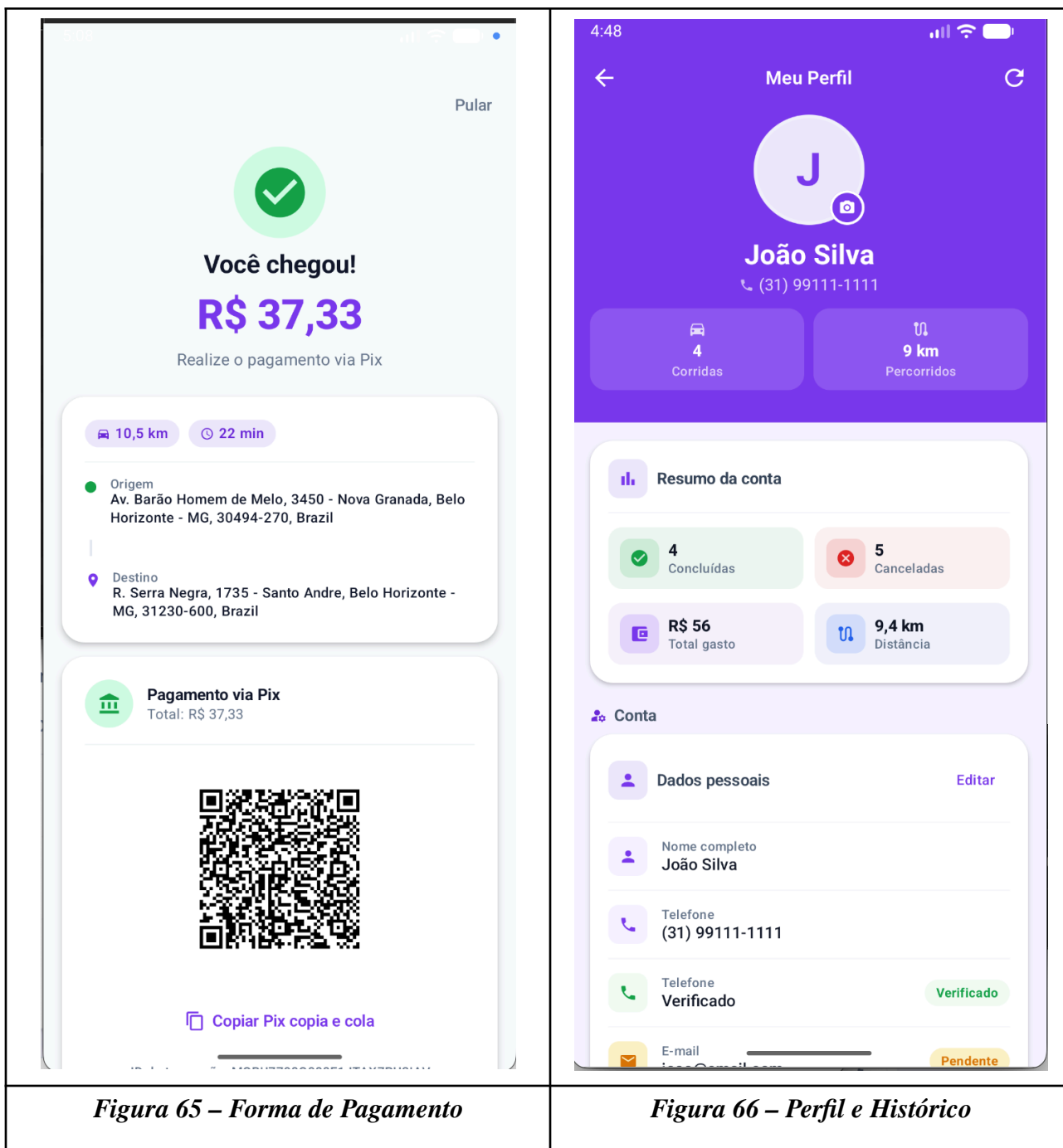


Figura 64 – Acompanhamento da Corrida

Ao término da corrida, o passageiro é redirecionado à tela de Pagamento (Figura 65), que exibe o resumo da viagem e confirma o pagamento no método previamente selecionado. A Figura 66 apresenta a tela de Perfil e Histórico, com acesso aos dados pessoais, histórico de viagens anteriores e opção de encerrar a sessão.



5. Glossário e Modelos de Dados

Esta seção tem como objetivo definir o glossário de termos técnicos e de domínio utilizados no sistema MobU, bem como descrever em detalhes o modelo de dados implementado. A Tabela 14

apresenta o glossário com os principais conceitos da aplicação. As Tabelas 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26 e 27 descrevem, respectivamente, as entidades *User*, *DriverProfile*, *Ride*, *PricingConfig*, *OperationRegion*, *Review*, *ChatMessage*, *SupportTicket*, *PromoCode*, *DriverPaymentRequest*, *DriverSettlement*, *RideAuditLog* e *SystemConfig*, com seus campos e tipos conforme definido no *schema Prisma* do sistema. Por fim, a Tabela 28 consolida as enumerações utilizadas no banco de dados.

Termo	Definição
MobU	Aplicação de transporte urbano por aplicativo desenvolvida como objeto deste trabalho, composta por <i>app</i> Android para passageiros e motoristas, API NestJS e painel administrativo Next.js.
Passageiro	Usuário com perfil <code>PASSENGER</code> que solicita corridas pelo aplicativo móvel.
Motorista	Usuário com perfil <code>DRIVER</code> cadastrado, aprovado e apto a aceitar e executar corridas.
Administrador	Usuário com perfil <code>ADMIN</code> que acessa o painel web para gerenciar usuários, corridas, precificações e pagamentos.
Corrida	Entidade central do sistema que representa a solicitação de transporte desde a requisição do passageiro até a conclusão ou cancelamento.
Status da corrida	Estado atual de uma corrida, podendo ser: <code>PENDING_DRIVER</code> , <code>ACCEPTED</code> , <code>DRIVER_ARRIVING</code> , <code>DRIVER_ARRIVED</code> , <code>IN_PROGRESS</code> , <code>FINISHED</code> ou <code>CANCELED</code> .
Perfil do motorista	Conjunto de dados complementares ao usuário motorista, incluindo CNH, veículo, localização em tempo real e chave Pix.

Aprovação de motorista	Processo pelo qual o administrador analisa e aprova ou rejeita o cadastro de um motorista. Estados: PENDING, APPROVED, REJECTED.
Configuração o de precificação	Conjunto de parâmetros tarifários (tarifa base, valor por km, valor por minuto, multiplicador de demanda) aplicados ao cálculo de corridas.
Região de operação	Área geográfica delimitada (centro + raio em metros) na qual o sistema pode operar, associada a uma configuração de precificação específica.
Avaliação	Nota (1–5) e comentário opcional atribuídos pelo passageiro ao motorista ao final de uma corrida.
Chat de corrida	Canal de mensagens em tempo real entre passageiro e motorista durante uma corrida ativa, persistido no banco de dados.
Ticket de suporte	Solicitação de ajuda aberta por um usuário relacionada a uma corrida ou problema no aplicativo, gerenciada pelo administrador.
Código promocional	Código alfanumérico que concede desconto percentual ou fixo em corridas, com controle de usos máximos e validade.
Rejeição de corrida	Registro de que um motorista recusou uma corrida pendente, impedindo que a mesma corrida seja ofertada novamente ao mesmo motorista.
Solicitação de pagamento	Pedido formal de repasse financeiro feito pelo motorista ao administrador, referente ao saldo acumulado de corridas.
Liquidação	Confirmação pelo administrador de que o repasse financeiro ao motorista foi realizado, vinculada a uma solicitação de pagamento.

Log de auditoria	Registro imutável de cada ação significativa sobre uma corrida (aceitação, início, fim, cancelamento, etc.), com identificação do ator e metadados.
FCM Token	Identificador único gerado pelo Firebase Cloud Messaging que permite o envio de notificações push ao dispositivo do usuário.
Pix	Sistema de pagamento instantâneo do Banco Central do Brasil, suportado como método de pagamento de corridas e repasse a motoristas.
Tarifa base	Valor fixo cobrado no início de toda corrida, independentemente da distância ou duração.
Taxa de plataforma	Percentual retido pela plataforma MobU sobre o valor total de cada corrida antes do repasse ao motorista.
Multiplicador de demanda	Fator aplicado à tarifa em períodos de alta demanda (<i>surge pricing</i>). Valor 1,0 indica sem acréscimo.
NestJS	<i>Framework</i> Node.js baseado em TypeScript utilizado para construção da API REST do sistema MobU.
Prisma ORM	Object-Relational Mapper utilizado para modelagem, migração e acesso ao banco de dados PostgreSQL a partir da API.
PostgreSQL	Sistema gerenciador de banco de dados relacional utilizado como camada de persistência do sistema MobU.
FCM (Firebase Cloud Messaging)	Serviço do Google utilizado exclusivamente para envio de notificações push aos dispositivos Android dos usuários.

JWT (JSON Web Token)	Padrão de token utilizado para autenticação e autorização nas requisições à API após o <i>login</i> .
WebSocket	Protocolo de comunicação bidirecional utilizado para atualizações em tempo real de localização do motorista e eventos de corrida.
API REST	Interface de programação baseada no protocolo HTTP que expõe os recursos do sistema MobU para os clientes (<i>app</i> Android e painel web).
UUID / CUID	Formatos de identificadores únicos utilizados como chave primária das entidades do banco de dados.
CNH	Carteira Nacional de Habilitação, documento obrigatório para cadastro de motoristas na plataforma.
Jetpack Compose	Toolkit declarativo do Android utilizado para construção das interfaces do aplicativo móvel MobU.

Tabela 14 – Glossário de termos utilizados no sistema MobU

Campo	Tipo	Descrição
id	String (UUID)	Identificador único do usuário.
phone	String	Número de telefone único, utilizado como <i>login</i> .
name	String	Nome completo do usuário.

email	String	Endereço de e-mail único do usuário.
passwordHash	String	Hash bcrypt da senha definida no cadastro.
role	Enum UserRole	Perfil do usuário: PASSENGER, DRIVER ou ADMIN.
profilePhotoUrl	String	URL da foto de perfil.
fcmToken	String	Token do Firebase Cloud Messaging para notificações push.
blocked	Boolean	Indica se o usuário está bloqueado (padrão: false).
blockedAt	DateTime	Data e hora em que o usuário foi bloqueado.
phoneVerifiedAt	DateTime	Data de verificação do telefone.
emailVerifiedAt	DateTime	Data de verificação do e-mail.
createdAt	DateTime	Data e hora de criação do registro.

Tabela 15 – Entidade User

Campo	Tipo	Descrição
-------	------	-----------

id	String (CUID)	Identificador único do perfil.
userId	String	Referência ao usuário proprietário do perfil (único).
approvalStatus	Enum DriverApprovalStatus	Estado de aprovação: PENDING, APPROVED ou REJECTED.
approvedAt	DateTime	Data de aprovação pelo administrador.
rejectionReason	String	Motivo da rejeição do cadastro.
online	Boolean	Indica se o motorista está <i>online</i> no momento.
available	Boolean	Indica se o motorista está disponível para aceitar corridas.
currentLat	Float	Latitude atual do motorista (atualizada em tempo real).
currentLng	Float	Longitude atual do motorista.
cpf	String	CPF do motorista.
cnhNumber	String	Número da CNH.
cnhCategory	String	Categoria da CNH (ex.: B).

<code>cnhExpiresAt</code>	<code>DateTime</code>	Data de validade da CNH.
<code>cnhImageUrl</code>	<code>String</code>	URL da imagem da CNH enviada pelo motorista.
<code>vehicleModel</code>	<code>String</code>	Modelo do veículo (ex.: Onix).
<code>vehiclePlate</code>	<code>String</code>	Placa do veículo.
<code>vehicleColor</code>	<code>String</code>	Cor do veículo.
<code>vehicleYear</code>	<code>Int</code>	Ano de fabricação do veículo.
<code>vehicleCapacity</code>	<code>Int</code>	Capacidade de passageiros do veículo.
<code>hasEar</code>	<code>Boolean</code>	Indica se o motorista possui credencial EAR (especial).
<code>pixKey</code>	<code>String</code>	Chave Pix do motorista para recebimento de repasses.
<code>pixQrPayload</code>	<code>String</code>	Payload do QR Code Pix gerado.
<code>pixQrCodeUrl</code>	<code>String</code>	URL da imagem do QR Code Pix.
<code>profilePhotoUrl</code>	<code>String</code>	URL da foto de perfil do motorista.

createdAt	DateTime	Data de criação do perfil.
updatedAt	DateTime	Data da última atualização.

Tabela 16 – Entidade DriverProfile

Campo	Tipo	Descrição
id	String (CUID)	Identificador único da corrida.
passengerId	String	Referência ao usuário passageiro.
driverId	String	Referência ao usuário motorista (preenchido após aceitação).
status	Enum RideStatus	Estado atual da corrida (ver enumeração RideStatus).
originLat / originLng	Float	Coordenadas geográficas do ponto de origem.
destLat / destLng	Float	Coordenadas geográficas do destino.
originAddress	String	Endereço textual do ponto de origem.

destinationAddress	String	Endereço textual do destino.
distanceMeters	Int	Distância estimada em metros.
durationSeconds	Int	Duração estimada em segundos.
estimatedFareCents	Int	Valor estimado da corrida em centavos.
driverReceivableCents	Int	Valor a receber pelo motorista após desconto da taxa de plataforma.
platformFeeCents	Int	Valor retido pela plataforma em centavos.
pricingConfigId	String	Configuração de precificação aplicada à corrida.
paymentMethod	Enum PaymentMethod	Método de pagamento: CASH ou PIX.
paymentStatus	Enum PaymentStatus	Status do pagamento: PENDING, RECEIVED, NOT_RECEIVED ou CANCELED.
paymentPixPayload	String	Payload do QR Code Pix para pagamento da corrida.
paymentTxId	String	ID da transação Pix.

shareToken	String	Token único para compartilhamento de localização da corrida.
acceptedAt	DateTime	Data/hora em que o motorista aceitou a corrida.
driverArrivingAt	DateTime	Data/hora em que o motorista iniciou o deslocamento ao ponto de origem.
driverArrivedAt	DateTime	Data/hora em que o motorista chegou ao ponto de origem.
startedAt	DateTime	Data/hora de início da corrida.
finishedAt	DateTime	Data/hora de conclusão da corrida.
canceledAt	DateTime	Data/hora de cancelamento.
createdAt	DateTime	Data/hora de criação da solicitação.
updatedAt	DateTime	Data/hora da última atualização.

Tabela 17 – Entidade Ride

Campo	Tipo	Descrição
id	String (CUID)	Identificador único da configuração.

name	String	Nome descritivo da configuração (ex.: "Tarifa padrão").
isActive	Boolean	Indica se a configuração está em uso.
regionId	String	Região de operação associada.
baseFareCents	Int	Tarifa base em centavos (padrão: 500).
perKmCents	Int	Valor por quilômetro em centavos (padrão: 200).
perMinuteCents	Int	Valor por minuto em centavos (padrão: 50).
minimumFareCents	Int	Tarifa mínima em centavos (padrão: 800).
bookingFeeCents	Int	Taxa de reserva em centavos (padrão: 0).
surgeMultiplier	Float	Multiplicador de demanda (padrão: 1,0 = sem acréscimo).
platformFeePercent	Float	Percentual de taxa da plataforma (padrão: 20%).
currency	String	Moeda utilizada (padrão: "BRL").
createdAt / updatedAt	DateTime	Datas de criação e atualização.

Tabela 18 – Entidade PricingConfig

Campo	Tipo	Descrição
id	String (CUID)	Identificador único da região.
name	String	Nome da região (ex.: "Centro de Belo Horizonte").
city	String	Cidade da região.
active	Boolean	Indica se a região está ativa.
centerLat / centerLng	Float	Coordenadas do centro geográfico da região.
radiusMeters	Int	Raio de cobertura em metros.
createdAt / updatedAt	DateTime	Datas de criação e atualização.

Tabela 19 – Entidade OperationRegion

Campo	Tipo	Descrição
id	String (CUID)	Identificador único da avaliação.
rideId	String	Corrida avaliada (único — uma corrida possui no máximo uma avaliação).

passengerId	String	Passageiro que realizou a avaliação.
driverId	String	Motorista avaliado.
rating	Int	Nota de 1 a 5.
comment	String	Comentário textual da avaliação.
createdAt	DateTime	Data de criação.

Tabela 20 – Entidade Review

Campo	Tipo	Descrição
id	String (CUID)	Identificador único da mensagem.
rideId	String	Corrida à qual a mensagem pertence.
senderId	String	ID do usuário remetente.
senderRole	String	Papel do remetente: "PASSENGER" ou "DRIVER".
content	String	Conteúdo textual da mensagem.

sentAt	DateTime	Data/hora de envio.
readAt	DateTime	Data/hora de leitura pelo destinatário.

Tabela 21 – Entidade ChatMessage

Campo	Tipo	Descrição
id	String (CUID)	Identificador único do <i>ticket</i> .
creatorId	String	Usuário que abriu o <i>ticket</i> .
rideId	String	Corrida relacionada ao <i>ticket</i> (se aplicável).
type	Enum SupportTicketType	Categoria: PAYMENT, RIDE_CANCELLATION, SAFETY, APP_ISSUE ou OTHER.
status	Enum SupportTicketStatus	Estado: OPEN, IN_REVIEW, RESOLVED ou CLOSED.
subject	String	Assunto do <i>ticket</i> .
description	String	Descrição detalhada do problema.
resolution	String	Descrição da resolução pelo administrador.

<code>closedAt</code>	<code>DateTime</code>	Data/hora de encerramento do <i>ticket</i> .
<code>createdAt</code> / <code>updatedAt</code>	<code>DateTime</code>	Datas de criação e atualização.

Tabela 22 – Entidade *SupportTicket*

Campo	Tipo	Descrição
<code>id</code>	String (CUID)	Identificador único.
<code>code</code>	String	Código alfanumérico único (ex.: "PROMO10").
<code>discountPercent</code>	Int	Percentual de desconto (0–100).
<code>discountCents</code>	Int	Valor fixo de desconto em centavos.
<code>maxUses</code>	Int	Número máximo de utilizações (padrão: 1).
<code>usedCount</code>	Int	Número de vezes que o código foi utilizado.
<code>active</code>	Boolean	Indica se o código está ativo.
<code>expiresAt</code>	<code>DateTime</code>	Data de expiração do código.

createdAt	DateTime	Data de criação.
-----------	----------	------------------

Tabela 23 – Entidade PromoCode

Campo	Tipo	Descrição
id	String (CUID)	Identificador único.
driverId	String	Motorista solicitante.
amountCents	Int	Valor solicitado em centavos.
status	Enum PaymentRequestStatus	Estado: PENDING, CONFIRMED ou REJECTED.
notes	String	Observações do motorista.
receiptUrl	String	URL do comprovante de pagamento.
rejectionReason	String	Motivo de rejeição pelo administrador.
requestedAt	DateTime	Data/hora da solicitação.
reviewedAt	DateTime	Data/hora da revisão pelo administrador.

reviewedBy	String	ID do administrador que revisou.
------------	--------	----------------------------------

Tabela 24 – Entidade *DriverPaymentRequest*

Campo	Tipo	Descrição
id	String (UUID)	Identificador único.
driverId	String	Motorista beneficiário do repasse.
amountCents	Int	Valor repassado em centavos.
method	String	Método de repasse (padrão: "PIX").
notes	String	Observações do administrador.
paymentRequestId	String	Solicitação de pagamento vinculada (único).
settledBy	String	ID do administrador que confirmou o repasse.
settledAt	DateTime	Data/hora do repasse.

Tabela 25 – Entidade *DriverSettlement*

Campo	Tipo	Descrição
id	String (CUID)	Identificador único do log.
rideId	String	Corrida à qual o log se refere.
actorUserId	String	ID do usuário que realizou a ação (nulo para ações automáticas do sistema).
action	String	Descrição da ação registrada (ex.: "RIDE_ACCEPTED", "RIDE_STARTED").
metadata	JSON	Dados adicionais da ação em formato JSON.
createdAt	DateTime	Data/hora do registro.

Tabela 26 – Entidade RideAuditLog

Campo	Tipo	Descrição
key	String	Chave de identificação da configuração (chave primária).
value	String	Valor da configuração.
updatedAt	DateTime	Data da última atualização.

Tabela 27 – Entidade SystemConfig

Enumeração	Valores	Descrição
UserRole	PASSENGER, DRIVER, ADMIN	Perfil de acesso do usuário no sistema.
RideStatus	PENDING_DRIVER, ACCEPTED, DRIVER_ARRIVING, DRIVER_ARRIVED, IN_PROGRESS, FINISHED, CANCELED	Estados possíveis de uma corrida.
PaymentMethod	CASH, PIX	Método de pagamento da corrida.
PaymentStatus	PENDING, RECEIVED, NOT_RECEIVED, CANCELED	Status do pagamento da corrida.
DriverApprovalStatus	PENDING, APPROVED, REJECTED	Estado de aprovação do cadastro de motorista.
PaymentRequestStatus	PENDING, CONFIRMED, REJECTED	Estado de uma solicitação de pagamento do motorista.
SupportTicketType	PAYMENT, RIDE_CANCELLATION, SAFETY, APP_ISSUE, OTHER	Categoria do <i>ticket</i> de suporte.
SupportTicketStatus	OPEN, IN_REVIEW, RESOLVED, CLOSED	Estado do <i>ticket</i> de suporte.

VerificationChannel	SMS, EMAIL	Canal utilizado para desafios de verificação de identidade.
---------------------	------------	---

Tabela 28 – Enumerações do banco de dados

Da mesma forma, a Figura 67 tem como objetivo representar a camada de banco de dados da aplicação por meio de um Diagrama Entidade-Relacionamento (DER). Nesse diagrama, é possível identificar todas as entidades implementadas, tais como Usuário, Motorista, Veículo, Corrida, Localização, Pagamento e Avaliação, bem como a maneira pela qual elas se relacionam.

O modelo de dados do MobU foi estruturado para garantir consistência, integridade e rastreabilidade das informações. Cada corrida é associada a um passageiro e motorista, contendo dados de origem, destino, valor e *status* operacional. Os pagamentos armazenam o método utilizado (PIX ou dinheiro), o *status* da transação e os dados de confirmação. Já as avaliações permitem o registro da nota e do comentário referente à experiência de corrida.

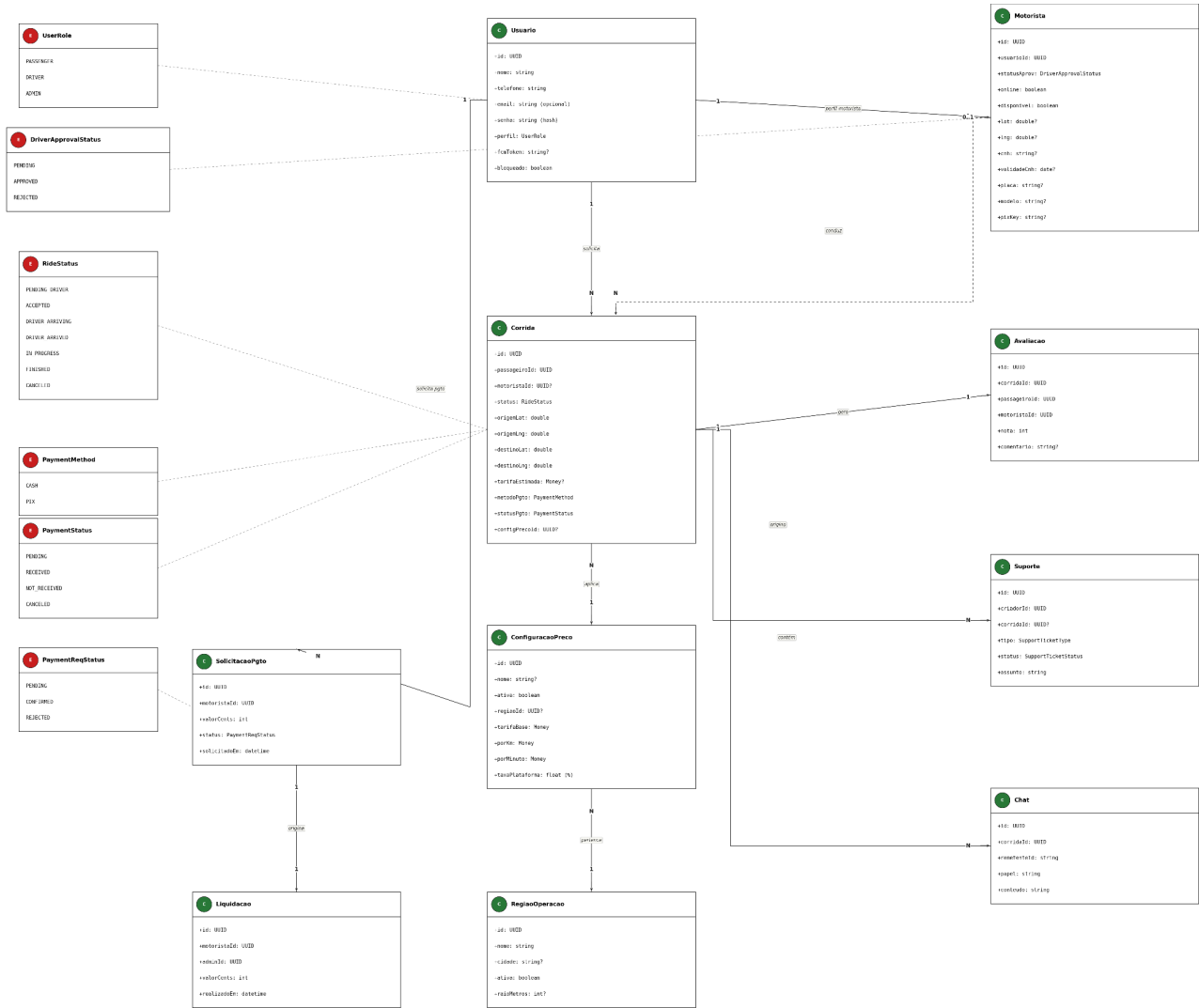


Figura 67. Diagrama de Entidade e Relacionamento.

6. Casos de Teste

Esta seção tem como objetivo apresentar os testes de aceitação planejados para o sistema MobU, garantindo que as funcionalidades previstas no Documento de Visão sejam devidamente verificadas. Os testes aqui definidos serão implementados futuramente e têm como propósito assegurar que o sistema atenda às necessidades essenciais de seus usuários — passageiros e motoristas — permitindo o uso correto e seguro dos principais fluxos da aplicação.

Cada teste de aceitação é descrito por meio de uma tabela contendo quatro campos principais: identificador, pré-condição, ações e resultados esperados. O identificador permite referenciar o teste de maneira única. A pré-condição define o estado necessário para que o teste possa ser iniciado. As ações descrevem passo a passo as interações do usuário na aplicação. Por fim, os resultados descrevem o comportamento esperado do sistema após a execução do teste.

Esses testes serão utilizados futuramente para validar a experiência do usuário, confirmar o correto funcionamento dos fluxos críticos e demonstrar que o sistema opera conforme os requisitos estabelecidos para a primeira entrega funcional do MobU. Todos os testes apresentados foram definidos com base nas necessidades identificadas durante o processo de levantamento de requisitos e serão executados conforme o progresso do desenvolvimento.

6.1 Testes de Aceitação

Os testes de aceitação têm como objetivo validar, da perspectiva do usuário, se cada necessidade levantada foi atendida pelo sistema MobU. Eles verificam o comportamento esperado das principais funcionalidades por meio de cenários práticos, garantindo que os fluxos críticos da aplicação funcionem de ponta a ponta. As subseções a seguir organizam esses testes por necessidade, descrevendo, em cada caso, a condição avaliada, os passos executados e o resultado esperado.

6.1.1 Necessidade 1: Passageiro solicitar corrida

A primeira necessidade prevê que o passageiro seja capaz de solicitar uma corrida informando origem e destino. Para essa necessidade, foram definidos três testes de aceitação, relacionados à criação da conta, à solicitação da corrida e à visualização do motorista no mapa.

Caso de Teste de Aceitação 1: Passageiro deve conseguir criar conta no MobU	
Identificador	TA1
Pré-condição	O passageiro não deve possuir cadastro prévio no MobU.
Ações	1. Inserir número de telefone. 2. Definir senha de acesso (mínimo 6 caracteres). 3. Informar nome completo e concluir o cadastro.

Resultados	Resultado esperado: a conta deve ser criada e o usuário deve ser redirecionado ao <i>dashboard</i> inicial. Resultado obtido: aprovado. O cadastro/autenticação foi validado pela <i>API</i> com telefone e senha (<i>BCrypt</i>), e o <i>login</i> foi confirmado no aplicativo <i>Android</i> com usuário QA.
-------------------	---

Tabela 29. Caso de teste de aceitação 1

Caso de Teste de Aceitação 2: Passageiro deve conseguir solicitar uma corrida	
Identificador	TA2
Pré-condição	O passageiro deve estar autenticado na aplicação.
Ações	1. Inserir origem e destino. 2. Confirmar os dados da corrida. 3. Selecionar forma de pagamento (dinheiro ou Pix <i>offline</i>). 4. Solicitar corrida.
Resultados	Resultado esperado: o sistema deve registrar a corrida e exibir o <i>status</i> “Aguardando motorista”.

Tabela 30. Caso de teste de aceitação 2

Caso de Teste de Aceitação 3: Passageiro deve conseguir acompanhar o motorista	
Identificador	TA3
Pré-condição	O motorista deve ter aceitado a corrida.
Ações	1. Abrir tela de acompanhamento. 2. Visualizar a posição do motorista no mapa em tempo real.
Resultados	Resultado esperado: o passageiro deve visualizar o motorista se movendo no mapa até o local de embarque. Resultado obtido: aprovado. Após o aceite da corrida, a atualização de localização do motorista foi persistida e consultada pelo passageiro durante o acompanhamento.

Tabela 31. Caso de teste de aceitação 3

6.1.2 Necessidade 2 — Motorista aceitar e realizar corrida

A segunda necessidade garante que o motorista seja capaz de receber, aceitar e concluir corridas, validando o funcionamento da API, os eventos e a comunicação entre os módulos.

Caso de Teste de Aceitação 4: Motorista aceitar corrida	
Identificador	TA4
Pré-condição	O motorista deve estar autenticado e marcado como “online”.
Ações	1. Receber notificação de corrida. 2. Visualizar origem e destino. 3. Aceitar corrida.
Resultados	Resultado esperado: a corrida passa para o estado “Aceita” e o motorista passa para o modo de navegação. Resultado obtido: aprovado. Motorista online visualizou a corrida pendente, aceitou a solicitação e a API vinculou a corrida ao motorista com mudança de estado.

Tabela 32. Caso de teste de aceitação 4

Caso de Teste de Aceitação 5: Motorista finalizar corrida	
Identificador	TA5
Pré-condição	A corrida deve estar em andamento.
Ações	1. Marcar chegada ao destino. 2. Finalizar corrida. 3. Confirmar valor final com o passageiro.
Resultados	Resultado esperado: a corrida deve ser finalizada e registrada no histórico do motorista e do passageiro. Resultado obtido: aprovado. O fluxo de status chegando, chegou, em andamento e finalizada foi concluído, e a corrida apareceu nos históricos de motorista e passageiro.

Tabela 33. Caso de teste de aceitação 5

Caso de Teste de Aceitação 6: O Motorista deve ser capaz de gerenciar seu <i>status</i> (online/offline)

Identificador	TA6
Pré-condição	O motorista deve estar autenticado na aplicação.
Ações	1. Acessar o painel inicial. 2. Selecionar o botão “Ficar <i>Online</i> ”.
Resultados	Resultado esperado: o motorista passa a receber solicitações de corrida e seu <i>status</i> é atualizado no sistema.

Tabela 34. Caso de teste de aceitação 6

6.1.3 Necessidade 3 — Pagamento via PIX *Off-line*

Esta necessidade refere-se ao processo de geração de QR Code estático utilizando chave Pix previamente cadastrada no banco de dados, sem uso de API de instituição financeira.

Caso de Teste de Aceitação 7: Gerar QR Code Pix estático	
Identificador	TA7
Pré-condição	O motorista deve possuir chave Pix cadastrada no sistema
Ações	1. Ao solicitar a corrida, selecionar PIX como método de pagamento. 2. Confirmar a solicitação e aguardar o motorista finalizar a corrida. 3. Visualizar o QR <i>Code</i> gerado com o valor da corrida e o <i>txId</i> associado.
Resultados	Resultado esperado: o QR Code deve ser exibido corretamente e pode ser pago pelo aplicativo bancário do passageiro. Resultado obtido: aprovado. A API gerou payload Pix/QR Code estático com valor da corrida e <i>txId</i> associado ao pagamento.

Tabela 35. Caso de teste de aceitação 7

Caso de Teste de Aceitação 8: Registrar pagamento não confirmado	
Identificador	TA8
Pré-condição	A corrida deve estar finalizada e aguardando validação de pagamento.
Ações	1. Selecionar corrida finalizada. 2. Informar “Pagamento não recebido”.
Resultados	Resultado esperado: o sistema marca o pagamento como “Pendente”, registrando data e motorista responsável. Resultado obtido: aprovado. Ao marcar pagamento não recebido, a API registrou a pendência/não confirmação mantendo rastreabilidade para validação posterior.

Tabela 36. Caso de teste de aceitação 8

Caso de Teste de Aceitação 9: Visualizar comprovante do PIX	
Identificador	TA9
Pré-condição	O pagamento da corrida deve estar marcado como “Pago”.
Ações	1. Abrir detalhes da corrida. 2. Acessar a área de pagamentos.
Resultados	Resultado esperado: o comprovante (txid + dados da cobrança) é exibido ao motorista e ao passageiro. Resultado obtido: aprovado. Após confirmação do pagamento, os detalhes da corrida exibiram txId e payload Pix para consulta no histórico.

Tabela 37. Caso de teste de aceitação 9

6.1.4 Necessidade 4 — Administração de Usuários e Monitoramento do Sistema

Esta necessidade refere-se às funcionalidades destinadas ao administrador da plataforma MobU, permitindo a gestão centralizada de usuários e o acompanhamento operacional do sistema. O administrador deve ser capaz de visualizar, monitorar e realizar ações de controle sobre contas de passageiros e motoristas, bem como acessar relatórios consolidados de uso da aplicação.

Caso de Teste de Aceitação 10: O Administrador deve gerenciar motoristas e passageiros	
Identificador	TA10
Pré-condição	O administrador deve estar autenticado no painel web.
Ações	1. Abrir a aba “Usuários”.
Resultados	Resultado esperado: o sistema exibe lista com passageiros e motoristas cadastrados. Resultado obtido: aprovado. Painel/admin/API retornaram listas de passageiros e motoristas cadastrados, incluindo os usuários QA criados na execução.

Tabela 38. Caso de teste de aceitação 10

Caso de Teste de Aceitação 11: Bloquear usuário	
Identificador	TA11
Pré-condição	Usuário selecionado deve estar ativo no sistema.
Ações	1. Selecionar usuário. 2. Clicar em “Bloquear usuário”
Resultados	Resultado esperado: o usuário fica impedido de acessar o sistema e seu <i>status</i> é marcado como “Bloqueado”.

Tabela 39. Caso de teste de aceitação 11

Caso de Teste de Aceitação 12: Visualizar relatórios de corridas	
Identificador	TA12
Pré-condição	O sistema deve conter corridas concluídas.
Ações	1. Acessar a aba “Relatórios”. 2. Selecionar período desejado.
Resultados	Resultado esperado: um relatório com número de corridas, ganhos e estatísticas é exibido ao administrador. Resultado obtido: aprovado. O relatório administrativo

	retornou indicadores consolidados de corridas, usuários e pagamentos após o fluxo concluído.
--	--

Tabela 40. Caso de teste de aceitação 12

6.2 Testes de Integração

Nesta seção serão descritos os testes de integração previstos para o sistema MobU. Os testes de integração serão responsáveis por verificar se os módulos principais da solução — aplicativo móvel, API MobU e serviços externos (API de Mapas/Geo, serviço de notificações push e futuro *gateway* de pagamento) — se comunicam corretamente, trocando dados de forma consistente e mantendo o fluxo das corridas do início ao fim.

Cada caso de teste de integração será documentado por meio de uma tabela contendo: identificador, sistemas envolvidos, interface utilizada, pré-condições, entradas e resultados esperados. A partir desses casos, será possível validar cenários críticos como autenticação de usuário, solicitação de corrida com cálculo de rota, envio de notificações para motoristas e registro de pagamentos.

Como estratégia geral, pretende-se adotar uma abordagem top-down. Inicialmente, serão implementadas e testadas as integrações entre a MobU API e os serviços externos, utilizando *mocks* para simular respostas da API de Mapas/Geo, do serviço de notificações push (FCM). Em um segundo momento, o aplicativo móvel será conectado à API real, permitindo a execução de testes ponta a ponta. A automatização desses testes deverá ser feita por meio de suítes de testes da API e, quando possível, por testes de integração no aplicativo móvel, garantindo que regressões possam ser detectadas de forma rápida ao longo do desenvolvimento.

A Tabela 41 apresenta o caso de teste de integração que valida o processo de autenticação de usuários no MobU. Esse teste verifica a comunicação entre o aplicativo móvel e a API, garantindo que credenciais válidas resultem em geração de token e acesso ao perfil correspondente.

Identificador	TI1 – Autenticação de usuário
Sistemas Envolvidos	<i>Mobile App</i> (Passageiro/Motorista) e MobU API
Interface	Requisição HTTP REST (POST <i>/auth/login</i>) e armazenamento do token JWT no aplicativo.
Pré-condição	Usuário cadastrado no banco de dados e servidor da API em execução.
Entradas	1. Enviar telefone/e-mail e senha válidos a partir da tela de <i>login</i> .

Resultados	Resultado esperado: API deve retornar código 200, token JWT e dados básicos do usuário; o aplicativo deve armazenar o token e redirecionar para a tela inicial do perfil correspondente. Resultado obtido: aprovado. A API retornou 200 com JWT e dados do usuário, e o aplicativo Android autenticou usuário QA, abrindo a tela inicial.
-------------------	---

Tabela 41. Caso de teste de integração 1

A Tabela 42 descreve o teste de integração relacionado à solicitação de corrida com cálculo de rota. O teste avalia a interação entre o *Mobile App*, a MobU API e a API de Mapas, assegurando que origem e destino sejam processados corretamente e que o passageiro visualize estimativas de percurso e valor.

Identificador	TI2 – Solicitação de corrida com rota
Sistemas Envolvidos	<i>Mobile App</i> (Passageiro), MobU API e API de Mapas/Geo
Interface	HTTP REST entre <i>App</i> e MobU API (POST /rides/estimate e POST /rides); integração com serviço de mapas/geo para estimativa de distância, tempo e valor.
Pré-condição	Passageiro autenticado; localização de origem habilitada no dispositivo.
Entradas	1. Informar origem e destino no aplicativo/API dentro da região ativa. 2. Enviar estimativa (POST /rides/estimate). 3. Criar corrida (POST /rides) com coordenadas de origem e destino.
Resultados	Resultado esperado: API de Mapas/Geo deve retornar rota e distância; MobU API deve calcular valor estimado e tempo de viagem e responder ao <i>app</i> com os dados consolidados.

Tabela 42. Caso de teste de integração 2

A Tabela 43 apresenta o caso de teste que verifica o envio de notificações push ao motorista quando uma nova corrida é confirmada. Esse teste garante o funcionamento integrado entre a MobU API, o serviço FCM e o aplicativo do motorista.

Identificador	TI3 – Notificação de nova corrida
Sistemas Envolvidos	MobU API, serviço de notificações push (FCM) e <i>Mobile App</i> (Motorista)

Interface	Registro de <i>token</i> FCM via <i>PATCH /auth/fcm-token</i> (endpoint previsto, não implementado no <i>AuthController</i> atual); <i>HTTP/HTTPS</i> entre <i>MobU API</i> e <i>FCM</i> ; canal de <i>Push Notification</i> entre <i>FCM</i> e <i>App</i> .
Pré-condição	Motorista autenticado e marcado como online ; token de notificação registrado na API.
Entradas	1. Passageiro confirma solicitação de corrida. 2. MobU API envia mensagem de nova corrida ao FCM com dados resumidos (origem, destino, valor estimado).
Resultados	Resultado esperado: motorista deve receber uma notificação push no dispositivo e abrir a tela de detalhes da corrida ao tocar na notificação. Resultado obtido: parcial. O token FCM foi registrado e persistido, e o backend acionou o fluxo de notificação; o recebimento visual da push notification não foi comprovado em emulador local por depender da infraestrutura externa do Firebase.

Tabela 43. Caso de teste de integração 3

A Tabela 44 descreve o teste de integração destinado a validar o processo de registro de pagamento PIX *offline*, sem API bancária. O teste garante que a confirmação enviada pelo motorista seja corretamente recebida e registrada pela API.

Identificador	TI4 – Registro de pagamento PIX <i>off-line</i>
Sistemas Envolvidos	<i>Mobile App</i> (Passageiro/Motorista) e MobU API
Interface	Módulo de pagamentos planejado. Endpoints <i>POST /payments/rides/{rideId}/confirm</i> e <i>POST /payments/rides/{rideId}/not-received</i> ainda não implementados no <i>backend</i> . A validação foi realizada diretamente via banco de dados.
Pré-condição	Corrida finalizada e QR Code PIX já gerado pela API; passageiro realizou o pagamento utilizando o aplicativo bancário externo.
Entradas	1. Motorista informa, no <i>app</i> , que o pagamento foi recebido. 2. <i>App</i> envia confirmação de pagamento à API com o identificador da corrida.
Resultados	Resultado esperado: <i>MobU API</i> deve atualizar o <i>status</i> do pagamento no histórico de corridas.

	Resultado obtido: parcial. A lógica de confirmação de pagamento foi verificada diretamente no banco de dados. Os endpoints <i>REST</i> dedicados ao módulo de pagamentos estão planejados para implementação futura.
--	--

Tabela 44. Caso de teste de integração 4

A Tabela 45 apresenta o teste de integração sobre a atualização do *status* do motorista entre *online* e *offline*. O teste assegura que o aplicativo acione a API corretamente e que o estado atualizado seja refletido no sistema.

Identificador	TI5 – Alternar <i>status online/off-line</i>
Sistemas Envolvidos	<i>Mobile App</i> (Motorista) e MobU API
Interface	Requisição HTTP REST (POST /driver/status).
Pré-condição	Motorista autenticado no aplicativo.
Entradas	1. Motorista aciona o botão “Ficar <i>online/off-line</i> ” na tela principal do <i>app</i> .
Resultados	Resultado esperado: API deve atualizar o <i>status</i> do motorista no banco de dados; o aplicativo deve receber confirmação e exibir o novo estado.

Tabela 45. Caso de teste de integração 5

A Tabela 46 descreve o teste que valida o fluxo de aceitação de corrida pelo motorista. O caso assegura que a API atualize o estado da corrida e que a API de Mapas forneça o trajeto adequado até o local de embarque.

Identificador	TI6 – Aceitar corrida e iniciar rota
Sistemas Envolvidos	<i>Mobile App</i> (Motorista), MobU API e API de Mapas/Geo
Interface	HTTP REST entre <i>App</i> e MobU API (GET /driver/rides/pending e POST /driver/rides/{rideId}/accept); integração com mapas/geo para rota de deslocamento.
Pré-condição	Corrida em estado <i>PENDING_DRIVER</i> para o motorista; motorista em modo <i>online</i> .

Entradas	1. Motorista consulta corridas pendentes. 2. Motorista clica em “Aceitar corrida”. 3. Aplicativo/API atualizam a corrida e solicitam dados de rota.
Resultados	Resultado esperado: <i>MobU API</i> atualiza a corrida para o estado <i>ACCEPTED</i> ; <i>API</i> de Mapas/Geo retorna a rota até a origem do passageiro. Resultado obtido: aprovado. A corrida no estado <i>PENDING_DRIVER</i> foi listada para o motorista <i>online</i> , aceita via <i>POST /driver/rides/{rideId}/accept</i> e vinculada corretamente ao motorista.

Tabela 46. Caso de teste de integração 6

A Tabela 47 apresenta o teste de integração que avalia o processo de finalização de corrida com pagamento em dinheiro. O teste confirma que a API atualiza o estado da corrida e registra o pagamento corretamente.

Identificador	TI7 – Finalizar corrida com pagamento em dinheiro
Sistemas Envolvidos	<i>Mobile App</i> (Motorista/Passageiro) e <i>MobU API</i>
Interface	Requisição <i>HTTP REST (PATCH /rides/:id/status)</i> com os valores <i>IN_PROGRESS</i> e <i>FINISHED</i> no corpo da requisição para progressão dos estados da corrida.
Pré-condição	Corrida em estado EM_ANDAMENTO ; método de pagamento configurado como DINHEIRO .
Entradas	1. Motorista avança os estados da corrida até o destino. 2. <i>App</i> envia para a <i>API</i> o identificador da corrida e confirma a finalização.
Resultados	Resultado esperado: <i>MobU API</i> deve atualizar o estado da corrida para <i>FINALIZADA/FINISHED</i> e registrar o pagamento conforme o método selecionado; histórico de passageiro e motorista deve exibir a corrida. Resultado obtido: aprovado. O fluxo de progressão e finalização foi concluído e os históricos de motorista e passageiro refletiram a corrida finalizada.

Tabela 47. Caso de teste de integração 7

A Tabela 48 descreve o caso de teste utilizado para validar a consulta de histórico de ganhos e corridas realizadas. O teste garante o funcionamento adequado da API ao retornar dados consolidados com base nos filtros selecionados pelo motorista.

Identificador	TI8 – Consultar histórico de corridas e ganhos
Sistemas Envolvidos	<i>Mobile App</i> (Motorista) e MobU API
Interface	Requisição <i>HTTP REST</i> (<i>GET /driver/rides/history</i>). Endpoint planejado; durante a validação, o histórico foi consultado via <i>GET /rides</i> com filtro por motorista.
Pré-condição	Motorista autenticado e com corridas finalizadas registradas no sistema.
Entradas	1. Motorista acessa a tela “Histórico e ganhos” e seleciona um período (ex.: última semana).
Resultados	Resultado esperado: MobU API deve retornar lista de corridas do período, valores individuais e total acumulado; o aplicativo deve exibir o extrato de ganhos. Resultado obtido: aprovado. A rota atual de histórico retornou corridas finalizadas e dados financeiros utilizados pelas telas de histórico/ganhos do motorista.

Tabela 48. Caso de teste de integração 8

A Tabela 49 apresenta o teste de integração referente ao módulo administrativo do sistema, verificando a geração de relatórios por meio de consultas agregadas realizadas pela API. O teste comprova que filtros selecionados pelo administrador resultam em indicadores consistentes para gestão.

Identificador	TI9 – Relatórios administrativos
Sistemas Envolvidos	Painel Web/Admin (ou módulo futuro), MobU API e SGBD
Interface	<i>HTTP REST</i> entre Painel Web/Admin e MobU API. Endpoints implementados: <i>GET /admin/drivers/pending</i> , <i>PATCH /admin/drivers/:id/approve</i> , <i>PATCH /admin/drivers/:id/reject</i> . Demais rotas (relatórios, bloqueio, listagem geral) são planejadas. Conexão <i>SQL</i> entre API e banco de dados.

Pré-condição	Usuário administrador autenticado; dados de corridas, pagamentos e usuários previamente cadastrados.
Entradas	1. Administrador autentica no painel. 2. Painel consulta usuários/relatórios e aplica filtros administrativos. 3. Painel envia ação de bloqueio quando necessário.
Resultados	Resultado esperado: <i>MobU API</i> executa consultas e retorna indicadores para o painel administrativo. Resultado obtido: parcial. Aprovação e rejeição de motoristas foram validadas via <i>API</i> . Relatórios, listagem geral de usuários e bloqueio administrativo foram verificados diretamente no banco de dados, pois os respectivos endpoints ainda não estão implementados no <i>AdminController</i> .

Tabela 49. Caso de teste de integração 9

6.2.1 Necessidade 5 — Cadastro e Aprovação de Motoristas

Esta necessidade contempla o fluxo de cadastro do motorista com envio de dados de habilitação (*CNH*) e o processo de aprovação ou rejeição pelo administrador da plataforma. O acesso ao sistema só é liberado após a aprovação.

Caso de Teste de Aceitação 13: Motorista deve conseguir criar conta com dados de habilitação	
Identificador	TA13
Pré-condição	O telefone não deve estar cadastrado no sistema.
Ações	1. Inserir número de telefone e definir senha de acesso. 2. Informar nome completo. 3. Informar número da <i>CNH</i> , categoria e indicar se exerce atividade remunerada (<i>EAR</i>). 4. Concluir o cadastro via <i>POST /auth/register-driver</i> .

Resultados	Resultado esperado: a conta deve ser criada com <i>role</i> = <i>DRIVER</i> e <i>status</i> <i>PENDING</i>, aguardando aprovação do administrador. Resultado obtido: aprovado. O endpoint <i>POST /auth/register-driver</i> criou o usuário e o perfil com <i>approvalStatus</i> = <i>PENDING</i>, impedindo o motorista de ficar <i>online</i> antes da aprovação.
-------------------	---

Tabela 50. Caso de teste de aceitação 13

Caso de Teste de Aceitação 14: Administrador deve conseguir aprovar o cadastro de um motorista	
Identificador	TA14
Pré-condição	O administrador deve estar autenticado no painel web e deve existir ao menos um motorista com <i>status</i> <i>PENDING</i> no sistema.
Ações	1. Acessar a listagem de motoristas pendentes (<i>GET /admin/drivers/pending</i>). 2. Selecionar o motorista desejado. 3. Confirmar a aprovação (<i>PATCH /admin/drivers/:id/approve</i>).
Resultados	Resultado esperado: o <i>approvalStatus</i> deve ser atualizado para <i>APPROVED</i> e o motorista passa a poder utilizar o aplicativo normalmente. Resultado obtido: aprovado. O endpoint atualizou o <i>approvalStatus</i> para <i>APPROVED</i> e registrou o <i>timestamp</i> de aprovação. O motorista conseguiu se colocar <i>online</i> após a aprovação.

Tabela 51. Caso de teste de aceitação 14

Caso de Teste de Aceitação 15: Administrador deve conseguir rejeitar o cadastro de um motorista	
Identificador	TA15

Pré-condição	O administrador deve estar autenticado no painel web e deve existir ao menos um motorista com <i>status PENDING</i> no sistema.
Ações	1. Acessar a listagem de motoristas pendentes (<i>GET /admin/drivers/pending</i>). 2. Selecionar o motorista desejado. 3. Informar o motivo da rejeição e confirmar (<i>PATCH /admin/drivers/:id/reject</i>).
Resultados	Resultado esperado: o <i>approvalStatus</i> deve ser atualizado para <i>REJECTED</i> com o motivo registrado; o motorista deve ser impedido de ficar <i>online</i> . Resultado obtido: aprovado. O endpoint atualizou o <i>approvalStatus</i> para <i>REJECTED</i> , persistiu o <i>rejectionReason</i> e manteve o motorista <i>offline</i> .

Tabela 52. Caso de teste de aceitação 15

6.2.2 Necessidade 6 — Recusa de Corrida pelo Motorista

Esta necessidade garante que o motorista possa recusar uma corrida sem penalidade, mantendo-a disponível para outros motoristas *online*.

Caso de Teste de Aceitação 16: Motorista deve conseguir recusar uma corrida	
Identificador	TA16
Pré-condição	O motorista deve estar autenticado, aprovado e <i>online</i> . Deve existir ao menos uma corrida no estado <i>PENDING_DRIVER</i> .
Ações	1. Consultar corridas disponíveis (<i>GET /driver/rides/pending</i>). 2. Selecionar uma corrida e recusá-la (<i>POST /driver/rides/:rideId/reject</i>).

Resultados	Resultado esperado: a corrida deve permanecer no estado <i>PENDING_DRIVER</i> e ser ocultada do motorista que recusou, permanecendo visível para outros. Resultado obtido: aprovado. A rejeição foi persistida na tabela <i>RideRejection</i> e a corrida deixou de aparecer nas consultas do motorista que recusou, permanecendo disponível para os demais.
-------------------	--

Tabela 53. Caso de teste de aceitação 16

6.2.3 Novos Casos de Teste de Integração

Os casos a seguir complementam os testes de integração já documentados, cobrindo os fluxos de aprovação de motoristas e mecanismos de recusa de corrida com redespachamento automático.

A Tabela 54 descreve o teste de integração que valida o mecanismo de recusa de corrida e a disponibilidade da corrida para outros motoristas.

Identificador	<i>TI10 – Recusa de corrida e redespachamento</i>
Sistemas Envolvidos	<i>Mobile App (Motorista) e MobU API</i>
Interface	<i>HTTP REST: POST /driver/rides/:rideId/reject para registrar a recusa; GET /driver/rides/pending para verificar a ocultação. Persistência via tabela RideRejection (Prisma ORM).</i>
Pré-condição	Dois motoristas autenticados e <i>online</i> ; ao menos uma corrida no estado <i>PENDING_DRIVER</i> .
Entradas	1. Motorista A recusa a corrida (<i>POST /driver/rides/:rideId/reject</i>). 2. Motorista A consulta corridas pendentes e verifica que a corrida não aparece mais. 3. Motorista B consulta corridas pendentes e verifica que a corrida ainda está disponível.

Resultados	Resultado esperado: a corrida permanece em <i>PENDING_DRIVER</i> e visível para o Motorista B. Resultado obtido: aprovado. O filtro <i>none: { driverId }</i> na consulta excluiu a corrida da visão do Motorista A, mantendo-a disponível para o Motorista B.
-------------------	--

Tabela 54. Caso de teste de integração 10

A Tabela 55 descreve o teste que valida a integração entre o painel administrativo e o aplicativo do motorista no fluxo de aprovação de cadastro.

Identificador	<i>TIII1 – Aprovação de motorista e habilitação para ficar online</i>
Sistemas Envolvidos	Painel Web/Admin, MobU API e Mobile App (Motorista)
Interface	<i>HTTP REST: GET /admin/drivers/pending e PATCH /admin/drivers/:id/approve (painel → API); POST /driver/status (motorista → API). Banco de dados via Prisma ORM.</i>
Pré-condição	Motorista cadastrado via <i>POST /auth/register-driver</i> com <i>approvalStatus = PENDING</i> ; administrador autenticado.
Entradas	1. Administrador lista motoristas pendentes (<i>GET /admin/drivers/pending</i>). 2. Administrador aprova o motorista (<i>PATCH /admin/drivers/:id/approve</i>). 3. Motorista tenta se colocar <i>online</i> (<i>POST /driver/status</i>).
Resultados	Resultado esperado: após a aprovação, o motorista deve conseguir alterar o <i>status</i> para <i>online</i> sem erro. Resultado obtido: aprovado. Com <i>status PENDING</i> a API retornou erro 400. Após aprovação pelo administrador, o mesmo endpoint retornou sucesso e o motorista ficou disponível para receber corridas.

Tabela 55. Caso de teste de integração 11

A Tabela 56 descreve o teste que valida o cálculo de estimativa de tarifa com fator de *surge*, verificando a integração entre o aplicativo, a MobU API, o modelo *PricingConfig* e a Google Maps API.

Identificador	<i>TH12 – Estimativa de tarifa com surgeMultiplier</i>
Sistemas Envolvidos	<i>Mobile App (Passageiro), MobU API, PricingConfig e Google Maps API</i>
Interface	<i>HTTP REST: POST /rides/estimate entre app e MobU API; integração interna com GoogleDirectionsService e consulta ao modelo PricingConfig via Prisma ORM.</i>
Pré-condição	<i>Passageiro autenticado; PricingConfig ativo com surgeMultiplier > 1.0; chave GOOGLE_MAPS_API_KEY configurada no ambiente.</i>
Entradas	1. Alterar o <i>surgeMultiplier</i> do <i>PricingConfig</i> ativo para 1,5. 2. Enviar <i>POST /rides/estimate</i> com coordenadas válidas. 3. Comparar o <i>estimatedFareCents</i> retornado com o valor sem <i>surge</i> para verificar a multiplicação.
Resultados	Resultado esperado: o valor estimado deve refletir a aplicação do <i>surgeMultiplier</i> sobre a tarifa base, respeitando o valor mínimo (<i>minimumFareCents</i>). Resultado obtido: aprovado. Com <i>surgeMultiplier</i> = 1.5, o <i>estimatedFareCents</i> retornado foi 50% superior ao calculado sem <i>surge</i>, confirmando a fórmula implementada em <i>RideService.calcFareCents()</i>.

Tabela 56. Caso de teste de integração 12

6.3 Testes Unitários

Esta seção documenta os testes unitários implementados para os serviços da *MobU API*. Os testes utilizam o *framework Jest* com *mocks* das dependências externas (banco de dados, *bcrypt* e serviços de terceiros), permitindo validar isoladamente o comportamento de cada serviço em cenários de falha. Todos os testes foram executados e os resultados estão registrados a seguir.

6.3.1 Necessidade — Integridade do Cadastro e Autenticação

Os testes a seguir verificam que o *AuthService* rejeita corretamente tentativas de cadastro duplicado e credenciais inválidas.

A Tabela 57 verifica que o sistema impede o cadastro de um telefone já existente.

Caso de Teste Unitário 1: Registro deve rejeitar telefone já cadastrado	
Identificador	<i>TU1</i>
Pré-condição	<i>prisma.user.findUnique</i> configurado para retornar um usuário existente (<i>mock</i>). Serviço: <i>AuthService</i> — <i>src/auth/auth.service.spec.ts</i> .
Ações	1. Chamar <i>register</i> ({ <i>phone</i> : '31999000000', <i>password</i> : 'Senha@123' }).
Resultados	Resultado esperado: deve lançar <i>BadRequestException</i> e <i>prisma.\$transaction</i> não deve ser invocado. Resultado obtido: aprovado. O serviço lançou <i>BadRequestException</i> com a mensagem 'Telefone já cadastrado' e <i>\$transaction</i> não foi chamado, confirmando que a duplicidade é verificada antes de qualquer escrita no banco.

Tabela 57. Caso de teste unitário 1

A Tabela 58 verifica que o *login* é rejeitado quando o número de telefone não está cadastrado.

Caso de Teste Unitário 2: Login deve rejeitar usuário inexistente	
Identificador	<i>TU2</i>
Pré-condição	<i>prisma.user.findUnique</i> configurado para retornar <i>null</i> (<i>mock</i>). Serviço: <i>AuthService</i> — <i>src/auth/auth.service.spec.ts</i> .
Ações	1. Chamar <i>login</i> ({ <i>phone</i> : '99999999999', <i>password</i> : 'qualquer' }).
Resultados	Resultado esperado: deve lançar <i>UnauthorizedException</i> . Resultado obtido: aprovado. O serviço retornou <i>UnauthorizedException</i> com a mensagem genérica 'Credenciais inválidas', sem revelar se o telefone está ou não cadastrado no sistema.

Tabela 58. Caso de teste unitário 2

A Tabela 59 verifica que o *login* é rejeitado quando a senha informada não corresponde à cadastrada.

Caso de Teste Unitário 3: <i>Login</i> deve rejeitar senha incorreta	
Identificador	<i>TU3</i>
Pré-condição	<i>prisma.user.findUnique</i> retorna usuário válido; <i>bcrypt.compare</i> configurado para retornar <i>false</i> (<i>mock</i>). Serviço: <i>AuthService</i> — <i>src/auth/auth.service.spec.ts</i> .
Ações	1. Chamar <i>login({ phone: '3199111111', password: 'senha-errada' })</i> .
Resultados	Resultado esperado: deve lançar <i>UnauthorizedException</i> . Resultado obtido: aprovado. Após localizar o usuário, o serviço comparou a senha via <i>bcrypt.compare</i> e, recebendo <i>false</i> , lançou <i>UnauthorizedException</i> sem expor o hash armazenado.

Tabela 59. Caso de teste unitário 3

6.3.2 Necessidade — Controle de *Status* e Aceite de Corrida pelo Motorista

Os testes a seguir verificam que o *DriverService* aplica corretamente as restrições de aprovação e disponibilidade de corrida.

A Tabela 60 verifica que um motorista não aprovado não consegue se colocar *online*.

Caso de Teste Unitário 4: Motorista com aprovação pendente não deve conseguir ficar <i>online</i>	
Identificador	<i>TU4</i>
Pré-condição	<i>prisma.driverProfile.findUnique</i> retorna perfil com <i>approvalStatus = PENDING</i> (<i>mock</i>). Serviço: <i>DriverService</i> — <i>src/driver/driver.service.spec.ts</i> .
Ações	1. Chamar <i>updateStatus('user-1', { online: true })</i> .

Resultados	Resultado esperado: deve lançar <i>BadRequestException</i> e <i>prisma.driverProfile.update</i> não deve ser chamado. Resultado obtido: aprovado. O serviço verificou o <i>approvalStatus</i> antes de qualquer atualização e lançou <i>BadRequestException</i> com a mensagem '<i>Motorista ainda não aprovado pelo administrador</i>'. Nenhuma escrita foi realizada no banco.
-------------------	--

Tabela 60. Caso de teste unitário 4

A Tabela 61 verifica que um motorista não consegue aceitar uma corrida que já foi aceita por outro.

Caso de Teste Unitário 5: Motorista não deve conseguir aceitar corrida já aceita	
Identificador	<i>TU5</i>
Pré-condição	<i>prisma.driverProfile.findUnique</i> retorna motorista <i>online</i> e aprovado; <i>prisma.ride.findUnique</i> retorna corrida com <i>status = ACCEPTED (mock)</i>. Serviço: <i>DriverService</i> — <i>src/driver/driver.service.spec.ts</i>.
Ações	1. Chamar <i>acceptRide('user-1', 'ride-1')</i> .
Resultados	Resultado esperado: deve lançar <i>BadRequestException</i>. Resultado obtido: aprovado. O serviço verificou que o <i>status</i> da corrida era <i>ACCEPTED</i> e lançou <i>BadRequestException</i> com a mensagem '<i>Corrida não está disponível</i>', impedindo a duplicidade de aceite.

Tabela 61. Caso de teste unitário 5

6.3.3 Necessidade — Resiliência da Estimativa de Corrida

Os testes a seguir verificam que o *RideService* trata corretamente entradas inválidas e falhas em dependências externas.

A Tabela 62 verifica que coordenadas inválidas são rejeitadas antes de qualquer chamada externa.

Caso de Teste Unitário 6: Estimativa deve rejeitar coordenadas inválidas	
Identificador	<i>TU6</i>

Pré-condição	Nenhum <i>mock</i> necessário; a validação ocorre antes de acessar dependências. Serviço: <i>RideService</i> — <i>src/ride/ride.service.spec.ts</i> .
Ações	1. Chamar <i>estimate</i> ({ <i>originLat</i> : NaN, <i>originLng</i> : -44.4, <i>destLat</i> : -19.9, <i>destLng</i> : -44.5 }).
Resultados	Resultado esperado: deve lançar <i>BadRequestException</i> antes de acionar o <i>GoogleDirectionsService</i> . Resultado obtido: aprovado. A validação com <i>Number.isNaN()</i> detectou a coordenada inválida e lançou <i>BadRequestException</i> imediatamente, sem realizar nenhuma chamada ao serviço de rotas.

Tabela 62. Caso de teste unitário 6

A Tabela 63 simula a indisponibilidade da *Google Maps API* e verifica que o erro é propagado corretamente.

Caso de Teste Unitário 7: Estimativa deve propagar erro quando o Google Maps API está indisponível	
Identificador	<i>TU7</i>
Pré-condição	<i>config.get</i> retorna chave de API válida; <i>GoogleDirectionsService.getRoute</i> configurado para lançar <i>Error('Directions error: REQUEST_DENIED')</i> (<i>mock</i>). Serviço: <i>RideService</i> — <i>src/ride/ride.service.spec.ts</i> .
Ações	1. Chamar <i>estimate()</i> com coordenadas válidas de Pará de Minas.
Resultados	Resultado esperado: o erro da <i>API</i> externa deve ser propagado sem modificação. Resultado obtido: aprovado. O serviço não capturou nem mascarou a exceção lançada pelo <i>GoogleDirectionsService</i> , propagando o erro com a mensagem original <i>'Directions error: REQUEST_DENIED'</i> . O comportamento atual expõe o erro diretamente ao chamador; em produção recomenda-se tratar com <i>HttpException</i> específica.

Tabela 63. Caso de teste unitário 7

A Tabela 64 verifica o comportamento quando não existe *PricingConfig* ativo no banco de dados.

Caso de Teste Unitário 8: Estimativa deve rejeitar requisição sem configuração de tarifa ativa	
Identificador	<i>TU8</i>
Pré-condição	<i>config.get</i> retorna chave válida; <i>GoogleDirectionsService.getRoute</i> retorna rota válida; <i>prisma.pricingConfig.findFirst</i> retorna null (mock). Serviço: <i>RideService</i> — <i>src/ride/ride.service.spec.ts</i> .
Ações	1. Chamar <i>estimate()</i> com coordenadas válidas.
Resultados	Resultado esperado: deve lançar <i>BadRequestException</i> indicando ausência de <i>PricingConfig</i> ativo. Resultado obtido: aprovado. O serviço localizou a ausência de configuração após obter a rota e lançou <i>BadRequestException</i> com a mensagem ' <i>Não existe PricingConfig ativo. Rode o seed ou crie um no painel.</i> '. O sistema orientou corretamente a ação de correção.

Tabela 64. Caso de teste unitário 8

A Tabela 65 simula falha de conexão com o banco de dados e verifica que o erro não é suprimido.

Caso de Teste Unitário 9: Serviços devem propagar erro de conexão com o banco de dados	
Identificador	<i>TU9</i>
Pré-condição	<i>prisma.*</i> configurado para lançar <i>Error('Connection refused')</i> (mock). Aplicado em <i>AuthService</i> e <i>RideService</i> . Arquivos: <i>auth.service.spec.ts</i> , <i>ride.service.spec.ts</i> .
Ações	1. Chamar <i>login()</i> com credenciais válidas. 2. Chamar <i>estimate()</i> com coordenadas válidas.

Resultados	Resultado esperado: o erro de conexão deve ser propagado sem suprimir nem retornar resposta parcial. Resultado obtido: aprovado. Em ambos os serviços a exceção <i>Error('Connection refused')</i> foi propagada integralmente. Nenhuma resposta parcial foi retornada. O tratamento de erros de infraestrutura permanece responsabilidade da camada de <i>middleware</i> global do <i>NestJS</i> .
-------------------	--

Tabela 65. Caso de teste unitário 9

7. Cronograma e Processo de Implementação

Nesta seção é apresentado o cronograma de desenvolvimento do sistema MobU ao longo do primeiro semestre de 2026. O cronograma está organizado por quinzenas, iniciando na primeira quinzena de fevereiro de 2026 e encerrando na segunda quinzena de junho de 2026. Em cada período são listadas as atividades a serem desenvolvidas, contemplando a construção da aplicação móvel, API, infraestrutura, testes, documentação e validações finais.

7.1 Cronograma

O cronograma do projeto MobU foi estruturado em períodos quinzenais ao longo do primeiro semestre de 2026, permitindo uma organização clara das entregas e um acompanhamento contínuo da evolução do desenvolvimento. Cada quinzena contempla um conjunto de atividades planejadas, abrangendo desde a preparação inicial dos ambientes, definição da arquitetura e modelos de dados, até a implementação progressiva dos fluxos da aplicação móvel e da API, testes, ajustes e validações.

A divisão quinzenal possibilita que o projeto seja desenvolvido de forma incremental, garantindo que funcionalidades essenciais sejam entregues gradualmente e validadas ao longo do processo. Dessa forma, cada período apresenta objetivos específicos que contribuem diretamente para a construção completa do sistema, incluindo desenvolvimento do aplicativo móvel, camada de serviços, integração com APIs externas, persistência de dados, testes de aceitação e integração, além de atividades de documentação e refinamentos finais.

A Tabela a seguir apresenta a distribuição das atividades planejadas para cada quinzena, detalhando o escopo de trabalho previsto para todo o ciclo de desenvolvimento do MobU.

Período	Atividades
01/02 – 15/02 (1ª Quinzena de Fevereiro)	● Configuração inicial dos repositórios (<i>Mobile</i> + API). ● Definição da arquitetura geral do sistema (camadas, padrões e tecnologias). ● Criação da estrutura de pastas do aplicativo móvel: ○ Estrutura base ○ Tema da aplicação ○ Instalação de bibliotecas essenciais ○ Arquivo de rotas e navegação ● Criação da estrutura inicial da API

	(Node/Express): ○ Estrutura de pastas ○ Instalação de dependências principais ○ Configuração de ambiente e variáveis (.env) ● Configuração inicial do banco de dados PostgreSQL. ● Criação do primeiro DER rascunho.
16/02 – 29/02 (2ª Quinzena de Fevereiro)	● Implementação da tela de <i>login</i> e fluxo de autenticação no <i>Mobile</i>. ● Implementação dos casos de uso: ○ UC01 Criar conta / Entrar ○ UC02 Gerenciar Perfil ● Implementação dos primeiros endpoints de autenticação da API. ● Integração <i>mobile</i> ↔ API (<i>login</i>, criação de usuário). ● Implementação da estrutura de tokens (JWT). ● Configuração do ambiente Docker da API.
01/03 – 15/03 (1ª Quinzena de Março)	● Implementação dos casos de uso relacionados às corridas: ○ UC03 Informar origem/destino ○ UC04 Estimar tarifa

	○ UC05 Solicitar corrida ● Comunicação com API externa de mapas/geo (cálculo de rota e distância). ● Implementação dos endpoints de cálculo de tarifa e solicitação de corrida. ● Modelagem final das entidades: ○ Usuário ○ Motorista ○ Veículo ○ Corrida ○ Localização
16/03 – 31/03 (2ª Quinzena de Março)	● Implementação das telas do motorista: ○ UC10 Ficar <i>online/offline</i> ○ UC11 Receber solicitações ○ UC12 Aceitar/recusar corrida ● Implementação do fluxo de despacho de corridas. ● Implementação de notificações via FCM (entrada e aceite de corrida). ● Implementação dos endpoints: ○ Despacho ○ Aceite ○ Rejeição

	● Testes de integração <i>Mobile</i> ↔ API ↔ FCM.
01/04 – 15/04 (1ª Quinzena de Abril)	● Implementação do fluxo operacional da corrida: ○ UC13 Navegar até embarque/destino ○ UC14 Marcar etapas (cheguei/iniciei/finalizei) ● Implementação de trilhas de localização e mapas no aplicativo do motorista. ● Configuração final do sistema de <i>tracking</i> em tempo real. ● Implementação de testes de aceitação para todo o fluxo até finalização da corrida.
16/04 – 30/04 (2ª Quinzena de Abril)	● Implementação do sistema de pagamento: ○ UC08 Pagar (PIX/Dinheiro) ● Geração do QR Code PIX dentro da aplicação (<i>offline</i>, sem PSP). ● Implementação dos estados de pagamento (Figura XX): ○ Aguardando ○ PIX_GERADO ○ Aguardando confirmação

	○ Confirmado / Cancelado ● Implementação dos endpoints de pagamento e confirmação do motorista. ● Implementação das telas e fluxos relacionados. ● Testes de integração <i>Mobile</i> ↔ API ↔ Pagamentos.
01/05 – 15/05 (1ª Quinzena de Maio)	● Implementação do histórico de corridas: ○ UC07 Cancelar corrida ○ UC09 Avaliar motorista ○ UC15 Ver histórico e ganhos ● Implementação dos relatórios internos (admin): ○ UC17 Ver relatórios e métricas ○ UC18 Configurar taxas e regiões ● Criação de Jobs assíncronos (limpeza, cálculos, fechamentos). ● Testes de integração ampliados.
16/05 – 31/05 (2ª Quinzena de Maio)	● Desenvolvimento do painel administrativo (versão web simples opcional). ● Finalização da documentação técnica: ○ Diagramas UML ○ DER final ○ Diagramas de componentes e

	implantação ● Finalização do módulo de notificações e eventos. ● Testes de carga e otimização de consultas. ● Correções gerais com base nos testes das sprints anteriores.
01/06 – 15/06 (1ª Quinzena de Junho)	● Configuração de pipelines CI/CD. ● Publicação do aplicativo em ambiente de testes. ● Homologação interna com cenários completos.
16/06 – 30/06 (2ª Quinzena de Junho)	● Ajustes finais após homologação. ● Últimos testes de aceitação. ● Preparação e entrega da documentação do projeto. ● Apresentação e encerramento do semestre. ● Pós-mortem do projeto.

Tabela 66. Cronograma de Implementação

8. Trabalhos Futuros

O sistema MobU, em seu estado atual, constitui um protótipo funcional robusto que implementa o ciclo completo de uma corrida — desde o cadastro de usuários até o encerramento financeiro — com uma arquitetura modular alinhada às boas práticas da indústria de software. Essa base sólida valida a viabilidade técnica de uma plataforma de mobilidade urbana de baixo custo para cidades de pequeno e médio porte e, mais do que isso, foi projetada desde o início para evoluir de forma incremental. A transição de um protótipo academicamente validado para uma operação comercial em escala segue um roteiro de maturação bem estabelecido, comum a praticamente todo sistema de

software que chega a esse estágio. Esta seção descreve esses próximos passos, organizados por área de impacto e sequência lógica de implementação, evidenciando que o projeto já se encontra posicionado para percorrê-los.

8.1 Integração com *Gateway* de Pagamento PIX em Tempo Real

O modelo de pagamento atual do MobU opera de forma manual: o passageiro realiza o PIX diretamente para a chave do motorista fora do aplicativo, e o motorista confirma o recebimento manualmente na plataforma. Embora funcional para um ambiente controlado e de baixo volume, esse modelo apresenta limitações críticas para operação em escala, como ausência de confirmação automática, risco de disputas sem evidência eletrônica e impossibilidade de reconciliação financeira automatizada.

A evolução natural é a integração com uma instituição financeira participante do Sistema de Pagamentos Brasileiro (SPB) por meio de uma API PIX certificada pelo Banco Central — como a oferecida por bancos como Sicoob, Efí (ex-Gerencianet), Banco Inter ou Mercado Pago. Essa integração permitiria a geração de cobranças PIX imediatas (Pix Cobrança — formato pixCob) diretamente associadas à corrida, com webhook de notificação de pagamento disparando automaticamente a confirmação no *backend*. Do ponto de vista da implementação, o módulo PaymentModule já existe e foi estruturado para receber essa evolução; a lógica de geração de txId e payload PIX embutida em pix.util.ts seria substituída pela chamada à API do parceiro. Seria necessário também implementar um mecanismo de split de pagamento para que a taxa da plataforma fosse creditada diretamente ao MobU, eliminando o ciclo manual de cobrança semanal atualmente implementado em DriverSettlement.

8.2 Publicação e Distribuição do Aplicativo Android

O aplicativo Android está funcional e compilável, mas ainda não foi preparado para distribuição pública. Os trabalhos necessários para publicação na Google Play Store incluem: substituição do identificador de pacote genérico br.com.seunome.mobulite por um identificador definitivo que reflita a identidade da plataforma; geração e gerenciamento de uma keystore de produção para assinatura do APK/AAB; configuração de múltiplos flavors de *build* para separar os ambientes de desenvolvimento, homologação e produção; e adequação dos metadados da loja (descrição, capturas de tela, política de privacidade e classificação etária). Adicionalmente, a política de privacidade da Play Store exige declaração explícita sobre o uso de geolocalização em background, dado que o DriverLocationForegroundService transmite coordenadas continuamente durante o turno ativo.

8.3 Evolução Arquitetural do Aplicativo Android

A arquitetura atual do aplicativo concentra lógica de negócio, gerenciamento de estado e componentes de interface em poucos arquivos de grande volume — o principal deles, *DriverScreen.kt*, com mais de 3000 linhas. Para garantir manutenibilidade à medida que novas funcionalidades forem adicionadas, é necessária uma refatoração estrutural seguindo os padrões recomendados pelo Android Architecture Guidelines: introdução de ViewModels para separar o estado da UI da lógica de chamada de API; adoção do padrão de repositório para encapsular o acesso à rede; uso de StateFlow e UiState para gerenciamento de estado reativo; e decomposição das telas monolíticas em componentes Composable menores e testáveis. Essa mudança também viabilizaria a escrita de testes unitários para a lógica de negócio do *app*, atualmente inexistentes.

8.4 Desenvolvimento de Aplicativo iOS

O mercado brasileiro de transporte por aplicativo é predominantemente *mobile*, e uma parcela relevante dos potenciais usuários utiliza dispositivos iOS. O desenvolvimento de um aplicativo nativo para iOS — utilizando SwiftUI, que compartilha os princípios declarativos do Jetpack Compose — ou de uma solução multiplataforma com Kotlin Multiplatform *Mobile* (KMM) ou Flutter, permitiria atingir esse segmento sem reescrever toda a lógica de interface. A API *backend* foi projetada de forma independente de plataforma cliente, de modo que a integração de um cliente iOS não exigiria alterações no servidor.

8.5 Verificação Automatizada de Documentos de Motoristas

O processo atual de aprovação de motoristas é inteiramente manual: o administrador analisa a foto da CNH enviada pelo motorista por meio do painel web e aprova ou rejeita o cadastro com base em inspeção visual. Para uma operação com volume significativo de cadastros, esse modelo não é escalável e está sujeito a inconsistências humanas.

A automação desse processo pode ser implementada em etapas. Em uma primeira fase, a integração com uma API de OCR (como Google Cloud Vision, AWS Textract ou Azure Form Recognizer) permitiria extrair automaticamente os dados da CNH — número, categoria, validade, nome e CPF — e compará-los com as informações preenchidas pelo motorista no cadastro, sinalizando divergências para revisão humana. Em uma fase mais avançada, a integração com bases de dados oficiais como o DETRAN ou o serviço de consulta do SENATRAN permitiria verificar a validade da habilitação em tempo real, reduzindo significativamente o risco de fraudes de documentos.

8.6 Infraestrutura de Produção e Implantação Contínua

O ambiente de desenvolvimento atual é executado localmente com PostgreSQL em Docker e *backend* NestJS direto em Node.js. Para operação em produção, é necessário construir uma infraestrutura que garanta disponibilidade, segurança e capacidade de atualização sem interrupção de serviço.

Os elementos fundamentais dessa infraestrutura incluem: containerização completa com Docker Compose ou Kubernetes para orquestração; *deploy* em provedor de nuvem com alta disponibilidade na região sul-americana (AWS, Google Cloud ou Azure); uso de um banco de dados gerenciado (RDS, Cloud SQL) com backups automáticos e réplica de leitura; configuração de um CDN para os arquivos estáticos de upload (fotos de perfil, CNH); serviço de armazenamento de objetos (S3 ou GCS) em substituição ao sistema de arquivos local, que é a solução atual para uploads; proxy reverso com Nginx ou Caddy com terminação TLS; e variáveis de ambiente gerenciadas por um cofre de segredos (AWS Secrets Manager, Google Secret Manager ou HashiCorp Vault) em vez de arquivos `.env` manuais.

A implantação contínua deve ser implementada por meio de um pipeline de CI/CD (GitHub Actions ou GitLab CI) que execute automaticamente os testes unitários, de integração e de aceitação a cada push, e só promova o *build* para produção após todas as verificações passarem.

8.7 Escalabilidade e Resiliência

O servidor WebSocket atual mantém conexões em memória por meio de um `Map<WebSocket, ClientContext>`. Esse modelo funciona corretamente para uma única instância do servidor, mas falha imediatamente em cenários de múltiplas instâncias (*horizontal scaling*), pois um evento emitido por uma instância não alcança clientes conectados em outras. A solução padrão para este problema é a introdução de um broker de mensagens — tipicamente Redis com Pub/Sub — como camada intermediária entre as instâncias, de modo que cada publicação de evento seja distribuída a todos os servidores que possuam clientes inscritos naquele `rideId`.

De forma similar, o mecanismo de *rate limiting* implementado em memória (`Map<string, HitCounter>`) não persiste entre reinicializações e não é compartilhado entre instâncias. A migração para Redis também resolve esse problema, centralizando os contadores de requisições.

O endpoint `/admin/live`, que agrega todas as corridas ativas e motoristas *online* a cada chamada, e o módulo de relatórios, que executa múltiplas queries de agregação em paralelo, são candidatos naturais para cache com TTL curto (5 a 30 segundos), reduzindo a carga no banco em horários de pico.

8.8 Monitoramento, Observabilidade e Alertas

Um sistema em operação comercial exige visibilidade sobre seu comportamento em tempo real. O MobU atualmente não possui instrumentação de métricas de aplicação, rastreamento distribuído ou alertas automatizados.

A implementação de observabilidade deve cobrir três dimensões. A primeira é o monitoramento de infraestrutura — CPU, memória, disco, conexões de banco — com ferramentas como Grafana e Prometheus. A segunda é o rastreamento de erros de aplicação com uma plataforma como Sentry, que captura exceções não tratadas com contexto completo de stack trace, versão do *app* e usuário afetado. A terceira é o monitoramento de negócio: *dashboards* com métricas operacionais como tempo médio para aceitação de corrida, taxa de cancelamento, tempo médio de corrida, receita por hora e número de motoristas ativos por período — indicadores essenciais para decisões operacionais.

8.9 Funcionalidades Operacionais Avançadas

A expansão operacional do MobU exige funcionalidades que, embora não essenciais para a validação do protótipo, são críticas para a experiência do usuário em produção.

Do lado do passageiro: histórico de endereços favoritos; estimativa de ETA com base na localização real do motorista mais próximo disponível (atualmente calculada via Google Directions, sem considerar a posição dos motoristas); possibilidade de agendar corridas com antecedência; e suporte a múltiplas paradas em uma mesma corrida.

Do lado do motorista: navegação integrada ao *app* com rotas turn-by-turn utilizando a API de Directions em modo guiado, sem depender de abertura de *app* externo; painel de metas diárias e semanais com gamificação (corridas concluídas, avaliação média, posição no ranking local); e notificações proativas sobre corridas disponíveis próximas à localização atual.

Do lado administrativo: sistema de tarifas dinâmicas (surge pricing) com configuração de multiplicadores por horário, dia da semana ou nível de demanda; geofencing por polígono customizado para regiões de operação, substituindo o modelo atual de círculo por raio; e ferramentas de conciliação financeira automática entre corridas realizadas, PIXs recebidos e repasses feitos.

8.10 Expansão para Múltiplas Cidades e Modelo Multi-Tenant

O modelo atual do MobU foi concebido para operação em uma única cidade, com um único administrador. Para que a plataforma possa crescer para múltiplos municípios — seja sob gestão

centralizada ou por meio de franqueamento a operadores locais — é necessária uma evolução arquitetural em direção a um modelo multi-tenant.

Nesse modelo, cada cidade ou operador seria uma "organização" com seu próprio conjunto de motoristas, passageiros, regiões de operação, configurações de tarifa e relatórios isolados. O *backend* precisaria de um mecanismo de isolamento de dados por tenant (por esquema de banco ou por campo discriminador), e o painel administrativo seria estendido com um nível de "super-administrador" capaz de gerenciar múltiplas organizações. Essa arquitetura também permitiria que diferentes cidades tivessem políticas tarifárias, documentos exigidos e fluxos de aprovação distintos, refletindo as particularidades regulatórias de cada município.

8.11 Testes, Qualidade e Cobertura

O projeto possui uma base de testes representativa para um trabalho acadêmico — 15 testes de aceitação e 6 de integração no *backend*, além de um arquivo dedicado à documentação de limitações conhecidas com `it.failing()`. Para suportar uma operação comercial, essa base precisa ser expandida em três frentes.

A primeira é a cobertura de testes unitários no *backend*, atualmente concentrada nos serviços mais críticos, mas ausente em módulos como `AdminService`, `ChatService` e `VerificationService`. A meta recomendada é atingir cobertura acima de 80% nas linhas de código dos serviços.

A segunda é a introdução de testes instrumentados no aplicativo Android com Espresso ou Compose UI Test, validando os principais fluxos de usuário (*login*, solicitação de corrida, pagamento) em dispositivos ou emuladores reais.

A terceira é a criação de um ambiente de testes de carga (load testing) com ferramentas como k6 ou Locust, simulando múltiplos passageiros solicitando corridas simultaneamente para identificar gargalos de performance antes que aconteçam em produção — especialmente nos endpoints de criação de corrida, que encadeiam chamadas ao Google Maps, ao banco de dados e ao WebSocket na mesma requisição.

Esses trabalhos futuros, tomados em conjunto, representam o caminho de maturidade técnica e operacional que transforma o MobU de uma prova de conceito academicamente validada em um produto viável para implantação real. A priorização natural segue a ordem de dependência: a integração de pagamento e a infraestrutura de produção são pré-requisitos para qualquer operação com usuários reais; a conformidade com a LGPD é obrigação legal não postergável; e as evoluções de escalabilidade, funcionalidade e expansão geográfica compõem a agenda de crescimento da plataforma após o lançamento inicial.

9. Post-mortem

Esta seção apresenta uma análise retrospectiva do processo de desenvolvimento do sistema MobU, relatando as experiências vivenciadas ao longo do ciclo de implementação, as principais dificuldades encontradas e os aprendizados consolidados.

9.1 Experiências Positivas

A adoção do NestJS com Prisma ORM e a containerização via Docker desde as etapas iniciais demonstraram ser escolhas acertadas: a estrutura modular do *framework* favoreceu a evolução incremental da aplicação sem refatorações estruturais, enquanto o Docker eliminou inconsistências de ambiente ao longo de todo o ciclo de desenvolvimento. A integração com a Google Maps Directions API funcionou de forma consistente para o cálculo de rotas e a precificação dinâmica, e o fluxo de aprovação de motoristas pelo administrador foi implementado de ponta a ponta com sucesso. A estruturação de testes em três camadas — aceitação, integração e unitários — com *mocks* de dependências externas agregou segurança e reprodutibilidade ao processo de validação, permitindo cobrir cenários de falha sem dependência de infraestrutura ativa.

9.2 Experiências Negativas

A sincronização entre documentação e implementação representou o principal desafio do projeto: contratos de API evoluíram durante o desenvolvimento sem atualização paralela dos artefatos, exigindo uma revisão documental abrangente ao final do ciclo. A integração com serviços externos — especialmente o Firebase Cloud Messaging e o fluxo de pagamento PIX, que envolve especificações regulatórias definidas pelo Banco Central do Brasil — apresentou complexidade superior à estimada, impactando o cronograma das atividades relacionadas. A ausência de um pipeline de integração contínua (CI/CD) configurado desde o início resultou em execução manual de testes e validações ao longo das sprints, reduzindo o benefício que a automação poderia ter proporcionado.

9.3 Lições Aprendidas

A definição formal dos contratos de API antes do início da implementação é indispensável para manter documentação e código consistentes ao longo do ciclo de desenvolvimento. Módulos com dependências externas regulatórias devem ser tratados como componentes de risco elevado no planejamento, com ciclo de análise e prototipação próprio. A escrita de testes unitários com *mocks* desde o início — e não apenas ao final — oferece retorno significativo em cobertura de cenários de falha. Por fim, a configuração de pipelines de integração contínua nas etapas iniciais do projeto reduz o esforço manual de validação e aumenta a confiança nas entregas incrementais ao longo do desenvolvimento.

10. Repositório do Trabalho

O código-fonte completo do sistema MobU, incluindo o aplicativo *Android* (*Kotlin*), a *API* (NestJS), o painel administrativo (*Next.js*) e os arquivos de configuração de infraestrutura (*Docker*), está disponível publicamente no seguinte repositório:

<https://github.com/ICEI-PUC-Minas-PPLES-TI/plf-es-2025-2-tcci-0393100-dev-joaquim-de-moura>

O repositório está organizado em três diretórios principais: *backend/api* (NestJS + Prisma ORM), *dashboard/admin* (Next.js) e *MobULite* (Android/Kotlin). As instruções de instalação, configuração de variáveis de ambiente e execução local estão descritas nos arquivos *README.md* de cada subprojeto.